KU LEUVEN

FACULTY OF SCIENCE

Maximising Edge Density in Graphs Subject to Connectivity Constraints

Erdős Problem 915, from Proof to Search

Louis Vandenbruwaene

Promotor: Dr. Stijn Cambie
Readers: Prof. Jan Goedgebeur
Prof. Karel Dekimpe
KU Leuven

Thesis presented in
fulfilment of the requirements
for the degree of Master of Science
in Mathematics

Academic year 2026–2027

Copyright information

Student paper as part of an academic education and examination. No correction was made to the paper after examination.

© Copyright by KU Leuven

Without prior written permission from both the supervisor(s) and the author(s), copying, reproducing, using, or realizing this publication or parts thereof is prohibited. For requests or information regarding the copying and/or use and/or realization of parts of this publication, please contact KU Leuven, Faculty of Science, Celestijnenlaan 200H box 2100, 3001 Leuven (Heverlee), Telephone +32 16 32 14 01.

Prior written permission from the supervisor(s) is also required for using the (original) methods, products, circuits, and programs described in this work for industrial or commercial purposes, and for submitting this publication for scientific awards or contests.

Foreword

This thesis asks how many edges a network can have when no two vertices may be joined by too many independent routes. The starting point is Erdős Problem 915, which asks this for edge-disjoint paths in simple graphs. Edges may also be directed, may be repeated, or may contain more than two vertices, and independent routes may share no edge or share no vertex. These choices give sixteen problems, and the thesis treats all of them.

Chapter 1 states the problem and its variants, proves the main results, and lists the open problems. The definitions are in its first three sections, and the notation is also listed under Abbreviations and Symbols. A reader who wants only the results can read these sections and then go to Section 1.5, where a table and a figure give what is known about each of the sixteen problems, followed by the open problems. The shorter proofs are in Chapter 1, and the longer ones are in Appendix A. That appendix has three parts, on undirected graphs, directed graphs and hypergraphs.

Chapter 2 explains how a Python program can measure connectivity and search for dense examples, using pruning and metaheuristics to make the search faster. No proof uses the program, so a reader interested only in the mathematics can skip this chapter. It needs only the definitions of Chapter 1. Appendix B prints the parts of the program that Chapter 2 leaves out, and the output of one full run.

I thank my girlfriend Sara for her unwavering support and love. I also thank my friends and family for their help throughout this journey.

Contribution Statement

I used artificial intelligence to write code, to suggest proofs, to help check them, to format LaTeX and to check spelling. Results from the literature are cited where they are used. The code is available in the repository [1].

Area	Main results
Undirected graphs	A proof of the edge bounds with the Gomory–Hu tree, and extremal graphs for every $n \geq m$.
Hypergraphs, edge version	An upper bound for every r-uniform hypergraph, attained by simple hypergraphs on a stated range.
Hypergraphs, vertex version	The exact value at $m = 2$, and an upper bound at $m = 3$ that is attained on stated ranges.
Directed multigraphs	The exact value for every n and m, and the extremal multigraphs for $n \geq \max(8, 2m)$.
Simple digraphs	The exact value at $m = 2$, for both separations.
Multigraphs, vertex version	The exact value at $m = 2$ and $m = 3$.
Directed graphs and hypergraphs	Upper bounds with the right leading term, $n^2/4$ for simple digraphs and $(m-1)n^2/(4(r-1))$ for directed hypergraphs with any number of tails, valid for every n.

Short Summary

Erdős Problem 915 asks how many edges a simple graph on n vertices can have if no pair is joined by m edge-disjoint paths [2, 3]. Writing the maximum as $\ell_m(n)$, Mader's bound is

$$\ell_m(n) \leq \left\lfloor \frac{m(n-1)}{2} \right\rfloor .$$

We prove this bound with the Gomory–Hu tree and give graphs that reach it for every $n \geq m \geq 2$.

The thesis also studies the problem in eight models. Edges may be directed or not, may be repeated or not, and may join two vertices or more (hypergraphs). In each model we can ask for edge-disjoint routes or for vertex-disjoint routes, which gives sixteen problems.

The proved results include:

- Multigraphs: at most $(m-1)(n-1)$ edges.
- r-uniform hypergraphs: at most $\lfloor (m-1)(n-1)/(r-1) \rfloor$ hyperedges.
- Hypergraphs with vertex-disjoint routes: exactly $\lfloor (n-1)/(r-1) \rfloor$ at $m = 2$, and at most $\lfloor 2(n-1)/(r-1) \rfloor$ at $m = 3$.
- Simple digraphs at $m = 2$: exactly $\max\bigl(2(n-1), \lfloor n^2/4 \rfloor\bigr)$ arcs, for both separations.
- Directed multigraphs: exactly $(m-1)\max\bigl(2(n-1), \lfloor n^2/4 \rfloor\bigr)$ arcs.
- Simple digraphs: $n^2/4 + \Theta_m(n)$ arcs for fixed $m \geq 3$.

The exact value for simple digraphs is open for $m \geq 3$.

Chapter 2 explains how a program measures connectivity with maximum flow and searches for dense examples. No result depends on this program.

Abbreviations and Symbols

Symbol	Meaning
$G = (V, E)$, $D = (V, A)$	vertex set V, with edge multiset E or arc multiset A
n, m, r	vertex count, forbidden-connectivity threshold, hyperedge size
$\lambda_G(u, v)$	maximum number of edge-disjoint u–v paths
$\kappa_G(u, v)$	maximum number of incidence-separated u–v paths
$\lambda_G^{\max}$, $\kappa_G^{\max}$	maximum local connectivity over distinct vertex pairs, ordered in directed models
$\ell_m(n)$, $L_m(n)$	simple and multigraph undirected edge maxima
$k_m(n)$, $K_m(n)$	simple and multigraph undirected vertex maxima
$\ell_m^{\mathrm{dir}}(n)$, $L_m^{\mathrm{dir}}(n)$	simple and multigraph directed arc maxima
$k_m^{\mathrm{dir}}(n)$, $K_m^{\mathrm{dir}}(n)$	simple and multigraph directed vertex maxima
$\ell_m^{(r)}(n)$, $L_m^{(r)}(n)$	simple and multi r-uniform hypergraph edge maxima
$k_m^{(r)}(n)$, $K_m^{(r)}(n)$	simple and multi r-uniform hypergraph vertex maxima
$M(n)$	$\max(2(n-1), \lfloor n^2/4 \rfloor)$, the directed arc maximum at $m = 2$
$\mu(u, v)$	edge or arc multiplicity
$O(f)$, $\Theta(f)$	at most, or between two positive constant multiples of, f for large n. A subscript such as Θ_m means the constants depend on m
$o(1)$	a quantity that tends to 0

Contents

Foreword I

Contribution Statement II

Short Summary III

Abbreviations and Symbols IV

1 The Problem and Its Results 1

1.1 Graphs and routes . . . 1

1.2 Erdős Problem 915 . . . 2

1.2.1 Proof of Mader's bound . . . 3

1.3 Variants of the problem . . . 4

1.3.1 The incidence convention . . . 6

1.4 Results and proofs . . . 7

1.4.1 Undirected graphs . . . 7

1.4.2 Directed graphs at the first threshold . . . 8

1.4.3 Undirected hypergraphs . . . 9

1.4.4 Directed bounds beyond the first threshold . . . 12

1.5 Overview and open problems . . . 14

2 Measuring and Searching by Machine 16

2.1 Modelling . . . 16

2.2 Measuring . . . 18

2.2.1 Transformations . . . 18

2.3 Checking every graph of a fixed order . . . 20

2.3.1 Pruned search . . . 21

2.3.2 Generating search . . . 22

2.4 The heuristic search . . . 23

2.4.1 Random greedy growth for hypergraphs . . . 25

2.4.2 Measuring only near the connectivity cap . . . 26

2.5 Checking the output without trusting the program . . . 26

2.6 Running the program . . . 28

A Proofs **32**

PART I. UNDIRECTED GRAPHS 33

A.1 Graphs that attain Mader's bound 33

A.1.1 The vertex problem is a block problem 35

PART II. DIRECTED GRAPHS 37

A.2 The directed constructions 37

A.3 The reachability skeleton 38

A.4 Simple digraphs beyond the first threshold 41

A.4.1 The two-step budget 41

A.5 The extremal directed multigraphs 43

PART III. HYPERGRAPHS 45

A.6 The hypergraph bounds 45

A.7 The hypergraph vertex problem 47

A.8 The multigraph incidence problems 51

A.8.1 The value is a block problem 54

A.8.2 The directed problem 56

A.9 The directed hypergraph 58

B The Program **63**

B.1 The remaining parts of the program 63

B.2 Output 76

Chapter 1

The Problem and Its Results

This thesis asks how many connections a network can have before some pair of points has too many independent routes between them. This chapter defines the problem and its variants, and states the results that are proved.

1.1 Graphs and routes

A *multigraph* G consists of a finite set $V(G)$ of *vertices* and a multiset $E(G)$ of *edges*. Each edge joins two different vertices, and edges can have identical copies. The number of such identical edges between u and v is the multiplicity $\mu(u, v)$. If $\mu(u, v) \leq 1$ for every pair, then G is a *simple graph*. The *size* of G is its number of edges, counted with multiplicity: $|E(G)| = \sum_{\{u,v\}} \mu(u, v)$. This thesis tries to maximize the size. Drawing vertices as dots and edges as lines is the standard way to visualize graphs.

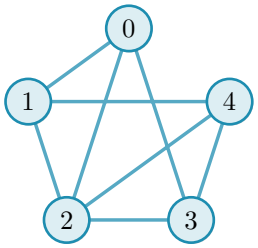


Figure 1.1. The simple graph $K_5 - \{04, 13\}$ has five vertices and eight edges, where K_n is the complete graph on n vertices.

We write $n := |V(G)|$. The *degree* of a vertex is the number of edges at it. From now on, graphs are simple unless we say otherwise. A *path* from u to v is a sequence $v_0, e_1, v_1, \ldots, e_q, v_q$ of distinct vertices v_i and edges e_i, where $v_0 = u$, $v_q = v$ and e_i joins v_{i-1} and v_i. In a multigraph, two parallel edges give two different paths of length one. In a simple graph we can write a path as a list of vertices. A graph is *connected* if every two vertices are joined by a path. A *tree* is a connected graph in which every two vertices are joined by exactly one path. A tree on n vertices has $n - 1$ edges [4]. A *forest* is a graph without cycles. A *spanning tree* of a graph G is a tree on all vertices of G that uses only edges of G. We also call a path a *route*. Two u–v paths are *edge-disjoint* if they have no edge in common. The *local edge-connectivity* $\lambda_G(u, v)$ is the largest number of pairwise edge-disjoint u–v paths. Its largest value over all pairs is

$$\lambda_G^{\max} = \max_{u \neq v} \lambda_G(u, v).$$

If G has fewer than two vertices, we set $\lambda_G^{\max} = 0$.

Theorem 1.1 (Menger [5, 6])**.** *Let u and v be distinct vertices of a multigraph G. The largest number of pairwise edge-disjoint u–v paths equals the smallest number of edges whose removal leaves no u–v path.*

Figure 1.2 shows an example.

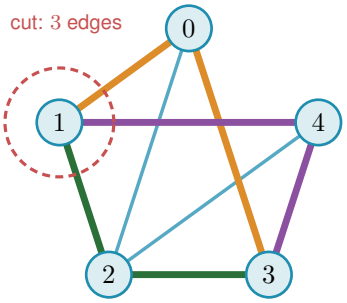


Figure 1.2. Menger's theorem on Figure 1.1: three edge-disjoint 1–3 routes (coloured) meet the minimum three-edge cut around vertex 1, so $\lambda(1,3) = 3$.

1.2 Erdős Problem 915

Fix an integer $m \geq 2$. The condition $\lambda_G^{\max} \leq m-1$ means that no two vertices are joined by m edge-disjoint paths. By Menger's theorem, it also means that any two vertices can be separated by removing at most $m-1$ edges. We ask how many edges a graph can have under this condition.

Question 1.2 (Erdős Problem 915 [3]). How many edges can a graph on n vertices have if no two of its vertices are joined by m disjoint paths?

The database [3] notes that the original statement leaves open whether "disjoint" means that the paths share no edge, or that they share no vertex except their two ends. This thesis takes edge-disjoint paths as the main question and treats vertex-disjoint paths as a variant (Section 1.3).

We write

$$\ell_m(n) = \max\{\, |E(G)| : |V(G)| = n,\ \lambda_G^{\max} \leq m-1 \,\}$$

for the answer. A graph with $\lambda_G^{\max} \leq m-1$ is called *feasible*. Mader proved the following upper bound. Here $\lfloor x \rfloor$ is x rounded down and $\lceil x \rceil$ is x rounded up.

Theorem 1.3 (Mader [7])**.** *For all $n \geq 2$ and $m \geq 2$,*

$$\ell_m(n) \leq \left\lfloor \frac{m(n-1)}{2} \right\rfloor .$$

Bollobás and Erdős [2] found graphs that attain this bound when $m-1$ divides $n-1$: copies of K_m that share one vertex. For $n \geq m$ the bound is always attained (Theorem A.3). For $2 \leq n < m$ the complete graph K_n satisfies the condition, so $\ell_m(n) = \binom{n}{2}$. This is smaller than $\lfloor m(n-1)/2 \rfloor$, except when $(n,m) = (2,3)$.

We prove Theorem 1.3 in a different way from Mader, with the Gomory–Hu tree. It stores the $\binom{n}{2}$ values $\lambda_G(u,v)$ in only $n-1$ weights.

Theorem 1.4 (Gomory–Hu [8]). *Let G be a multigraph on the vertex set V. There is a tree T on V, with a weight $c(t)$ on each tree edge t, such that:*

(1) Removing t from T splits V into two parts S and $V \setminus S$, and $c(t)$ is the number of edges of G with one end in S and one end in $V \setminus S$.

(2) For distinct u, v, the value $\lambda_G(u,v)$ is the smallest weight on the u–v path of T.

Such a tree is a *Gomory–Hu tree* of G. Its edges need not be edges of G. If $t = xy$ is a tree edge, then the x–y path of T is t itself, so $c(t) = \lambda_G(x,y)$ by (2). This thesis uses the theorem only for undirected graphs.

1.2.1 Proof of Mader's bound

Let T be a Gomory–Hu tree of G. For an edge e of G, let $\mathrm{dist}_T(e)$ be the number of tree edges on the path in T between the ends of e (Figure 1.3). An edge $e = uv$ of G *crosses* a tree edge t if u and v lie in different parts when t is removed from T. This happens exactly when t lies on the u–v path of T. So e crosses $\mathrm{dist}_T(e)$ tree edges.

The proof has three steps.

(1) The tree weights add up to at most $(m-1)(n-1)$.

(2) The same sum counts every edge of G once for each tree edge it crosses (Lemma 1.6).

(3) In a simple graph, at most $n-1$ edges cross only one tree edge, and every other edge crosses at least two (Lemma 1.5).

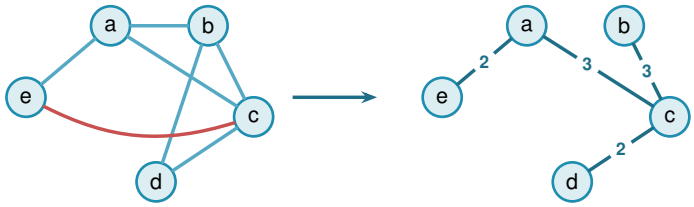


Figure 1.3. A graph G and a Gomory–Hu tree T on the same vertices. Each edge e of G crosses $\mathrm{dist}_T(e)$ tree edges. In a simple graph, at most $n-1$ edges have $\mathrm{dist}_T(e) = 1$.

Lemma 1.5 (Sum of tree distances). *For a simple graph G,* $\displaystyle\sum_{e \in E(G)} \mathrm{dist}_T(e) \;\geq\; 2|E(G)| - (n-1).$

Proof. Let E_1 be the set of edges with $\mathrm{dist}_T(e) = 1$. Such an edge joins two vertices that are adjacent in T. The tree T has $n-1$ edges, and since G is simple, each of them gives at most one edge of E_1. So $|E_1| \leq n-1$. Every other edge has $\mathrm{dist}_T(e) \geq 2$. So

$$\sum_{e \in E(G)} \mathrm{dist}_T(e) \;\geq\; |E_1| + 2\big(|E(G)| - |E_1|\big) \;=\; 2|E(G)| - |E_1| \;\geq\; 2|E(G)| - (n-1). \qquad \square$$

Lemma 1.6 (Double-counting identity)**.** *Let G be a multigraph and T a Gomory–Hu tree of G with weights c. Then*

$$\sum_{t \in E(T)} c(t) \;=\; \sum_{e \in E(G)} \operatorname{dist}_T(e).$$

Parallel edges are counted separately in the right-hand sum.

Proof. Count the pairs (t, e) where t is a tree edge, e is an edge of G, and e crosses t. For a fixed t, exactly $c(t)$ edges cross t, by property (1) of the Gomory–Hu theorem. For a fixed e, exactly $\operatorname{dist}_T(e)$ tree edges are crossed by e. So both sums count the same pairs.

□

Proof of Theorem 1.3, upper bound. Let G be a simple graph on n vertices with $\lambda_G^{\max} \le m-1$, and let T be a Gomory–Hu tree of G. Each tree edge $t = xy$ has weight $c(t) = \lambda_G(x, y) \le m-1$. There are $n-1$ tree edges, so $\sum_t c(t) \le (m-1)(n-1)$. With Lemmas 1.5 and 1.6 this gives

$$2|E(G)| - (n-1) \;\le\; \sum_{e \in E(G)} \operatorname{dist}_T(e) \;=\; \sum_{t \in E(T)} c(t) \;\le\; (m-1)(n-1).$$

So $2|E(G)| \le m(n-1)$. Since $|E(G)|$ is an integer, $|E(G)| \le \left\lfloor \frac{m(n-1)}{2} \right\rfloor$. □

The factor $\frac{1}{2}$ comes from step (3). In a multigraph step (3) fails, and the bound is $(m-1)(n-1)$ (Theorem 1.8).

1.3 Variants of the problem

We change four assumptions. The first three give eight graph models. In each model we can ask for edge-disjoint or for vertex-disjoint routes (Section 1.3.1).

The first assumption is the size of an edge. A *hyperedge* is a set of $r \ge 2$ vertices. In an *r-uniform hypergraph* every hyperedge has exactly r vertices. A *Berge path* is a sequence of distinct vertices and distinct hyperedges, alternating, in which each hyperedge contains the vertex before it and the vertex after it. For $r = 2$ it is an ordinary path. The size of a hypergraph is its number of hyperedges, counted with multiplicity.

The second assumption is repetition. A *simple* graph or hypergraph has each edge at most once. A *multigraph* or *multihypergraph* may repeat edges.

The third assumption is direction. A directed edge, or *arc*, is an ordered pair of vertices. A directed hyperedge splits its r vertices into a non-empty set of *tails* and a non-empty set of *heads*. A directed Berge route enters a hyperedge at a tail and leaves it at a head. The allowed numbers of tails are a parameter of the directed hypergraph models. In the *forward* model every hyperedge has one tail and $r-1$ heads, in the *backward* model it has $r-1$ tails and one head, and the *general* model allows every split. For $r = 2$ the three coincide, and a hyperedge is an arc. Unless a statement names one of them, the directed hypergraph models are forward. In the figures, one colour is one hyperedge, not a path of ordinary edges. Figure 1.4 shows the eight models.

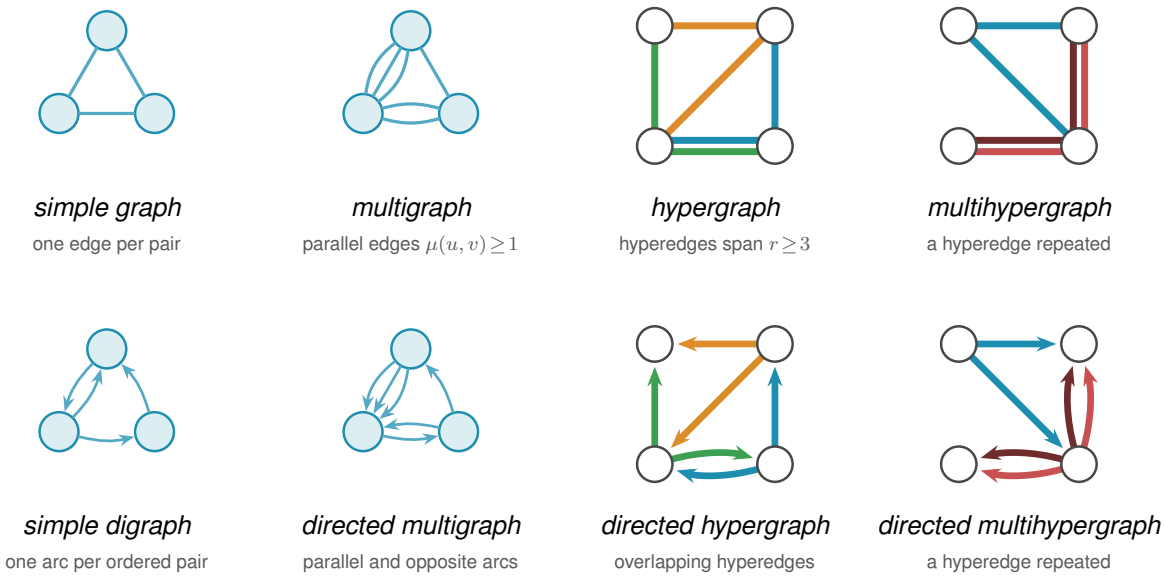


Figure 1.4. The eight models side by side, with the four undirected on top and the four directed below. The simple graph is a triangle, the multigraph has parallel edges, the hypergraph has three hyperedges, and the multihypergraph has one repeated hyperedge. The directed versions have arcs instead of edges, with the same pattern of repetition and hyperedges. In the two multihypergraph panels the two shades of red are the two copies of one hyperedge. Each directed hyperedge drawn has one tail.

Digraphs have no Gomory–Hu tree, but Menger's theorem (Theorem 1.1) also holds for directed multigraphs, with directed paths and arcs. In a digraph, the *out-degree* $d^+(v)$ is the number of arcs that leave v. The vertices these arcs point to are the *out-neighbours* of v. The *in-degree* $d^-(v)$ is the number of arcs that enter v, and they come from the *in-neighbours* of v. The *total degree* is $d(v) = d^+(v) + d^-(v)$. Figure 1.5 shows three arc-disjoint routes from u to v. Removing three arcs cuts all of them, as Menger's theorem predicts. The out-degree of u and the in-degree of v also bound the number of routes:

Lemma 1.7 (Degree bound)**.** *Every pair u, v of distinct vertices of a graph G satisfies $\lambda_G(u,v) \leq \min(\deg u, \deg v)$, and every ordered pair of a digraph D satisfies $\lambda_D(u,v) \leq \min(d^+(u), d^-(v))$.*

Proof. Every edge-disjoint u–v path uses a distinct edge at u, so there are at most $\deg u$ of them, and likewise at most $\deg v$. In the directed case each arc-disjoint path leaves u by a distinct out-arc and enters v by a distinct in-arc. □

The fourth assumption is *separation*. Two u–v paths are *internally vertex-disjoint* if they have no vertex in common except u and v. The largest number of such paths is the *local vertex-connectivity* $\kappa_G(u,v)$. Its largest value over all pairs is $\kappa_G^{\max}$. Internally vertex-disjoint paths are also edge-disjoint, so $\kappa_G(u,v) \leq \lambda_G(u,v)$. Section 1.3.1 extends this to multigraphs and hypergraphs. Figure 1.6 shows that the vertex and edge-disjointness can differ.

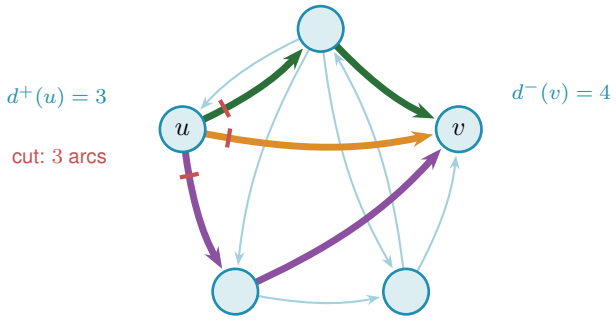


Figure 1.5. Three arc-disjoint $u \to v$ routes (coloured) and the corresponding three-arc directed cut (red) in a *simple* digraph. Thus $\lambda(u, v) = 3$, meeting the degree bound $\min(d^+(u), d^-(v)) = 3$.

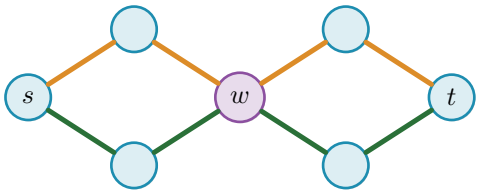


Figure 1.6. The separation axis: two edge-disjoint s–t routes both pass through w, so $\lambda(s, t) = 2$ but $\kappa(s, t) = 1$.

1.3.1 The incidence convention

For multigraphs and hypergraphs we must say when two routes count as vertex-disjoint. This thesis uses the following convention. For simple graphs it gives the usual notion.

The *incidence graph* $I(\mathcal{H})$ has a node for each vertex and a node for each edge copy. An edge copy is joined to the vertices it contains. A route from u to v is a path in $I(\mathcal{H})$. Two routes are *separated* if they have no inner node in common. In the directed case we use the directed incidence graph: each tail has an arc to the edge copy, and the edge copy has an arc to each head. For example, an arc $u \to v$ becomes $u \to e \to v$.

The inner nodes are vertices and edge copies, so separated routes share no inner vertex and no edge copy. Sharing no vertex is not enough for hypergraphs, because two Berge routes can use the same hyperedge without meeting in a vertex. Since separated routes also share no edge copy, $\kappa \leq \lambda$ holds in every model.

In a multigraph, q parallel copies of uv give q separated routes. So the vertex problems for multigraphs, $K_m(n)$ and $K_m^{\mathrm{dir}}(n)$, differ from the simple ones. In a feasible multigraph no multiplicity is larger than $m - 1$, since m parallel copies already give m separated routes.

In every model, a graph is *feasible* if no two vertices are joined by m disjoint routes of the kind considered. Table 1.1 lists the notation for all sixteen problems. Figure 1.10 shows what is known about each of them.

Menger's theorem, the Gomory–Hu tree and Mader's bound are classical. So are the results of Bártfai, Bollobás, Leonard, Sørensen and Thomassen. The main new results are: the directed values at $m = 2$ (Theorems 1.11 and 1.12), the hypergraph edge bound (Theorem 1.13), the exact value for directed multigraphs (Theorem 1.18), the hypergraph vertex problem at $m = 2$ and $m = 3$ (Theorems 1.15 and 1.16), and the quadratic bounds for directed graphs and hypergraphs

Table 1.1. Notation for the extremal functions. The third column uses the incidence convention defined above, in every model.

Model	edge-disjoint routes	vertex-disjoint routes
Simple graph	$\ell_m(n)$	$k_m(n)$
Multigraph	$L_m(n)$	$K_m(n)$
r-uniform simple hypergraph	$\ell_m^{(r)}(n)$	$k_m^{(r)}(n)$
r-uniform multihypergraph	$L_m^{(r)}(n)$	$K_m^{(r)}(n)$
Simple digraph	$\ell_m^{\mathrm{dir}}(n)$	$k_m^{\mathrm{dir}}(n)$
Directed multigraph	$L_m^{\mathrm{dir}}(n)$	$K_m^{\mathrm{dir}}(n)$
Directed r-uniform simple hypergraph	$\ell_m^{\mathrm{dir}\,(r)}(n)$	$k_m^{\mathrm{dir}\,(r)}(n)$
Directed r-uniform multihypergraph	$L_m^{\mathrm{dir}\,(r)}(n)$	$K_m^{\mathrm{dir}\,(r)}(n)$

(Theorems 1.17, 1.22 and A.49).

1.4 Results and proofs

1.4.1 Undirected graphs

Theorem 1.8. *For $n \geq 1$ and $m \geq 2$, undirected multigraphs satisfy $L_m(n) = (m-1)(n-1)$.*

Proof of Theorem 1.8. For the lower bound, take any tree on the n vertices and give every edge multiplicity $m-1$. This multigraph has $(m-1)(n-1)$ edges. Two vertices u, v are joined by exactly one path in the tree. Removing the $m-1$ copies of one edge of this path separates u from v. So $\lambda(u,v) \leq m-1$ for every pair.

For the upper bound, let $\lambda_G^{\max} \leq m-1$ and let T be a Gomory–Hu tree of G. Every edge of G crosses at least one tree edge. So by Lemma 1.6, $|E(G)| \leq \sum_e \mathrm{dist}_T(e) = \sum_t c(t) \leq (m-1)(n-1)$. □

In simple graphs, the edge and vertex problems have the same answer for $m \leq 4$.

Theorem 1.9 (Bártfai [9], Bollobás [10], Leonard [11])**.** *For all $n \geq m$ and $2 \leq m \leq 4$,*

$$k_m(n) = \ell_m(n) = \left\lfloor \frac{m(n-1)}{2} \right\rfloor.$$

Since $\kappa \leq \lambda$, every graph with $\lambda^{\max} \leq m-1$ also has $\kappa^{\max} \leq m-1$. So $k_m(n) \geq \ell_m(n)$ (Figure 1.7). Sørensen and Thomassen showed that the two differ for $m = 5$.

Theorem 1.10 (Sørensen–Thomassen [12])**.** *For simple graphs,*

$$k_5(n) = \left\lfloor \frac{5(n-1)}{2} \right\rfloor = \ell_5(n) \quad (5 \leq n \leq 13), \qquad k_5(n) = \left\lfloor \frac{8n}{3} \right\rfloor - 4 \quad (n \geq 14).$$

So $k_5(n) > \ell_5(n)$ for every $n \geq 14$. Sørensen and Thomassen count the least number of edges that forces m disjoint paths. This is one more than our maximum (Remark A.5). They give

one formula with exceptions at $n = 7$ and $n = 12$. Splitting the statement at $n = 14$, as above, removes these exceptions.

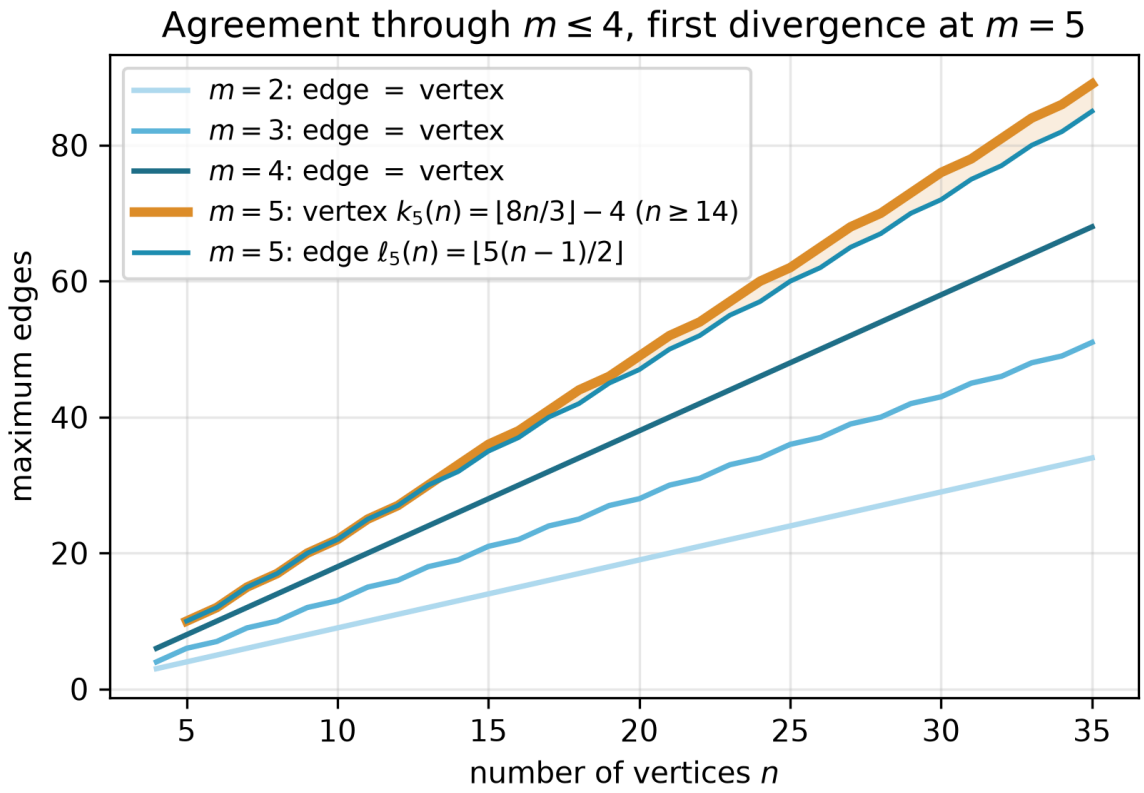


Figure 1.7. The edge and vertex extremal functions for $m = 2, 3, 4$ (blue shades, single curve per m since they agree) and $m = 5$ (thin blue for edge, thick orange for vertex). The vertex function diverges from the edge function at $n = 14$ and grows faster thereafter.

1.4.2 Directed graphs at the first threshold

Theorem 1.11 (Directed arc value at $m = 2$)**.** *For simple digraphs,*

$$\ell_2^{\mathrm{dir}}(n) = \max\left(2(n-1),\ \left\lfloor \frac{n^2}{4} \right\rfloor\right).$$

Two constructions attain the two terms (Figure 1.8). The *bidirected star* joins one vertex to every other vertex by an arc in each direction, so it has $2(n-1)$ arcs. The one-way complete bipartite digraph has all arcs from one half of the vertices to the other half, so it has $\lfloor n^2/4 \rfloor$ arcs. In both, every ordered pair is joined by at most one directed path. Section A.2 gives the details.

Theorem 1.12 (Directed vertex value at $m = 2$)**.** *For simple digraphs,* $k_2^{\mathrm{dir}}(n) = \ell_2^{\mathrm{dir}}(n)$.

The star has more arcs for $n < 7$, and the bipartite digraph has more for $n > 7$. So the directed value grows quadratically in n, while the undirected value grows linearly. In the bipartite digraph all arcs point the same way, so a route crosses between the parts only once and never comes back. An undirected graph with $\lambda^{\max} \le 1$ has no cycle, so it has at most $n - 1$ edges.

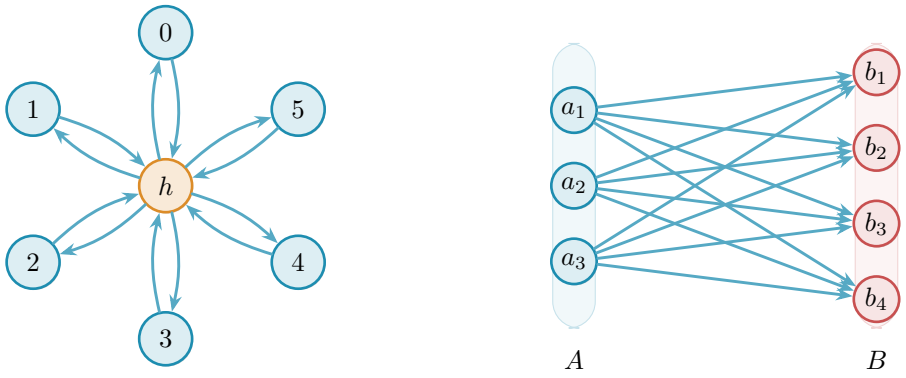


Figure 1.8. The two branches tie at $n = 7$, where each holds 12 arcs at $\lambda^{\max} = 1$. Left: the bidirected star, a hub joined to six spokes both ways, $2(n-1) = 12$. Right: the one-directional complete bipartite digraph $B_{3,4}$, every arc from A to B, $\lfloor n^2/4 \rfloor = 12$.

For the upper bound we use a *reachability skeleton*: a subdigraph, with at most one copy of each arc, in which u can reach v exactly when u can reach v in the whole digraph. Lemma A.12 in the appendix shows that there is always a skeleton with at most $M(n) = \max(2(n-1), \lfloor n^2/4 \rfloor)$ arcs.

Proof of Theorem 1.12. Write $M(n) = \max(2(n-1), \lfloor n^2/4 \rfloor)$.

Lower bound. The bidirected star (Construction A.8 at $m = 2$) and the one-way bipartite digraph (Construction A.9) have $\lambda^{\max} = 1$. Since $\kappa \le \lambda$, they also have $\kappa^{\max} \le 1$. So $\ell_2^{\mathrm{dir}}(n) \ge M(n)$ and $k_2^{\mathrm{dir}}(n) \ge M(n)$.

Upper bound. Let D be a simple digraph on $n \ge 2$ vertices with $\kappa_D^{\max} \le 1$. By Lemma A.12, D has a reachability skeleton R with $|A(R)| \le M(n)$.

We claim that R contains every arc of D. Suppose an arc (u, v) of D is not in R. Since u reaches v in D, it also reaches v in R, along a directed path P. The path P is not the arc (u, v), so it has length at least two. The arc $u \to v$ has no inner vertex, so P and the arc are two internally vertex-disjoint routes. So $\kappa_D(u, v) \ge 2$, a contradiction.

So $|A(D)| = |A(R)| \le M(n)$. With the lower bound, $k_2^{\mathrm{dir}}(n) = M(n)$. □

Proof of Theorem 1.11. The two constructions above give $\ell_2^{\mathrm{dir}}(n) \ge M(n)$. Since $\kappa \le \lambda$, every arc-feasible digraph is vertex-feasible, so Theorem 1.12 bounds it. So

$$M(n) \;\le\; \ell_2^{\mathrm{dir}}(n) \;\le\; k_2^{\mathrm{dir}}(n) \;\le\; M(n),$$

so all three are equal. □

1.4.3 Undirected hypergraphs

Theorem 1.13 (Hypergraph edge bound)**.** *For $n \ge r \ge 2$ and $m \ge 2$, an r-uniform multihypergraph on n vertices with $\lambda^{\max} \le m - 1$ has size at most $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$. There are simple hypergraphs attaining the bound when $m - 1 \le \binom{n-2}{r-2}$, and multihypergraphs attaining it when $(r-1) \mid n-1$.*

Lemma 1.14 (Cut induction for multihypergraphs)**.** *Let $\mathcal{H}$ be a multihypergraph on $n \geq 1$ vertices, not assumed uniform, with $\lambda_{\mathcal{H}}^{\max} \leq k$. Then, the sum running over hyperedge copies,*

$$\sum_e \big(|e| - 1\big) \;\leq\; k\,(n-1).$$

Proof. Induction on n. At $n = 1$ every hyperedge is a single vertex and both sides are 0.

Let $n \geq 2$ and fix two vertices $u \neq v$. By Theorem A.23, a set F of at most k hyperedge copies separates them. Let S be the set of vertices that u can reach in $\mathcal{H} - F$, and let $\bar{S} = V \setminus S$. Then $u \in S$ and $v \in \bar{S}$, and every copy that meets both sides lies in F. So at most k copies meet both sides.

Form two *trace* multihypergraphs $\mathcal{H}[S]$ and $\mathcal{H}[\bar{S}]$: replace every copy e by $e \cap S$, or by $e \cap \bar{S}$, and drop the sets with fewer than two vertices. A Berge route in a trace is also a Berge route in $\mathcal{H}$, using the original copies. So a family of disjoint routes in a trace gives one of the same size in $\mathcal{H}$. So both traces have $\lambda^{\max} \leq k$, and the induction hypothesis applies to both.

Now count. A copy inside one side contributes $|e| - 1$ to that side. A copy that meets both sides satisfies

$$|e| - 1 \;=\; \big(|e \cap S| - 1\big) \;+\; \big(|e \cap \bar{S}| - 1\big) \;+\; 1,$$

where each bracket is the contribution to one trace (zero if that part was dropped). Summing over all copies gives

$$\sum_e \big(|e|-1\big) \;\leq\; k\big(|S|-1\big) + k\big(|\bar{S}|-1\big) + \big|\{\text{crossing copies}\}\big| \;\leq\; k(n-2)+k \;=\; k(n-1). \;\square$$

A *hypertree* is a hypergraph whose incidence graph (Section 1.3.1) is a tree.

Proof of Theorem 1.13. Upper bound. Every hyperedge has $|e| = r$, so Lemma 1.14 with $k = m - 1$ gives $(r-1)|E(\mathcal{H})| \leq (m-1)(n-1)$. Since $|E(\mathcal{H})|$ is an integer, we can round down.

Lower bound. Suppose $r - 1$ divides $n - 1$. Take a hub h and split the other $n - 1$ vertices into groups $G_1, G_2, \ldots$ of size $r - 1$. The hyperedges are the sets $\{h\} \cup G_i$. Its incidence graph is a tree, so we call it a *star hypertree*. Give each hyperedge multiplicity $m - 1$. Every vertex other than h lies in exactly $m - 1$ hyperedge copies. Every pair contains such a vertex, and every route from or to it uses one of these copies. So every pair has $\lambda \leq m - 1$. There are $(m-1)(n-1)/(r-1)$ hyperedges.

If $m - 1 \leq \binom{n-2}{r-2}$ and $r \geq 3$, Theorem A.24 below attains the bound with a simple hypergraph. If $r = 2$, this condition means $m = 2$. Then the bound is $n - 1$, and a tree attains it. $\square$

The star hypertree of Figure 1.9 attains the bound at $m = 2$. It has a hub joined to disjoint groups of $r-1$ vertices. For $r \geq 3$, simple hypergraphs attain the bound whenever $m-1 \leq \binom{n-2}{r-2}$ (Theorem A.24). So for $r \geq 3$, unlike for graphs, repeated edges often do not help. The condition says that there are enough different hyperedges. When it fails, the simple value can be smaller. For example, at $n = r = m = 3$ there is only one possible hyperedge. A simple hypergraph has

one hyperedge, while a multihypergraph can have two copies of it (Remark A.25).

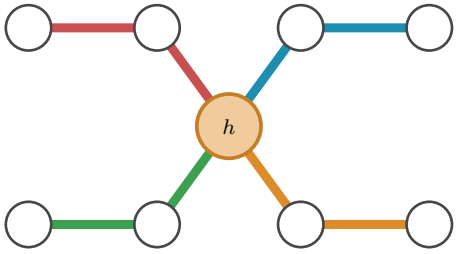


Figure 1.9. The $r = 3$, $n = 9$ star hypertree: four hyperedges meet only at the hub h, attaining $\lfloor (n-1)/(r-1) \rfloor = 4$ with $\lambda^{\max} = 1$.

Theorem 1.15 (Hypergraph vertex value at $m = 2$)**.** *For $r \geq 2$ and $n \geq 1$, the maximum number of hyperedges of an r-uniform multihypergraph on n vertices with $\kappa^{\max} \leq 1$ is*

$$K_2^{(r)}(n) \;=\; \left\lfloor \frac{n-1}{r-1} \right\rfloor,$$

attained by the star hypertree of Figure 1.9, which is simple. In particular $K_2^{(r)}(n) = k_2^{(r)}(n) = \ell_2^{(r)}(n)$: the vertex and edge problems agree at $m = 2$ for every uniformity, and repeated hyperedges never help.

Proof of Theorem 1.15. Let q be the number of hyperedges. We claim that $\kappa_{\mathcal{H}}^{\max} \leq 1$ exactly when $I(\mathcal{H})$ is a forest. Suppose $I(\mathcal{H})$ has a cycle $v_1, e_1, v_2, e_2, \ldots, v_\ell, e_\ell, v_1$, with $\ell \geq 2$ and all v_i and e_i distinct. Then v_1, e_1, v_2 and $v_1, e_\ell, v_\ell, \ldots, e_2, v_2$ are two Berge routes from v_1 to v_2 that share no hyperedge and no inner vertex. So $\kappa(v_1, v_2) \geq 2$. Conversely, two such routes between u and v together form a cycle in $I(\mathcal{H})$. This proves the claim.

A forest on $n+q$ nodes has at most $n+q-1$ edges, and $I(\mathcal{H})$ has rq edges. So $rq \leq n+q-1$, that is, $q \leq (n-1)/(r-1)$. The star hypertree has $\lfloor (n-1)/(r-1) \rfloor$ hyperedges. Its hyperedges meet only at the hub, and every other vertex lies in at most one hyperedge, so its incidence graph is a forest. □

Theorem 1.16 (Hypergraph vertex value at $m = 3$)**.** *For $r \geq 2$ and $n \geq 1$, every r-uniform multihypergraph on n vertices with $\kappa^{\max} \leq 2$ has at most $\left\lfloor \frac{2(n-1)}{r-1} \right\rfloor$ hyperedges. For $2 < r < n$, which is exactly the range where $2 \leq \binom{n-2}{r-2}$, the bound is attained by a simple hypergraph, so*

$$k_3^{(r)}(n) \;=\; \left\lfloor \frac{2(n-1)}{r-1} \right\rfloor \;=\; \ell_3^{(r)}(n).$$

When $(r-1) \mid (n-1)$, a multihypergraph attains it, so then $K_3^{(r)}(n) = \lfloor 2(n-1)/(r-1) \rfloor$ (Proposition A.28).

Both proofs use the incidence graph (Section A.7). At $m = 3$ the proof bounds the cycle rank of the incidence graph, its number of edges minus its number of nodes plus one. We have no such bound for $m \geq 4$.

For $r = 2$ these results are about multigraphs. They give $K_2(n) = n - 1$, attained by trees, and $K_3(n) = 2(n-1)$, attained by a tree with every edge doubled (Corollary A.34). For $m = 5$ the vertex and edge values differ. On four vertices, join two of them by 2 parallel edges, and join each of the other two to both of them by 3 parallel edges. This multigraph has 14 edges and $\kappa^{\max} = 4$ (Section A.8), while $L_5(4) = 12$. For $m = 4$ this is open. Section A.8 discusses larger m.

1.4.4 Directed bounds beyond the first threshold

Theorem 1.17 (The quadratic bound with a linear error)**.** *For all $m \geq 2$ and $n \geq 2$, every simple digraph D on n vertices with $\lambda_D^{\max} \leq m - 1$, or with $\kappa_D^{\max} \leq m - 1$, satisfies*

$$|A(D)| \;\leq\; \left\lfloor \frac{n^2}{4} \right\rfloor \;+\; 4(m-1)(n-1),$$

so for each fixed m,

$$\ell_m^{\mathrm{dir}}(n) \;=\; \frac{n^2}{4} \;+\; \Theta_m(n) \quad \textit{and} \quad k_m^{\mathrm{dir}}(n) \;=\; \frac{n^2}{4} \;+\; \Theta_m(n) \qquad (m \geq 3).$$

Appendix A gives the proof. It counts routes of length two. For an ordered pair (u, v), the routes $u \to x \to v$ with different midpoints x, together with the arc $u \to v$, are disjoint. So a feasible digraph has at most $m - 1$ of them. Summing over all pairs bounds $\sum_x d^+(x)\, d^-(x)$. The proof is by induction on n. If some vertex has total degree below $n/2$, we delete it and use the bound on $n - 1$ vertices. Otherwise every total degree is at least $n/2$, and the sum above makes $\sum_x \min(d^+(x), d^-(x))$ at most linear in n. At every vertex we then delete its in-arcs or its out-arcs, whichever are fewer. These deleted arcs give the linear term. What remains has all arcs from one part to the other, so it has at most $\lfloor n^2/4 \rfloor$ arcs. The routes of length two are also internally vertex-disjoint, so the same proof works for vertex separation.

Theorem 1.18 (Directed multigraph value, every n and m)**.** *For all $n \geq 2$ and $m \geq 2$,*

$$L_m^{\mathrm{dir}}(n) \;=\; (m-1)\, M(n), \qquad M(n) := \max\Big(2(n-1),\; \left\lfloor \frac{n^2}{4} \right\rfloor\Big).$$

The lower bound uses thickened trees and thickened one-way bipartite digraphs.

If we multiply every multiplicity by the same integer, every cut and so every connectivity is multiplied by that integer.

Lemma 1.19 (Thickening)**.** *Let D_0 be a directed multigraph and $k \geq 1$ an integer. Write $k \cdot D_0$ for the multigraph obtained by multiplying every multiplicity $\mu(u, v)$ by k. Then $\lambda_{k \cdot D_0}(u, v) = k \cdot \lambda_{D_0}(u, v)$ for every ordered pair $u \neq v$, and $|A(k \cdot D_0)| = k\, |A(D_0)|$.*

Proof. By Menger's theorem, $\lambda_{D_0}(u, v)$ is the smallest number of arcs, counted with multiplicity, in a u–v cut. Multiplying every multiplicity by k multiplies the size of every cut by k. So it multiplies the smallest cut by k too. The number of arcs is also multiplied by k. □

Construction 1.20 (Thickened tree and thickened bipartite digraph)**.** Let $m \geq 2$. (a) Take any tree on the n vertices and put $m-1$ arcs in each direction on every tree edge. With one arc in each direction this is the bidirected tree, with $\lambda^{\max} = 1$. So by Lemma 1.19 the thickened tree has $\lambda^{\max} = m-1$ and $2(m-1)(n-1)$ arcs. (b) Take the one-way bipartite digraph of Construction A.9 and repeat every arc $m-1$ times. By the same lemma it has $\lambda^{\max} = m-1$ and $(m-1)\lfloor n^2/4 \rfloor$ arcs.

The larger of the two gives $L_m^{\mathrm{dir}}(n) \geq (m-1)M(n)$.

For the upper bound we want to delete a part of D that lowers every positive connectivity, and whose size we can bound. Deleting one vertex does not work: it need not lower every connectivity. Instead we delete a reachability skeleton. By Lemma A.12 it has at most $M(n)$ arcs, and the next lemma shows that deleting it lowers every positive connectivity.

Lemma 1.21 (Deleting a skeleton lowers every positive connectivity)**.** *Let D be a directed multigraph with $\lambda^{\max}(D) \leq k$ for some $k \geq 1$, and let R be a reachability skeleton of D. Let $D-R$ be D with one copy of each arc of R removed. Then for every ordered pair $u \neq v$,*

$$\lambda_{D-R}(u,v) \;\leq\; \max\bigl(\lambda_D(u,v)-1,\,0\bigr),$$

and consequently $\lambda^{\max}(D-R) \leq k-1$.

The skeleton keeps reachability. So if routes from u to v remain after the deletion, one more route runs inside the deleted arcs.

Proof. Fix $u \neq v$ and let $t = \lambda_{D-R}(u,v)$. If $t = 0$ there is nothing to prove. Otherwise $D-R$ has $t \geq 1$ arc-disjoint u–v paths, so u reaches v in D. Since R is a reachability skeleton, there is a u–v path P in R. The arcs of P are the removed copies, so P shares no arc with the t paths in $D-R$. So D has $t+1$ arc-disjoint u–v paths, and $t+1 \leq \lambda_D(u,v)$. Taking the maximum over all pairs gives $\lambda^{\max}(D-R) \leq k-1$. □

Proof of Theorem 1.18. The lower bound is Construction 1.20. For the upper bound, fix n. We show by induction on $k \geq 0$ that every directed multigraph D on n vertices with $\lambda^{\max}(D) \leq k$ has $|A(D)| \leq k\,M(n)$. For $k = 0$ there are no arcs, since one arc is already one route.

Let $k \geq 1$, and let R be a reachability skeleton of D with $|A(R)| \leq M(n)$ (Lemma A.12). By Lemma 1.21, $\lambda^{\max}(D-R) \leq k-1$, so by induction $|A(D-R)| \leq (k-1)M(n)$. Every arc of D is either in R or in $D-R$. So

$$|A(D)| \;=\; |A(R)| \;+\; |A(D-R)| \;\leq\; M(n) \;+\; (k-1)M(n) \;=\; k\,M(n).$$

With $k = m-1$ this gives $L_m^{\mathrm{dir}}(n) \leq (m-1)M(n)$. □

For $n \geq \max(8, 2m)$ the bipartite construction is the only extremal multigraph, up to reversing all arcs (Theorem A.22).

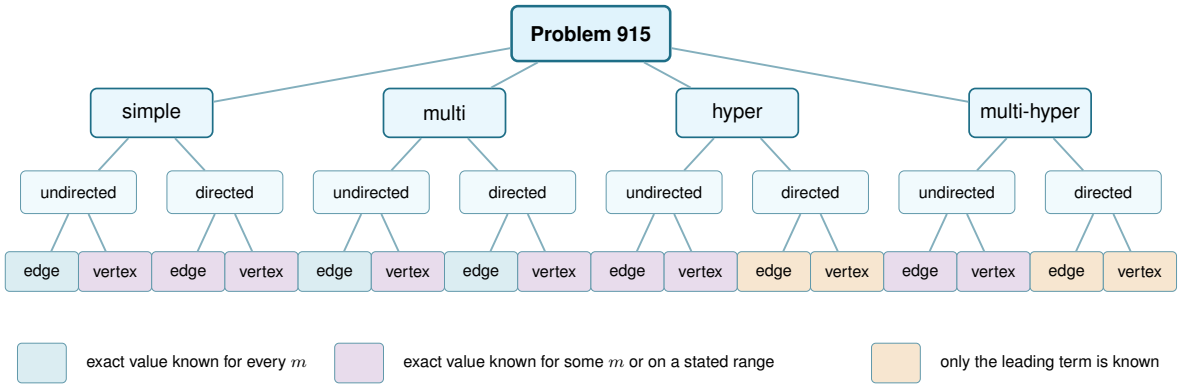


Figure 1.10. Overview of the sixteen variants. Blue: the exact value is known for every m. Purple: the exact value is known for some m or on a stated range. Amber: only the leading term is known.

Theorem 1.22 (The directed hypergraph constant)**.** *Let $r \geq 2$ and $m \geq 2$. Every forward directed r-uniform multihypergraph $\mathcal{H}$ on $n \geq 2$ vertices with $\kappa_{\mathcal{H}}^{\max} \leq m-1$ satisfies*

$$|E(\mathcal{H})| \;\leq\; \frac{m-1}{r-1}\left\lfloor \frac{n^2}{4} \right\rfloor \;+\; 4(m-1)^2(n-1).$$

Since $\kappa \leq \lambda$, the same bound holds when $\lambda_{\mathcal{H}}^{\max} \leq m-1$. The construction of Proposition A.44 has $(1+o(1))(m-1)n^2/(4(r-1))$ hyperedges, so the leading term is right.

Appendix A gives the proof. It uses the *shadow* digraph, which has an arc $u \to v$ when some hyperedge has tail u and head v. Each hyperedge gives $r-1$ arcs of the shadow, and each arc comes from at most $m-1$ hyperedges. The route condition on the hypergraph limits the routes of length two in the shadow. Lemma A.18 then bounds the number of arcs of the shadow.

The backward model is the forward model with every hyperedge reversed, so it has the same extremal values (Lemma A.46). The general model has the same leading term, with a larger linear term (Theorem A.49).

At $r = 2$ this also covers directed multigraphs with vertex separation. At $m = 2$ no arc can be repeated, so $K_2^{\mathrm{dir}}(n) = \ell_2^{\mathrm{dir}}(n)$. For fixed m, the extremal multigraphs of Theorem 1.18 are feasible here too, since $\kappa \leq \lambda$. With Theorem 1.22 at $r = 2$ this gives $K_m^{\mathrm{dir}}(n) = (m-1)n^2/4 + O_m(n)$ (Corollary A.41).

Section A.4.1 compares Theorem 1.17 with a theorem of Huang and Lyu [13].

1.5 Overview and open problems

Table 1.2 and Figure 1.10 collect what is known about the sixteen problems.

The main open problems are:

- Find $\ell_m^{\mathrm{dir}}(n)$ for $m \geq 3$, or at least the coefficient of the linear term.
- Is $k_m^{\mathrm{dir}}(n) = \ell_m^{\mathrm{dir}}(n)$ for $m \geq 3$?
- Find the simple vertex value $k_m(n)$ for $m \geq 6$, and the multigraph vertex value $K_m(n)$ for $m \geq 4$. Is $K_4(n) = L_4(n) = 3(n-1)$ for every n?

Model	Edge or arc separation	Vertex separation
Simple graph	$\lfloor m(n-1)/2 \rfloor$ for $n \geq m$	Exact for $m \leq 5$. Open for $m \geq 6$
Multigraph	$(m-1)(n-1)$	$n-1$ at $m=2$, $2(n-1)$ at $m=3$. Open for $m \geq 4$
Simple hypergraph	At most $\lfloor (m-1)(n-1)/(r-1) \rfloor$, equal when $m-1 \leq \binom{n-2}{r-2}$	$\lfloor (n-1)/(r-1) \rfloor$ at $m=2$, $\lfloor 2(n-1)/(r-1) \rfloor$ at $m=3$ for $r<n$. Open for $m \geq 4$
Multihypergraph	At most $\lfloor (m-1)(n-1)/(r-1) \rfloor$, equal when $(r-1) \mid (n-1)$	$\lfloor (n-1)/(r-1) \rfloor$ at $m=2$. At $m=3$ at most $\lfloor 2(n-1)/(r-1) \rfloor$, equal when $(r-1) \mid (n-1)$
Simple digraph	$M(n)$ at $m=2$. $n^2/4 + \Theta_m(n)$ for $m \geq 3$	Same
Directed multigraph	$(m-1)M(n)$	$M(n)$ at $m=2$. $(m-1)n^2/4 + O_m(n)$
Directed simple hypergraph	$(1+o(1))(m-1)n^2/(4(r-1))$	Same
Directed multihypergraph	$(1+o(1))(m-1)n^2/(4(r-1))$	Same

Table 1.2. Status of the sixteen problems. Here $m \geq 2$, hypergraphs have $r \geq 3$, and asymptotic statements are for fixed m and r as $n \to \infty$. The directed hypergraph rows hold in the forward, backward and general models.

- Find the hypergraph edge value outside the range where it is attained, and a vertex bound for hypergraphs at $m \geq 4$.
- Find exact values for directed hypergraphs, and compare the different ways to orient a hyperedge (Section A.9) beyond the leading term.

Chapter 2

Measuring and Searching by Machine

Chapter 1 stated the problem and the proved results. This chapter explains how a program can measure connectivity in every variant, check small graphs one by one, and search for dense examples. No result in this thesis depends on the output of this program. Its output for $n, m \leq 7$ is printed in Appendix B. Section 2.5 shows how an output can be checked without reading the code.

The program is one Python file of about 520 lines, `program/erdos915.py`. It uses the library `networkx` for maximum flows and the program `geng` of `nauty` for one search. This chapter prints the main parts of the program, each next to the idea it implements. The remaining parts, such as the imports and small helper functions, are in Appendix B, so every line of the program is printed once. The names in the code follow the words of the text. Figure 2.1 shows the structure of the program.

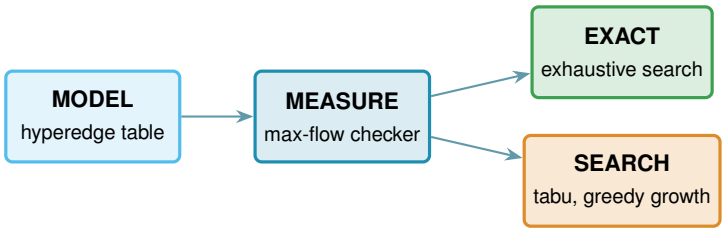


Figure 2.1. One model feeds the max-flow checker. Exhaustive search uses it to find exact values, and heuristic search uses it to find dense examples.

2.1 Modelling

The program treats all eight models in one way. A model is described by four facts: whether it is directed, whether edges may repeat, the edge size r, and, for directed hypergraphs, the allowed numbers of tails. A graph is the case $r = 2$. The program runs each directed hypergraph model in three versions, forward, backward and general (Section 1.3), so `ALL_MODELS` lists twelve entries for the eight models. The separation, edge or vertex, is a separate choice, so the eight models give the sixteen problems. A `Problem` bundles a model, a separation, the number of vertices n and the threshold m, and every function below takes one (Listing 2.1).

```
21 @dataclass(frozen=True)
22 class Model:
23     NAME: str
24     IS_DIRECTED: bool
25     ALLOWS_REPEATED_EDGES: bool
```

```
    EDGE_SIZE: int               # r: 2 for graphs, at least 3 otherwise
    TAIL_COUNTS: tuple = (1,)    # directed: the allowed numbers of tails


def ALL_MODELS(HYPEREDGE_SIZE=3):
    r = HYPEREDGE_SIZE
    splits = [("one tail", (1,)), ("one head", (r - 1,)),
              ("any split", tuple(range(1, r)))]
    return [
        Model("simple graph",             False, False, 2),
        Model("multigraph",               False, True,  2),
        Model("simple hypergraph",        False, False, r),
        Model("multihypergraph",          False, True,  r),
        Model("simple digraph",           True,  False, 2),
        Model("directed multigraph",      True,  True,  2),
    ] + [
        Model(f"{name}, {split}", True, repeated, r, tail_counts)
        for name, repeated in [("directed hypergraph", False),
                               ("directed multihypergraph", True)]
        for split, tail_counts in splits
    ]


class Separation(Enum):
    EDGE_DISJOINT = "edge"
    VERTEX_DISJOINT = "vertex"


@dataclass(frozen=True)
class Problem:
    MODEL: Model
    SEPARATION: Separation
    VERTEX_COUNT: int            # n
    THRESHOLD_M: int             # feasible: fewer than m disjoint routes
```

Listing 2.1. The models, with three versions of each directed hypergraph model, and the two separations.

Every edge is stored as a hyperedge: its vertices, and its tails if the model is directed. An ordinary edge uv is the hyperedge $\{u, v\}$ without tails, and an arc $u \to v$ is the hyperedge $\{u, v\}$ with tail u. A route may enter a hyperedge at an `entry_vertices` vertex and leave it at an `exit_vertices` vertex. In an undirected hyperedge every vertex is both. In a directed one only the tails are entries and only the heads are exits. A candidate graph is a table that gives every possible hyperedge its multiplicity. In a simple model every multiplicity is 0 or 1. In a multi model it is at most $m - 1$, since m copies of one edge already give m disjoint routes (Listing 2.2). Figure 2.2 shows two such tables.

```
@dataclass(frozen=True, order=True)
class Hyperedge:
    TAILS: tuple                 # empty for an undirected hyperedge
```

```
    MEMBERS: tuple               # all its vertices, sorted

    def entry_vertices(self):
        return self.TAILS or self.MEMBERS

    def exit_vertices(self):
        return tuple(v for v in self.MEMBERS if v not in self.TAILS)


def possible_hyperedges(problem):
    model, n = problem.MODEL, problem.VERTEX_COUNT
    hyperedges = []
    for members in itertools.combinations(range(n), model.EDGE_SIZE):
        if not model.IS_DIRECTED:
            hyperedges.append(Hyperedge((), members))
            continue
        for tail_count in model.TAIL_COUNTS:
            hyperedges += [Hyperedge(tails, members) for tails
                           in itertools.combinations(members, tail_count)]
    return hyperedges


def largest_multiplicity(problem):
    if problem.MODEL.ALLOWS_REPEATED_EDGES:
        return problem.THRESHOLD_M - 1
    return 1
```

Listing 2.2. Hyperedges, the list of all possible hyperedges, and the largest useful multiplicity.

2.2 Measuring

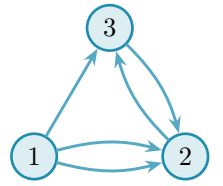
Every search below needs $\lambda^{\max}$ or $\kappa^{\max}$ of a candidate graph. This section explains how the program computes them.

To compute $\lambda(u, v)$ we use a maximum flow. In a network every link has a capacity, and a *maximum flow* is the largest amount that can be sent from u to v. We give each pair of vertices a link with capacity $\mu(u, v)$. In a digraph the link has a direction. In an undirected graph it can be used in both directions. The max-flow min-cut theorem of Ford and Fulkerson [6] says that the maximum flow equals the smallest capacity of a cut. By Menger's theorem, this is $\lambda(u, v)$. So computing a maximum flow computes $\lambda(u, v)$. We call this routine the *checker*.

2.2.1 Transformations

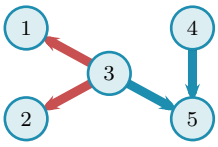
The checker works on one kind of network for all sixteen problems.

Every hyperedge, also an ordinary edge, becomes a *gate* (Figure 2.4b). The gate has an entry node and an exit node, joined by one arc whose capacity is the multiplicity of the hyperedge. Each vertex of the hyperedge gets an arc of unlimited capacity to the entry node and one from the exit node. So the arc inside the gate is the only bottleneck. For a directed hyperedge, only



directed multigraph
(a)

TAILS	MEMBERS	copies
$\{1\}$	$\{1,2\}$	**2**
$\{1\}$	$\{1,3\}$	1
$\{2\}$	$\{2,3\}$	1
$\{3\}$	$\{2,3\}$	1



directed hypergraph
(b)

TAILS	MEMBERS	copies
$\{\mathbf{3}\}$	$\{1,2,3\}$	1
$\{\mathbf{3},\mathbf{4}\}$	$\{3,4,5\}$	1

Figure 2.2. The program stores every model in one way: a table that gives each hyperedge its number of copies. (a) A directed multigraph. The arc $u \to v$ is the hyperedge with members $\{u, v\}$ and tail u, and the two parallel arcs from 1 to 2 are one row with 2 copies. (b) A directed 3-uniform hypergraph in the general model. Each hyperedge records its tails and its three members: the red one has tail 3, the blue one has tails 3 and 4 and head 5. In an undirected model there are no tails. A hyperedge that does not occur has 0 copies and is left out.

the tails have an arc to the entry, and only the heads have an arc from the exit (Figure 2.4c). So a route must go from a tail to a head. Theorem A.23 shows that the maximum flow in this network is the number of hyperedge-disjoint Berge routes. Figure 2.3 shows an example.

Every vertex v is split into two nodes v^{in} and v^{out}, joined by one arc (Figure 2.4a). Routes arrive at v^{in} and leave from v^{out}. For vertex separation this arc has capacity one, so at most one route can pass through v. For edge separation its capacity is unlimited, and it changes nothing. The gates keep their capacity, so parallel copies still give separate routes, as in the incidence convention of Section 1.3.1. Listing 2.3 builds this network.

```
110 def build_flow_network(problem, multiplicity):
111     network = networkx.DiGraph()
112     if problem.SEPARATION is Separation.VERTEX_DISJOINT:
113         through_vertex = 1
114     else:
115         through_vertex = math.inf
116     for v in range(problem.VERTEX_COUNT):
117         network.add_edge(("in", v), ("out", v), capacity=through_vertex)
118     for hyperedge, copies in multiplicity.items():
119         if copies == 0:
120             continue
121         enter, leave = ("enter", hyperedge), ("exit", hyperedge)
122         network.add_edge(enter, leave, capacity=copies)
123         for v in hyperedge.entry_vertices():
124             network.add_edge(("out", v), enter, capacity=math.inf)
125         for v in hyperedge.exit_vertices():
126             network.add_edge(leave, ("in", v), capacity=math.inf)
```

```
    return network
```

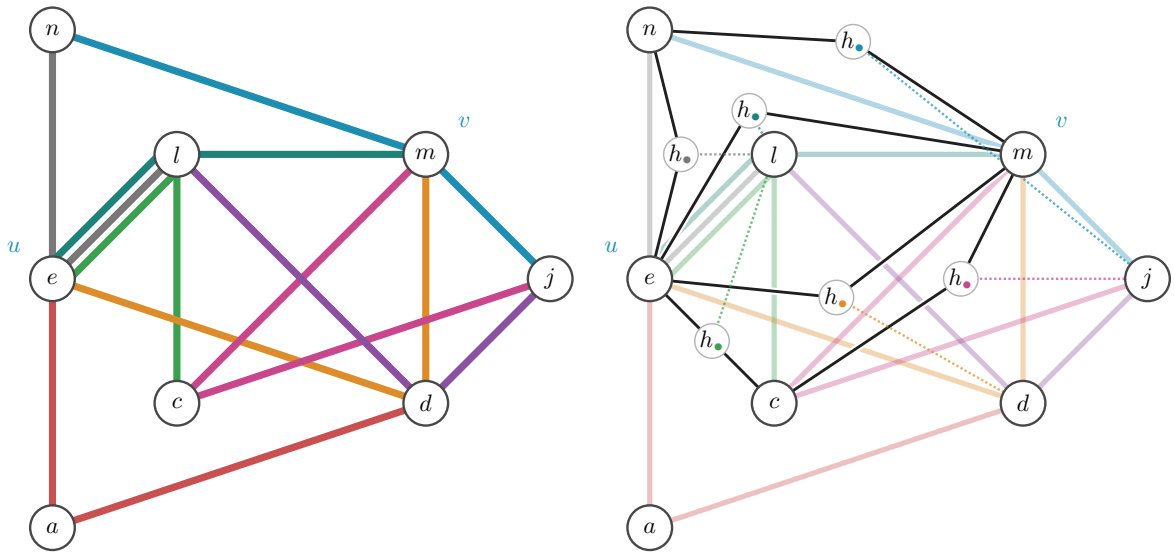
Listing 2.3. The flow network: split vertices and one gate per hyperedge.

The number of disjoint routes from u to v is the maximum flow from u^{out} to v^{in} (Listing 2.4). A graph is feasible when this number is below m for every pair, ordered in directed models. The flow may stop as soon as it reaches m, since a larger value changes nothing. This is the `stop_at` argument.

```
def disjoint_routes(network, source, target, stop_at=None):
    return networkx.maximum_flow_value(
        network, ("out", source), ("in", target),
        flow_func=networkx.algorithms.flow.edmonds_karp, cutoff=stop_at)


def is_feasible(problem, multiplicity):
    m = problem.THRESHOLD_M
    network = build_flow_network(problem, multiplicity)
    return all(disjoint_routes(network, u, v, stop_at=m) < m
               for u, v in vertex_pairs(problem))
```

Listing 2.4. Counting disjoint routes, and the feasibility test.



(a) (b)

Figure 2.3. (a) An undirected 3-uniform hypergraph. (b) Its gate network for the pair u, v. The four black routes use four different gates, so $\lambda(u, v) = 4$. The vertex v lies in only four hyperedges, so these four gates form a cut and no fifth route exists. Each gate is drawn as one point. Dotted lines join it to the third vertex of its hyperedge.

2.3 Checking every graph of a fixed order

To find an exact value, the program must check every graph, or exclude it by a correct pruning rule. A simple graph on n vertices is a choice of $\mu(u, v) \in \{0, 1\}$ for each pair, and a multigraph

is a choice in $\{0, \ldots, m-1\}$. A digraph makes this choice for each ordered pair. A hypergraph is a choice of some of the $\binom{n}{r}$ possible hyperedges. A multihypergraph also chooses a multiplicity for each. A forward directed hypergraph has r possible hyperedges on each r-set, one for each choice of tail, and chooses some of these $r\binom{n}{r}$ hyperedges. A backward one also has r, and a general one has $2^r - 2$, one for each non-empty proper subset of tails. The program goes through all choices, checks each one, and keeps the densest feasible graph. If the search is stopped early, it gives only a lower bound.

The number of vertices matters most. The number of candidates is (values per entry) to the power (number of entries). For graphs the number of entries grows like n^2, and for r-uniform hypergraphs like n^r. A larger m gives more values per entry. Direction gives more entries, since both directions of a pair are chosen separately. Directed hypergraphs have the most candidates, because each r-set carries several possible hyperedges. The separation does not change the candidates. It only changes the test for each candidate.

So there are $2^{\binom{n}{2}}$ simple graphs and $2^{n(n-1)}$ simple digraphs on n labelled vertices. With multiplicities at most $m-1$ there are $m^{\binom{n}{2}}$ multigraphs and $m^{n(n-1)}$ directed multigraphs. Figure 2.5 shows how fast this grows.

2.3.1 Pruned search

A pruned search skips whole families of graphs. Fix an order on the possible edges and decide them one at a time. A partly built graph stands for all graphs that agree with it so far. A pruning rule can discard this whole family.

The first rule uses that adding an edge never lowers connectivity. If a partly built graph already breaks the condition, so does every way to complete it, and the branch is dropped.

The second rule bounds the size of every completion: the edges already placed, plus the number of undecided edges times the largest allowed multiplicity. If this cannot beat the best graph found so far, the branch is dropped (Figure 2.6).

Both rules only drop graphs that are infeasible or cannot beat the best one. So a finished pruned search gives the same answer as the full search.

Listing 2.5 decides the possible hyperedges one at a time, in a fixed order. For each it tries $1, 2, \ldots$ copies, and stops at the first number of copies that makes the graph infeasible, since more copies stay infeasible. Then it also tries 0 copies. The first line of `decide` is the second rule: if even all undecided hyperedges at full multiplicity cannot beat the best graph so far, the branch is dropped. After the time limit the search stops and reports the best graph so far as a lower bound only.

```
159 def exhaustive_search(problem, deadline):
160     hyperedges = possible_hyperedges(problem)
161     cap = largest_multiplicity(problem)
162     best = {"size": -1, "graph": {}}
163     multiplicity = {}
164 
165     def decide(index, size):
```

```
        if time.time() > deadline:
            raise OutOfTime
        undecided = len(hyperedges) - index
        if size + undecided * cap <= best["size"]:
            return                          # cannot beat the best
        if index == len(hyperedges):
            best["size"], best["graph"] = size, dict(multiplicity)
            return
        hyperedge = hyperedges[index]
        for copies in range(1, cap + 1):
            multiplicity[hyperedge] = copies
            if not is_feasible(problem, multiplicity):
                break                       # more copies stay infeasible
            decide(index + 1, size + copies)
        multiplicity.pop(hyperedge, None)
        decide(index + 1, size)

    try:
        decide(0, 0)
        return best["size"], best["graph"], True
    except OutOfTime:
        return best["size"], best["graph"], False
```

Listing 2.5. The pruned exhaustive search.

2.3.2 Generating search

The full search meets the same graph many times, with the vertices renamed. Another method avoids this. Prof. Jan Goedgebeur pointed us to the `nauty` toolkit of McKay and Piperno [14]. Its program `geng` lists the undirected graphs on n vertices, each only once up to renaming the vertices. For directed multigraphs, our program uses each such graph as a *support*. Each support edge $\{u, v\}$ gets multiplicities $\mu(u, v)$ and $\mu(v, u)$ between 0 and $m - 1$, not both zero. Different supports give different digraphs, even after renaming vertices. After each choice the checker tests the partial digraph, and a branch that breaks the condition is dropped. Digraphs of the largest size are reported once each, up to renaming the vertices. For this the program compares digraphs after trying every renaming of the vertices. It does so only when the search finishes, which happens only for small n. The supports are read from `geng` one at a time (Listing B.4), so the search can stop at the time limit before all of them are listed. We wrote our own step for choosing the multiplicities, because it can prune after each choice. Listing 2.6 shows the whole search. For each support edge $\{a, b\}$, `choices` lists the pairs $(\mu(a, b), \mu(b, a))$ that are not both zero.

```
def best_digraphs_on_support(problem, support_edges, deadline):
    m = problem.THRESHOLD_M
    choices = [(forward, backward) for forward in range(m)
               for backward in range(m) if forward + backward > 0]
    best = {"size": -1, "graphs": []}
    multiplicity = {}
```

```
198
199     def choose(index, size):
200         if time.time() > deadline:
201             raise OutOfTime
202         undecided = len(support_edges) - index
203         if size + undecided * 2 * (m - 1) < best["size"]:
204             return
205         if index == len(support_edges):
206             if size > best["size"]:
207                 best["size"], best["graphs"] = size, []
208             best["graphs"].append(dict(multiplicity))
209             return
210         a, b = support_edges[index]
211         a_to_b, b_to_a = Hyperedge((a,), (a, b)), Hyperedge((b,), (a, b))
212         for forward, backward in choices:
213             multiplicity[a_to_b], multiplicity[b_to_a] = forward, backward
214             if is_feasible(problem, multiplicity):
215                 choose(index + 1, size + forward + backward)
216         del multiplicity[a_to_b], multiplicity[b_to_a]
217
218     try:
219         choose(0, 0)
220         return best["size"], best["graphs"], True
221     except OutOfTime:
222         return best["size"], best["graphs"], False
```

Listing 2.6. The generating search for directed multigraphs, with `geng` from `nauty`.

2.4 The heuristic search

For larger n the full search is impossible. Instead we search for dense graphs in a guided way. A *heuristic* is a rule that moves the search towards better graphs. The methods below also allow moves that make the graph worse, so that the search does not get stuck.

For graphs and digraphs, each candidate gets an *energy*

$$\Phi(G) = -|E(G)| + \Lambda \cdot \max\bigl(0,\ c^{\max}(G) - (m-1)\bigr).$$

Here $c^{\max}(G)$ is $\lambda^{\max}(G)$ or $\kappa^{\max}(G)$, depending on the separation, and $\Lambda > 0$ is a fixed penalty. Low energy means many edges and no excess connectivity. In the program Λ is `PENALTY_PER_EXCESS_ROUTE` (Listing 2.7).

```
268 PENALTY_PER_EXCESS_ROUTE = 10
269
270
271 def energy(size, highest_connectivity, threshold_m):
272     excess = max(0, highest_connectivity - (threshold_m - 1))
273     return -size + PENALTY_PER_EXCESS_ROUTE * excess
```

Listing 2.7. The energy of a candidate graph.

The search starts from the empty graph and changes one multiplicity by one at each step. We use *tabu search* [15]. At each step it makes the best allowed move, even if every move raises the energy. After a move, the changed pair is *tabu* for a fixed number of steps, so the search cannot undo the move at once. A move that gives a better graph than any seen so far is always allowed. When the search stalls, it makes a small change to escape: it removes all copies of three random hyperedges. Listing 2.8 lists the moves, and Listing 2.9 is the search itself. The search keeps the exact connectivity of every pair of the current graph in `connectivity`, and `highest_after` uses it to judge a move cheaply (Section 2.4.2).

```
def moves(problem, multiplicity):
    cap = largest_multiplicity(problem)
    for hyperedge in possible_hyperedges(problem):
        copies = multiplicity.get(hyperedge, 0)
        if copies < cap:
            yield hyperedge, +1
        if copies > 0:
            yield hyperedge, -1


def highest_after(problem, multiplicity, move, connectivity):
    """Measures only near the cap: one copy more raises a value by at
    most one, so only pairs already at m-1 need a check, and one copy
    less never raises a value."""
    cap = problem.THRESHOLD_M - 1
    current_highest = max(connectivity.values(), default=0)
    _, change = move
    if change < 0 and current_highest <= cap:
        return current_highest
    if change > 0 and current_highest < cap:
        return current_highest + 1
    network = build_flow_network(problem, apply(multiplicity, move))
    at_cap = [pair for pair, value in connectivity.items()
              if value >= cap]
    return max((disjoint_routes(network, u, v) for u, v in at_cap),
               default=0)
```

Listing 2.8. The moves: one copy more or one copy less of one hyperedge.

```
TABU_TENURE = 7
STEPS_WITHOUT_PROGRESS_BEFORE_KICK = 30


def tabu_search(problem, deadline, seed):
    generator = random.Random(seed)
    m = problem.THRESHOLD_M
    multiplicity, tabu_until, step = {}, {}, 0
    connectivity = all_connectivities(problem, multiplicity)
    best_size, best_graph, last_progress = 0, {}, 0
    while time.time() < deadline:
        step += 1
```

```
        scored = []
        for move in moves(problem, multiplicity):
            highest = highest_after(problem, multiplicity, move,
                                    connectivity)
            size = edge_count(multiplicity) + move[1]
            new_best = highest <= m - 1 and size > best_size
            if tabu_until.get(move[0], 0) > step and not new_best:
                continue                    # tabu, unless a new best
            scored.append((energy(size, highest, m),
                           generator.random(), move))
        if not scored:
            break
        _, _, move = min(scored)
        multiplicity = apply(multiplicity, move)
        tabu_until[move[0]] = step + TABU_TENURE
        connectivity = all_connectivities(problem, multiplicity)
        size = edge_count(multiplicity)
        if max(connectivity.values(), default=0) <= m - 1 \
                and size > best_size:
            best_size, best_graph = size, multiplicity
            last_progress = step
        if step - last_progress > STEPS_WITHOUT_PROGRESS_BEFORE_KICK:
            kicked = generator.sample(list(multiplicity),
                                      min(3, len(multiplicity)))
            multiplicity = dict(multiplicity)
            for hyperedge in kicked:
                multiplicity[hyperedge] = 0
            connectivity = all_connectivities(problem, multiplicity)
            last_progress = step
    return best_size, best_graph
```

Listing 2.9. Tabu search.

The program also implements simulated annealing [16] on the same energy. It accepts a worse move with a probability that decreases over time [17]. At temperature T a move that raises the energy by Δ is accepted with probability $e^{-\Delta/T}$, and T shrinks by a fixed factor at every step. The driver in Listing 2.13 does not use it, so its code is in Appendix B (Listing B.6).

Every feasible graph can be built from the empty graph by adding edges one at a time, staying feasible. Still, it can help to pass through infeasible graphs: exchanging two edges may need an extra edge first. Tabu search allows this, but it only keeps feasible results.

Each search keeps the densest feasible graph it has seen.

2.4.1 Random greedy growth for hypergraphs

For hypergraphs we use random greedy growth. Each run starts with the empty hypergraph and a shuffled list of all possible hyperedges: each hyperedge once in the simple model, and $m - 1$ times in the multi model. The procedure adds the hyperedges one by one, and keeps each one only if the hypergraph stays feasible. This is repeated with new random orders, and the densest

result is kept.

At the end of a run no single hyperedge can be added, because a rejected hyperedge stays infeasible when more hyperedges are added. This does not mean the result is as large as possible: an early choice can block several later ones. We use this method instead of tabu search because there are many possible hyperedges, for example $\binom{n}{r}$ in the undirected simple case.

```
def random_greedy_growth(problem, deadline, seed):
    generator = random.Random(seed)
    cap = largest_multiplicity(problem)
    best_size, best_graph = 0, {}
    while time.time() < deadline:
        copies = [hyperedge for hyperedge in possible_hyperedges(problem)
                  for _ in range(cap)]
        generator.shuffle(copies)
        multiplicity = {}
        for hyperedge in copies:
            if time.time() > deadline:
                break                          # the partial graph is feasible
            grown = apply(multiplicity, (hyperedge, +1))
            if is_feasible(problem, grown):
                multiplicity = grown
        if edge_count(multiplicity) > best_size:
            best_size, best_graph = edge_count(multiplicity), multiplicity
    return best_size, best_graph
```

Listing 2.10. Random greedy growth.

2.4.2 Measuring only near the connectivity cap

Removing an edge never raises $\lambda^{\max}$. Adding an edge raises it by at most one, because at most one route of a largest family uses the new edge. The same holds for $\kappa^{\max}$. So the search does not need to run the checker at every step. Removing an edge from a feasible graph keeps it feasible. Adding an edge to a graph with $\lambda^{\max} \leq m-2$ also keeps it feasible. Also, an addition can only push a pair above $m-1$ if that pair is already at $m-1$. So `highest_after` in Listing 2.8 answers without the checker when the graph stays feasible for sure. Otherwise it measures only the pairs at the cap. After a move is made, the search measures all pairs once, so the next step again starts from exact values.

2.5 Checking the output without trusting the program

The program is too long to check by hand, so a value it prints is not a proof. We solve this by separating *finding* from *checking*. The program may find a graph in any way, correct or not. Together with the graph it writes a short *certificate*: a list that a reader can check by hand, or with a few lines of independent code, without trusting the program.

Edge separation. For every pair u, v, the certificate gives a set of at most $m-1$ edges (or arcs, or hyperedge copies) whose removal leaves no route from u to v. To check it, remove the

set and follow the remaining edges from u: v must not be reached. Then every route from u to v uses an element of the set, so no pair has m disjoint routes. This check uses only that a route through a removed set must use one of its elements. It needs neither Menger's theorem nor a maximum flow. For undirected graphs one tree is enough: any tree on the vertices such that, for each tree edge, at most $m - 1$ edges of G cross it. Every pair u, v is then separated by the split of any tree edge on its tree path, so $n - 1$ counts replace $\binom{n}{2}$ checks. A Gomory–Hu tree (Theorem 1.4) of a feasible graph is such a tree.

Vertex separation. For every pair u, v, the certificate gives a set of at most $m - 1$ inner nodes of the incidence graph, vertices or edge copies, whose removal leaves no route from u to v. It is checked in the same way.

Size of the graph. The number of edges is counted directly.

The program writes the certificate from a smallest cut of the flow network (Listing 2.11). A cut uses only arcs of limited capacity: the arc inside a gate, which stands for a hyperedge, and, for vertex separation, the arc inside a split vertex. So the cut translates directly into removed hyperedges and removed vertices.

```
def write_certificate(problem, multiplicity):
    network = build_flow_network(problem, multiplicity)
    certificate = {}
    for u, v in vertex_pairs(problem):
        _, (source_side, _) = networkx.minimum_cut(
            network, ("out", u), ("in", v))
        removed_vertices, removed_hyperedges = [], []
        for node in source_side:
            for neighbour in network.successors(node):
                if neighbour in source_side:
                    continue
                if node[0] == "in":
                    removed_vertices.append(node[1])
                else:
                    removed_hyperedges.append(node[1])
        certificate[(u, v)] = (removed_vertices, removed_hyperedges)
    return certificate
```

Listing 2.11. Writing a certificate from smallest cuts.

The checker in Listing 2.12 uses none of the code above. It counts the removed elements, with every copy of a removed hyperedge, and then follows the remaining hyperedges from u, entering each only at an entry vertex. If v is never reached, the certificate is correct for this pair. Its correctness can be read off from its 30 lines alone.

```
def check_certificate(problem, multiplicity, certificate):
    """Independent of the code above: no flows, only following edges."""
    for u, v in vertex_pairs(problem):
        removed_vertices, removed_hyperedges = certificate[(u, v)]
        if problem.SEPARATION is Separation.EDGE_DISJOINT \
                and removed_vertices:
            return False
```

```
        if u in removed_vertices or v in removed_vertices:
            return False
        size = len(set(removed_vertices)) + sum(
            multiplicity[e] for e in set(removed_hyperedges))
        if size > problem.THRESHOLD_M - 1:
            return False
        reached, frontier = {u}, [u]
        while frontier:
            vertex = frontier.pop()
            for hyperedge, copies in multiplicity.items():
                if copies == 0 or hyperedge in removed_hyperedges:
                    continue
                if vertex not in hyperedge.entry_vertices():
                    continue
                for nxt in hyperedge.exit_vertices():
                    if nxt not in reached and nxt not in removed_vertices:
                        reached.add(nxt)
                        frontier.append(nxt)
        if v in reached:
            return False
    return True
```

Listing 2.12. The independent certificate checker.

So every lower bound the program finds can be checked: the graph is feasible by its certificate, and its size is counted. An upper bound is different. A certificate that no graph on n vertices has more edges would be the whole search, which is too long to check. So the thesis takes no upper bound or exact value from the program. Every upper bound is proved by hand, and every construction in the thesis is checked by hand in Chapter 1 or Appendix A. The exhaustive and heuristic searches of this chapter show which bounds are worth trying to prove, and which constructions to try.

2.6 Running the program

Listing 2.13 puts the parts together. For each problem, each m and each n it spends at most thirty seconds. The first half goes to an exact search: the generating search for directed multigraphs, and the pruned search for the other models. If that does not finish, the second half goes to tabu search for graphs and digraphs, and to random greedy growth for hypergraphs. Every result is then certified and checked. The cases are independent, so the program runs them in parallel, one per processor core. For each problem, and each version of a directed hypergraph problem, it prints a table with one row per n and one column per m (Listing B.7). Each entry is the number of edges of the densest feasible graph found, marked when the search finished, and marked differently if the certificate failed. A finished search gives an exact value, but only the lower bound is certified (Section 2.5).

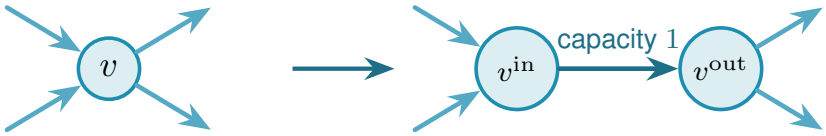
```
SECONDS_PER_CASE = 30


```

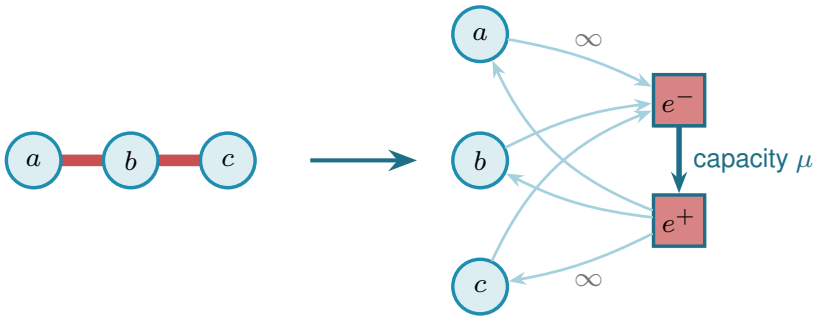
```
470 def solve(problem):
471     model = problem.MODEL
472     deadline = time.time() + SECONDS_PER_CASE / 2
473     if model.IS_DIRECTED and model.ALLOWS_REPEATED_EDGES \
474             and model.EDGE_SIZE == 2:
475         size, graph, _, exact = generating_search(problem, deadline)
476     else:
477         size, graph, exact = exhaustive_search(problem, deadline)
478     if not exact:
479         deadline = time.time() + SECONDS_PER_CASE / 2
480         if model.EDGE_SIZE == 2:
481             search = tabu_search
482         else:
483             search = random_greedy_growth
484         found_size, found_graph = search(problem, deadline, seed=1)
485         if found_size > size:
486             size, graph = found_size, found_graph
487     certificate = write_certificate(problem, graph)
488     return size, exact, check_certificate(problem, graph, certificate)
```

Listing 2.13. The driver: thirty seconds per case, then a certificate.

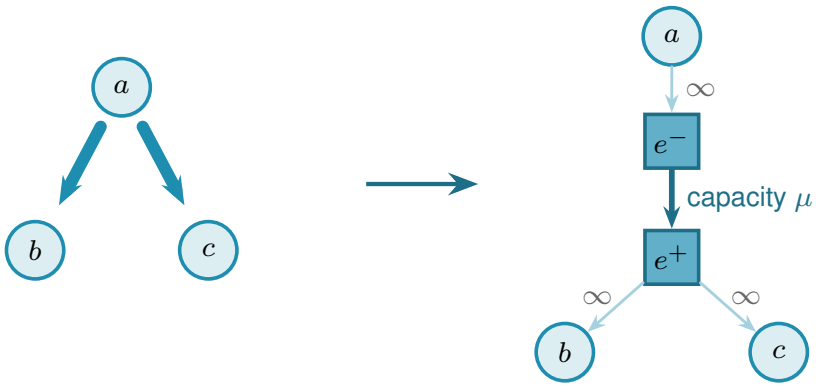
Section B.2 prints the full output for $2 \leq m \leq 7$ and $n \leq 7$.



(a) Splitting each vertex v into v^{in} and v^{out}, joined by an arc of capacity one, turns vertex-disjoint routes into arc-disjoint routes. Only this inner arc has capacity one.



(b) Transform a hyperedge into a gate with capacity equal to the multiplicity of the hyperedge.



(c) A directed hyperedge with one tail and two heads is transformed into a gate. This gate has capacity equal to the multiplicity of the hyperedge and every route must enter at the tail and leave at a head. With several tails, each tail has an arc to e^-.

Figure 2.4. The building blocks of the connectivity checker: vertex splitting, hyperedge gates and directed hyperedge gates.

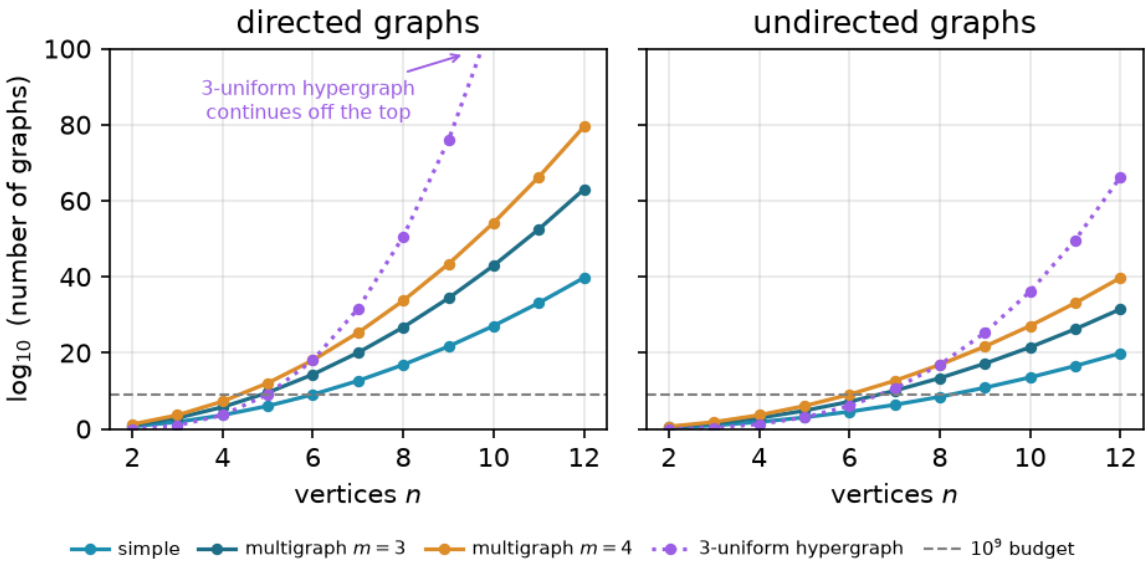


Figure 2.5. The number of candidates for blind enumeration, on a logarithmic scale. Every model passes 10^9 candidates before $n = 10$. Directed models grow fastest, because they have more entries to choose. The directed hypergraph curve is the forward model.

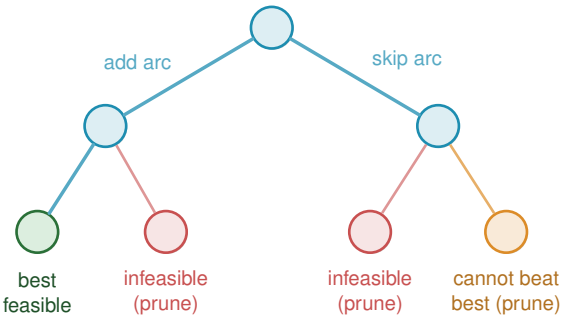


Figure 2.6. Each level of the partially built graph fixes one adjacency. Infeasible partial graphs (red) and branches unable to beat the incumbent (orange) are discarded.

Appendix A

Proofs

This appendix contains the proofs that are not in Chapter 1, and the lemmas they need. The definitions are in Chapter 1. New terms are defined where they are first used.

The appendix has three parts. PART I is about undirected graphs, PART II about directed graphs, and PART III about hypergraphs. The three main proof ideas are the reachability skeleton (directed multigraphs), the incidence-rank lemma (hypergraphs at $m = 3$), and the two-step budget (directed graphs and hypergraphs).

Part I. Undirected graphs

Graphs that attain Mader's bound, and the vertex problem as a problem about blocks.

A.1 Graphs that attain Mader's bound

The constructions below need hypergraphs in which no vertex lies in too many hyperedges. Recall that a graph is a 2-uniform hypergraph, and the *degree* of a vertex is the number of hyperedges that contain it.

Lemma A.1 (Sparse uniform hypergraphs exist)**.** *Let r, n and d be integers with $r \geq 1$, $n \geq r$ and $0 \leq d \leq \binom{n-1}{r-1}$. There is a simple r-uniform hypergraph on n vertices with exactly $\lfloor \frac{dn}{r} \rfloor$ hyperedges and maximum degree at most d.*

Proof. Let $N := \lfloor \frac{dn}{r} \rfloor$. Then $N \leq \frac{n}{r}\binom{n-1}{r-1} = \binom{n}{r}$, so there are at least N different r-sets. Among all families of N different r-sets, choose one with the smallest value of $\sum_x \deg(x)^2$. Suppose $\deg(x) \geq \deg(y) + 2$ for two vertices x, y. Let a be the number of sets in the family that contain x but not y, and b the number that contain y but not x. Then $a \geq b + 2$. The map $A \mapsto A - x + y$ sends the first kind of set to the second kind, and different sets have different images. Since $a > b$, some image $A - x + y$ is not in the family. Replace A by $A - x + y$. The family stays simple and has N sets, and the sum changes by

$$(\deg(x) - 1)^2 + (\deg(y) + 1)^2 - \deg(x)^2 - \deg(y)^2 = 2\big(\deg(y) - \deg(x)\big) + 2 \leq -2.$$

This contradicts the choice of the family. So any two degrees differ by at most one. The degrees add up to $rN \leq dn$, so the largest degree is $\lceil rN/n \rceil \leq d$. □

Construction A.2 (Hub-and-spokes graph)**.** Let $n \geq m \geq 2$. The graph $H_{m,n}$ has a vertex h and further vertices $v_1, \ldots, v_{n-1}$. Join h to every v_i. On $v_1, \ldots, v_{n-1}$, add a graph with $\lfloor (m-2)(n-1)/2 \rfloor$ edges in which every vertex has degree at most $m - 2$. For $n \geq 3$ such a graph exists by Lemma A.1 with uniformity 2. If $m - 1$ divides $n - 1$, one can take disjoint copies of K_{m-1}. Then $H_{m,n}$ consists of copies of K_m that share the vertex h.

Theorem A.3 (Extremal graphs exist)**.** *For all $n \geq m \geq 2$ the graph $H_{m,n}$ has $\left\lfloor \frac{m(n-1)}{2} \right\rfloor$ edges and $\lambda^{\max} \leq m - 1$.*

Proof. The number of edges is $(n-1) + \left\lfloor \frac{(m-2)(n-1)}{2} \right\rfloor = \left\lfloor \frac{m(n-1)}{2} \right\rfloor$. Of any two vertices, at least one is some v_i. This vertex has degree at most $1 + (m-2) = m - 1$. So Lemma 1.7 gives $\lambda(u, v) \leq m - 1$. □

So for $n \geq m$ the bound of Theorem 1.3 is attained. The next theorem describes what equality means for the Gomory–Hu tree.

Theorem A.4 (Equality in the Gomory–Hu bound)**.** *Let G be simple with $\lambda_G^{\max} \leq m-1$ and $|E(G)| = \left\lfloor \frac{m(n-1)}{2} \right\rfloor$, and let T be a Gomory–Hu tree of G.*

(a) If $m(n-1)$ is even, then every tree edge has weight $m-1$, every tree edge is an edge of G, and every edge of G has $\mathrm{dist}_T(e) \leq 2$.

(b) If $m(n-1)$ is odd, then exactly one of these three statements fails, and it fails by one: one tree edge has weight $m-2$, or one tree edge is not an edge of G, or one edge of G has $\mathrm{dist}_T(e) = 3$.

Proof. Each inequality in the proof of Theorem 1.3 has a non-negative integer gap:

$$S_1 = (m-1)(n-1) - \sum_t c(t), \qquad S_2 = \sum_{e:\,\mathrm{dist}_T(e)\geq 2} \big(\mathrm{dist}_T(e) - 2\big), \qquad S_3 = (n-1) - |E_1|.$$

Here S_1 counts tree weights below $m-1$, S_2 counts how far edges go beyond tree distance two, and S_3 counts tree edges that are not edges of G. The proof gives two expressions for the same sum:

$$\sum_e \mathrm{dist}_T(e) \;=\; 2|E| - |E_1| + S_2 \qquad \text{and} \qquad \sum_e \mathrm{dist}_T(e) \;=\; \sum_t c(t) \;=\; (m-1)(n-1) - S_1.$$

We substitute $|E_1| = (n-1) - S_3$ into the first and compare with the second:

$$\begin{aligned} 2|E| - (n-1) + S_2 + S_3 \;&=\; (m-1)(n-1) - S_1, \\ \text{so} \qquad 2|E| \;&=\; m(n-1) - (S_1 + S_2 + S_3). \end{aligned}$$

If $m(n-1)$ is even, then $|E| = m(n-1)/2$ forces $S_1 = S_2 = S_3 = 0$. If $m(n-1)$ is odd, then $|E| = (m(n-1)-1)/2$ gives $S_1 + S_2 + S_3 = 1$, so exactly one gap equals one. □

For $m = 2$ this shows that the extremal graphs are exactly the trees on the n vertices. Indeed, every tree edge then has weight one and is an edge of G, so G contains T and has only $n-1$ edges.

Remark A.5 (Forcing and avoiding counts). This thesis maximises the number of edges with no m disjoint routes between any pair. Sørensen and Thomassen, and the Erdős Problems database, give instead the least number of edges that forces such a pair [3, 12]. Their number is one more than ours.

For example, Theorem 4 of [12] gives the forcing number $\lfloor 8n/3 \rfloor - 3$ for $n \geq 6$, except 16 at $n = 7$ and 28 at $n = 12$. Subtracting one gives our values. For $6 \leq n \leq 13$ these numbers, the two exceptions included, equal $\lfloor 5(n-1)/2 \rfloor + 1$. Subtracting one gives the two cases of Theorem 1.10. At $n = 13$ both maxima are 30, and the edge and vertex values first differ at $n = 14$.

The same care is needed for the Bollobás–Erdős conjecture of Section A.1.1. Its forcing number $1 + \binom{m}{2}p$ is for graphs on $1 + (m-1)p$ vertices. Each copy of K_m adds $\binom{m}{2}$ edges but

only $m-1$ vertices, so in our notation the rate is $m/2$ edges per vertex.

A.1.1 The vertex problem is a block problem

The vertex problem $k_m(n)$ is open for $m \geq 6$. This section reduces it to 2-connected graphs (Theorem A.6).

A graph on more than k vertices is *k-connected* if removing any $k-1$ vertices leaves it connected. A *cut vertex* is a vertex whose removal disconnects the graph. A *bridge* is an edge whose removal disconnects its component. A *block* is a maximal connected subgraph that has an edge and no cut vertex of its own. In a simple graph, a block is either a bridge or a maximal 2-connected subgraph.

The next theorem says that the best graph is built from blocks: choose block sizes b_i, where a block of size b_i costs $b_i - 1$ vertices and gives at most $h_m(b_i)$ edges, with total cost at most $n-1$.

Theorem A.6 (The vertex problem reduces to blocks)**.** *For $b \geq 2$ let*

$$h_m(b) \;:=\; \max\bigl\{\, |E(B)| \;:\; B \text{ 2-connected or a single edge},\ |V(B)| = b,\ \kappa_B^{\max} \leq m-1 \,\bigr\},$$

with $h_m(b) := -\infty$ when no such B exists. Then for all $m \geq 2$ and $n \geq 2$,

$$k_m(n) \;=\; \max\Bigl\{ \sum_i h_m(b_i) \;:\; b_i \geq 2,\ \sum_i (b_i - 1) \leq n-1 \Bigr\}.$$

Consequently $k_m(n) \geq k_m(n_1) + k_m(n_2)$ whenever $n = n_1 + n_2 - 1$, and

$$c_m \;:=\; \lim_{n\to\infty} \frac{k_m(n)}{n-1} \;=\; \sup_{b\geq 2} \frac{h_m(b)}{b-1}$$

exists, the limit being approached from below. The supremum is finite: Remark A.7 gives $k_m(n) < 2(m-1)n$, so $c_m \leq 2(m-1)$.

Proof. Let G be feasible, and let u, v lie in a common block B. No u–v path leaves B: it would have to leave and come back through the same cut vertex, and a path visits each vertex once. So $\kappa_G(u,v) = \kappa_B(u,v)$. Every block is a subgraph of G, so it is feasible too. Every edge lies in exactly one block, so $|E(G)| = \sum_B |E(B)| \leq \sum_B h_m(|V(B)|)$. In each component, $\sum_B (|V(B)|-1)$ is the number of vertices minus one. So over all of G, $\sum_B (|V(B)|-1) \leq n-1$. This proves $\leq$.

For $\geq$, take blocks $B_1, \ldots, B_t$ with $\sum(b_i - 1) \leq n-1$. Glue them at one common vertex, and keep them disjoint otherwise. This graph has $1 + \sum(b_i - 1) \leq n$ vertices. Add isolated vertices to reach n. Its blocks are the B_i, so pairs inside one block keep their value of κ. Two vertices in different blocks are separated by the common vertex, so $\kappa \leq 1 \leq m-1$. So the graph is feasible, with $\sum_i |E(B_i)|$ edges.

Gluing two extremal graphs at one vertex gives $k_m(n) \geq k_m(n_1) + k_m(n_2)$ for $n = n_1 + n_2 - 1$.

For the limit, write $a_n = k_m(n)/(n-1)$ and $s = \sup_{b\geq 2} h_m(b)/(b-1)$. By the display, $k_m(n) \leq s\,(n-1)$, so $a_n \leq s$ for every n. Fekete's lemma [18] says: if numbers $x_1, x_2, \ldots$ satisfy $x_{i+j} \geq x_i + x_j$, then x_t/t tends to $\sup_t x_t/t$. We apply it to $x_t = k_m(t+1)$, which satisfies this by the gluing inequality. So a_n tends to $\sup_n a_n$. This supremum is s: it is at most s, and $a_b \geq h_m(b)/(b-1)$ for every b, since one block of size b is a graph on b vertices. □

We note three consequences.

First, at $m = 3$ the reduction gives Theorem 1.9 again. A block with $\kappa^{\max} \leq 2$ is an edge or a cycle. Otherwise it contains a *theta*: three internally disjoint paths with the same two ends, which break the condition. So $h_3(2) = 1$ and $h_3(b) = b$ for $b \geq 3$. The rate $b/(b-1)$ is largest at $b = 3$, so the best choice uses triangles: $k_3(n) = 3\lfloor (n-1)/2 \rfloor + ((n-1) \bmod 2) = \left\lfloor \frac{3(n-1)}{2} \right\rfloor$.

Second, at $m = 4$, Theorem 1.9 gives $h_4(b) = 2(b-1)$ for every $b \geq 4$. So the rate $h_4(b)/(b-1)$ is 2 for $b \geq 4$, against 1 at $b = 2$ and $3/2$ at $b = 3$. The best choice uses blocks of size at least four, and $k_4(n) = 2(n-1)$ for $n \geq 4$. This uses Theorem 1.9, so it is not a new proof. It does show the extremal block: the hub-and-spokes graph $H_{4,b}$ of Construction A.2, which is a hub joined to all vertices of a cycle. It is a single block with $2(b-1)$ edges.

Third, the Bollobás–Erdős construction, copies of K_m sharing one vertex, is the case $b = m$: $h_m(m) = \binom{m}{2}$, with rate $m/2$. Their conjecture says that $c_m = m/2$, so that no block beats K_m. A *bouquet* is a set of blocks that share one vertex and are otherwise disjoint. Its rate is a weighted average of the rates of its blocks. So a counterexample must contain one 2-connected block with rate above $m/2$.

The conjecture is false. Sørensen and Thomassen [12] showed that for each fixed m,

$$c_m \;\geq\; \frac{m(m-1)-2}{2m-3} \;=\; \frac{m}{2} + \frac{1}{4} + O(1/m).$$

This is larger than $m/2$ for every $m \geq 5$, because

$$2\big(m(m-1)-2\big) - m(2m-3) \;=\; m-4.$$

Their example is built step by step. Start with $G^0 = K_m$ minus an edge. To build G^j, take G^{j-1} and two new copies of G^0, and glue them in a cycle: each one shares one vertex with the next. Each step adds $2m-3$ vertices and $m(m-1)-2$ edges, which gives the rate above. Gluing in a cycle creates no cut vertex, so G^j is a single block. So Theorem A.6 sees G^j as one block of size $m + (2m-3)j$ and does not help to find it. The recursion beats the rate $m/2$ only when $j > 2/(m-4)$. So the smallest member of this family that beats copies of K_m has 26 vertices at $m = 5$, 24 at $m = 6$ and 18 at $m = 7$. Smaller examples exist. At $m = 5$, Theorem 1.10 gives 33 edges on 14 vertices, and by the bouquet argument above such a graph contains a block with rate above $5/2$. So $c_m = m/2$ for $m \leq 4$, $c_5 = 8/3$ by Theorem 1.10, and $m/2 < c_m \leq 2(m-1)$ for $m \geq 6$.

Remark A.7 (A question about small degrees). Mader's density theorem [19], in the form of [4, Theorem 1.4.3], says that a graph of average degree at least $4k$ has a $(k+1)$-connected

subgraph. A graph with at least $2(m-1)n$ edges has average degree at least $4(m-1)$, so it has an m-connected subgraph, and two of its vertices are joined by m internally disjoint paths. So $k_m(n) < 2(m-1)n$. Feasibility gives more. For example, two adjacent vertices have at most $m-2$ common neighbours, since each common neighbour gives a route of length two next to the edge.

We ask whether every feasible graph has a vertex of degree at most $m-1$. Feasibility passes to induced subgraphs. So a positive answer would mean that every non-empty induced subgraph has a vertex of degree at most $m-1$. Removing such vertices one by one would give

$$k_m(n) \;\le\; \sum_{i=1}^{n} \min(i-1,\, m-1) \;=\; (m-1)n - \binom{m}{2} \qquad (n \ge m-1),$$

half the constant of Mader's bound.

The answer is yes for $m = 3$. By a theorem of Dirac [20], a graph of minimum degree at least three contains a *subdivision* of K_4: a copy of K_4 in which edges are replaced by internally disjoint paths. Two of its four main vertices are joined by three internally disjoint paths, which breaks the condition at $m = 3$. For $m \ge 4$ the question is open.

Part II. Directed graphs

The directed constructions, the reachability skeleton, the bounds for $m \ge 3$, and the extremal directed multigraphs.

A.2 The directed constructions

Three constructions give the directed lower bounds. Figure 1.8 shows the first two. Here we define them and check their connectivity.

Construction A.8 (Directed hub)**.** For $n \ge m \ge 2$, take a hub h and label the other $N = n-1$ vertices $0, \dots, N-1$. Add the $2N$ hub arcs $h \to i$ and $i \to h$, and on the non-hub vertices the $(m-2)N$ ring arcs $i \to i+j \bmod N$ for $j = 1, \dots, m-2$. The total is $m(n-1)$ arcs and every non-hub vertex has $d^+ = d^- = m-1$.

Every ordered pair contains a vertex other than the hub, and that vertex has out-degree and in-degree $m-1$. So the degree bound gives $\lambda^{\max} \le m-1$. At $m = 2$ there are no ring arcs, and we get the *bidirected star*.

Construction A.9 (One-directional bipartite digraph)**.** Split the vertices into parts A and B of sizes $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$, and include every arc from A to B and none the other way. The total is $\lfloor n^2/4 \rfloor$ arcs.

No arc leaves B and no arc enters A. So every route has length one, and $\lambda^{\max} = 1$.

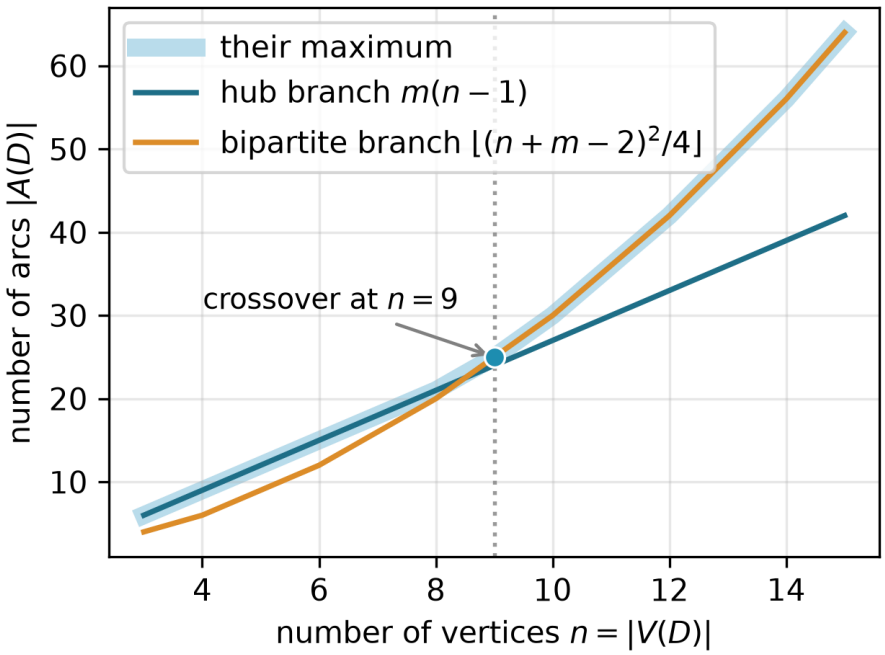


Figure A.1. The arc counts of the hub and of the augmented bipartite digraph at $m = 3$, and their maximum. These are lower bounds only.

Construction A.10 (Augmented bipartite digraph)**.** For $n \geq m \geq 2$, set $|B| = \lceil (n+m-2)/2 \rceil$ and $|A| = n - |B|$, include every arc from A to B, and inside B give each b_j the $m-2$ in-arcs from its cyclic predecessors $b_{j-1}, \ldots, b_{j-(m-2)}$. The total is $\lfloor (n+m-2)^2/4 \rfloor$ arcs.

Lemma A.13 shows that $\lambda^{\max} = m - 1$. For example, take $m = 3$ and $n = 10$. Then $|B| = 6$ and $|A| = 4$. There are 24 arcs from A to B, and 6 arcs forming a cycle inside B. Each pair a_i, b_j has the direct arc and one detour, so $\lambda^{\max} = 2$. This gives 30 arcs, more than the $\max(m(n-1), \lfloor n^2/4 \rfloor) = 27$ of the first two constructions. The gain comes from making B larger than half of the vertices. So for $n \geq m$,

$$\ell_m^{\mathrm{dir}}(n) \;\geq\; \max\Big(m(n-1),\; \Big\lfloor \frac{(n+m-2)^2}{4} \Big\rfloor\Big).$$

For fixed m and large n the augmented bipartite digraph has more arcs (Figures A.1 and A.2).

A.3 The reachability skeleton

The next lemma is used for both exact directed results. The proofs of Theorems 1.12 and 1.18 in Chapter 1 use it.

Definition A.11 (Reachability skeleton)**.** Let D be a directed multigraph. A *reachability skeleton* of D is a subdigraph R on the same vertices, with at most one copy of each arc, such that for all $u \neq v$: u can reach v in R exactly when u can reach v in D.

A skeleton keeps the information about who can reach whom. It does not keep the number of disjoint paths.

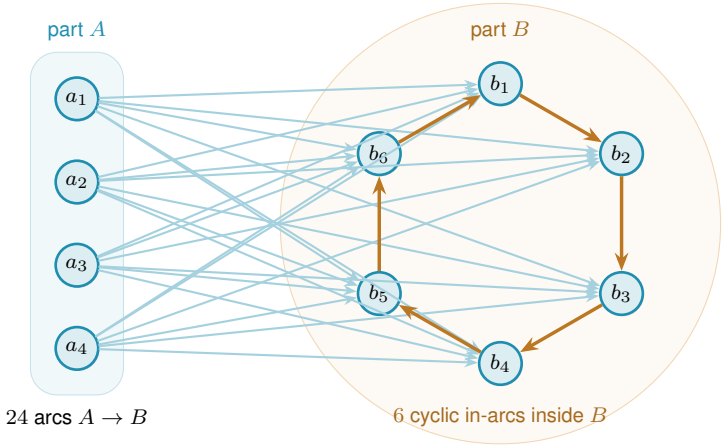


Figure A.2. The augmented bipartite digraph of Construction A.10 at $m = 3$, $n = 10$. All arcs between the parts go from A to B. Inside B, each vertex gets one arc from the previous vertex on a cycle. A pair a_i, b_j has the direct arc and one detour, so $\lambda^{\max} = 2$. The digraph has $24 + 6 = 30$ arcs, against 27 for the hub.

Lemma A.12 (A sparse reachability skeleton always exists)**.** *Every loopless directed multigraph D on $n \geq 2$ vertices has a reachability skeleton R with $|A(R)| \leq M(n)$.*

Proof. We build R in three steps: first inside each strongly connected component, then between components, and then we count. A *strongly connected component* is a maximal set of vertices in which any two can reach each other. Every digraph splits into such components. Figure A.3 shows an example.

Step 1: inside each component. Take a component with n_i vertices and choose a vertex r in it. Since r can reach every vertex of the component, there is an *out-tree*: $n_i - 1$ arcs, all pointing away from r, along which r reaches every vertex. (Grow it one arc at a time.) In the same way there is an *in-tree* of $n_i - 1$ arcs, along which every vertex reaches r. Any two vertices u, v of the component are then joined by a walk $u \to r \to v$ in these two trees, which contains a path. So at most $2(n_i - 1)$ arcs keep all reachability inside the component.

Step 2: between components. Replace each component by one point. Put an arc from one point to another if D has an arc from the first component to the second. This gives a digraph Q_0 on q points, one per component. Q_0 has no directed cycle, since a cycle would merge components. Now delete arcs from Q_0 one at a time, as long as this does not change who can reach whom. Call the result Q.

The graph Q, with directions ignored, has no triangle. Indeed, suppose three points x, y, z of Q are joined in pairs. Since Q has no directed cycle, the three arcs must look like $x \to y$, $y \to z$ and $x \to z$, after renaming. But then $x \to z$ could be deleted, since x still reaches z through y. This contradicts the choice of Q. By Mantel's theorem [21, 22], a triangle-free graph on q vertices has at most $\lfloor q^2/4 \rfloor$ edges. So

$$|A(Q)| \;\leq\; \left\lfloor \frac{q^2}{4} \right\rfloor .$$

Step 3: counting. Let R consist of the two trees of Step 1 in every component, and, for each

arc of Q, one arc of D between the corresponding components. By Step 1, reachability inside each component is kept. By Step 2, reachability between components is kept: a route can follow the arcs of Q and move inside each component along the trees. Since $R \subseteq D$, R creates no new reachability. So R is a reachability skeleton. If the components have sizes $n_1, \dots, n_q$, then

$$|A(R)| \;\le\; \sum_{i=1}^{q} 2(n_i - 1) \;+\; |A(Q)| \;\le\; 2(n-q) \;+\; \left\lfloor \frac{q^2}{4} \right\rfloor \;=:\; f(q).$$

It remains to bound $f(q)$ for $1 \le q \le n$. Since $\lfloor (q+1)^2/4 \rfloor - \lfloor q^2/4 \rfloor = \lfloor (q+1)/2 \rfloor$ (check even and odd q),

$$f(q+1) - f(q) \;=\; -2 \;+\; \left\lfloor \frac{q+1}{2} \right\rfloor.$$

These differences never decrease, so f is convex. A convex function on $\{1, \dots, n\}$ is largest at $q = 1$ or at $q = n$. Since $f(1) = 2(n-1)$ and $f(n) = \lfloor n^2/4 \rfloor$, we get $f(q) \le M(n)$. □

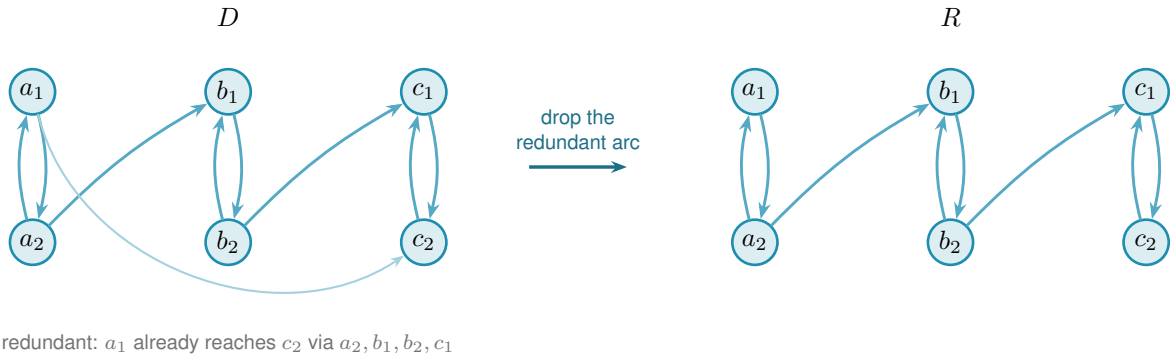


Figure A.3. The construction of Lemma A.12. Each bidirected pair A, B, C is a strongly connected component. Contracting the pairs gives $A \to B \to C$ and the redundant arc $A \to C$. Delete that arc to obtain the triangle-free digraph Q. The skeleton R keeps the internal arcs of each pair and the two remaining connections between pairs. Although $a_1 \to c_2$ is removed, a_1 still reaches c_2 through B.

Lemma A.13 (The augmented bipartite digraph is feasible)**.** *For $m \ge 2$ and $n \ge m$, the digraph of Construction A.10 satisfies $\lambda^{\max} = m - 1$ and has $\lfloor (n+m-2)^2/4 \rfloor$ arcs. Hence $\ell_m^{\mathrm{dir}}(n) \ge \lfloor (n+m-2)^2/4 \rfloor$.*

Proof. Each vertex of B receives one arc from every vertex of A and $m-2$ arcs from inside B. So the number of arcs is $|B|\,(|A| + m - 2)$. Here $|A| + m - 2 = \lfloor (n+m-2)/2 \rfloor$ and $|B| = \lceil (n+m-2)/2 \rceil$, and $\lfloor N/2 \rfloor \lceil N/2 \rceil = \lfloor N^2/4 \rfloor$ for every integer N.

Now take $a \in A$ and $b \in B$. The direct arc and the $m-2$ routes $a \to b' \to b$ through the in-neighbours b' of b in B are arc-disjoint. No route from a can enter A again. So every a–b route ends with $a \to b$ or with one of the $m-2$ arcs into b from B. Removing these $m-1$ arcs separates a from b, so $\lambda(a,b) = m-1$. For $b_i, b_j \in B$, a route from b_i stays in B, so it ends with one of the $m-2$ arcs into b_j from B, and $\lambda(b_i, b_j) \le m-2$. There are no routes from B to A or inside A. So $\lambda^{\max} = m - 1$. □

A.4 Simple digraphs beyond the first threshold

A.4.1 The two-step budget

Definition A.14 (Two-step budget)**.** Let D be a simple digraph and $C \geq 1$. For $u \neq v$, let $p_2(u,v)$ be the number of vertices $x \notin \{u,v\}$ with arcs $u \to x$ and $x \to v$. Such an x is the *midpoint* of the route $u \to x \to v$. We call D *C-budgeted* if every ordered pair (u,v) of distinct vertices satisfies

$$\mathbf{1}_{(u,v)\in A(D)} + p_2(u,v) \;\leq\; C.$$

Lemma A.15 (Feasible digraphs are budgeted)**.** *Let D be a simple digraph and $m \geq 2$. If $\lambda_D^{\max} \leq m-1$, or if $\kappa_D^{\max} \leq m-1$, then D is $(m-1)$-budgeted.*

Proof. Fix an ordered pair (u,v). Take the arc $u \to v$, if it exists, and the route $u \to x \to v$ for each of the $p_2(u,v)$ midpoints x. Two routes with different midpoints use different arcs, and none of them uses the arc $u \to v$. So these routes are arc-disjoint. Each two-step route has one inner vertex, its midpoint, and the arc has none. So the routes are also internally vertex-disjoint. So their number is at most $\lambda_D(u,v)$ and at most $\kappa_D(u,v)$, and either condition gives at most $m-1$. □

The next lemma turns the budget into a statement about degrees. A vertex with large in-degree and large out-degree is the midpoint of many two-step routes, and the budget limits these.

Lemma A.16 (The budget, read on degrees)**.** *Every C-budgeted simple digraph D on n vertices satisfies*

$$\sum_{x\in V} d^+(x)\, d^-(x) \;\leq\; C\, n(n-1).$$

Proof. Count the walks $u \to x \to v$ of length two. Choosing the middle vertex x, an in-neighbour u and an out-neighbour v gives $\sum_x d^-(x)\, d^+(x)$ walks. The walks with $u \neq v$ number $\sum_{u\neq v} p_2(u,v)$. A walk with $u = v$ is fixed by its first arc $u \to x$, so there are at most $|A(D)|$ of them. Summing the budget over all ordered pairs gives $|A(D)| + \sum_{u\neq v} p_2(u,v) \leq C\, n(n-1)$. So

$$\sum_{x} d^+(x)\, d^-(x) \;\leq\; \sum_{u\neq v} p_2(u,v) + |A(D)| \;\leq\; C\, n(n-1).$$ □

Lemma A.17 (Deleting the smaller half of every vertex)**.** *Every simple digraph D on n vertices can be made* one-way bipartite*, so that all remaining arcs go from one part to the other, by deleting at most $\sum_x \min(d^+(x), d^-(x))$ arcs. So*

$$|A(D)| \;\leq\; \left\lfloor \frac{n^2}{4} \right\rfloor \;+\; \sum_{x\in V} \min\bigl(d^+(x), d^-(x)\bigr).$$

Proof. Call x *source-like* if $d^+(x) \geq d^-(x)$, and *sink-like* otherwise. Let S and T be these two sets. Delete every arc into a source-like vertex and every arc out of a sink-like vertex. At each

vertex x this deletes $\min(d^+(x), d^-(x))$ arcs, so in total at most the sum above. An arc $a \to b$ survives only if $a \in S$ and $b \in T$. So the remaining arcs all go from S to T, and there are at most $|S|\,|T| \le \lfloor n^2/4 \rfloor$ of them. □

Lemma A.18 (The counting core)**.** *Every C-budgeted simple digraph D on $n \ge 2$ vertices satisfies*

$$|A(D)| \;\le\; \left\lfloor \frac{n^2}{4} \right\rfloor \;+\; 4C(n-1).$$

Proof. We use induction on n. For $n = 2$ a simple digraph has at most 2 arcs, and the right side is $1 + 4C \ge 5$. Let $n \ge 3$.

First suppose some vertex v has total degree $d(v) < n/2$, so $d(v) \le \lfloor n/2 \rfloor$. Deleting v only removes routes, so $D - v$ is still C-budgeted. By induction, and since $\lfloor (n-1)^2/4 \rfloor + \lfloor n/2 \rfloor = \lfloor n^2/4 \rfloor$ (check even and odd n),

$$\begin{aligned} |A(D)| &\le \left\lfloor \frac{(n-1)^2}{4} \right\rfloor + 4C(n-2) + \left\lfloor \frac{n}{2} \right\rfloor \\ &= \left\lfloor \frac{n^2}{4} \right\rfloor + 4C(n-2) \le \left\lfloor \frac{n^2}{4} \right\rfloor + 4C(n-1). \end{aligned}$$

Otherwise $d(x) \ge n/2$ for every vertex. For $a, b \ge 0$ with $a + b > 0$ we have $\min(a,b) \le 2ab/(a+b)$, since $\max(a,b) \ge (a+b)/2$. With $a = d^+(x)$, $b = d^-(x)$ and Lemma A.16 this gives

$$\sum_x \min\bigl(d^+(x), d^-(x)\bigr) \;\le\; \frac{2}{\min_x d(x)} \sum_x d^+(x) d^-(x) \;\le\; \frac{4}{n}\, C\, n(n-1) \;=\; 4C(n-1),$$

and Lemma A.17 gives the claim. □

We can now prove Theorem 1.17.

Proof of Theorem 1.17. By Lemma A.15 a simple digraph with $\lambda_D^{\max} \le m-1$ or with $\kappa_D^{\max} \le m-1$ is $(m-1)$-budgeted, so Lemma A.18 at $C = m-1$ gives the stated bound.

For the lower bound, Construction A.10 has $\lambda^{\max} \le m-1$, so also $\kappa^{\max} \le m-1$. For $n \ge m$ this gives

$$k_m^{\mathrm{dir}}(n) \;\ge\; \ell_m^{\mathrm{dir}}(n) \;\ge\; \left\lfloor \frac{(n+m-2)^2}{4} \right\rfloor \;=\; \frac{n^2}{4} + \frac{(m-2)n}{2} + O_m(1).$$

So for $m \ge 3$ the second term is linear in n on both sides. □

The linear coefficient is not known. The construction gives $(m-2)n/2$, and Theorem 1.17 allows $4(m-1)n$. For large n a theorem of Huang and Lyu [13, Theorem 2], with $t = m-1$, lowers this to $(m-1)n$. It applies because a feasible digraph has at most $m-1$ paths of length two between any two vertices (Lemma A.15). Our bound is weaker, but it holds for every n.

The next proposition is about the shape of a feasible digraph. After deleting $O(\sqrt{m}\, n^{3/2})$ arcs, all remaining arcs go from one part to the other. The parts need not be balanced. For fixed m this error is larger than the linear error of Theorem 1.17.

Proposition A.19 (Almost all arcs go one way)**.** *Let D be a simple digraph on $n \geq 2$ vertices with $\lambda_D^{\max} \leq m-1$. Then deleting at most $\sqrt{m}\, n^{3/2}$ arcs leaves D a subgraph of a one-directional bipartite digraph, and*

$$|A(D)| \;\leq\; \left\lfloor \frac{n^2}{4} \right\rfloor \;+\; \sqrt{m}\, n^{3/2}.$$

Proof. By Lemma A.15, D is $(m-1)$-budgeted. We use $\min(a,b) \leq \sqrt{ab}$, the Cauchy–Schwarz inequality $\sum_x \sqrt{a_x} \leq \sqrt{n \sum_x a_x}$, and Lemma A.16:

$$\begin{aligned} \sum_{x \in V} \min\bigl(d^+(x), d^-(x)\bigr) \;&\leq\; \sum_{x \in V} \sqrt{d^+(x)\, d^-(x)} \;\leq\; \sqrt{n \sum_{x \in V} d^+(x)\, d^-(x)} \\ &\leq\; \sqrt{n \cdot (m-1)\, n(n-1)} \;=\; n\sqrt{(m-1)(n-1)} \;\leq\; \sqrt{m}\, n^{3/2}. \end{aligned}$$

Lemma A.17 now gives both statements. □

So for $m \geq 3$ both directed problems have leading term $n^2/4$ and a linear second term. The linear coefficient and the exact values are open. It is also open whether the two values are equal.

A.5 The extremal directed multigraphs

At $m = 2$ a feasible directed multigraph has no parallel arcs. So Theorem 1.18 at $m = 2$ is Theorem 1.11.

Theorem 1.18 gives the value, but not the extremal multigraphs. This subsection describes them for large n. The idea is to look at the proof of Lemma A.12 in the case of equality. Equality fixes the number of strongly connected components, then the shape of the skeleton, and then the multigraph.

Lemma A.20 (What equality forces)**.** *Let $m \geq 2$ and $n \geq 8$, so that $\lfloor n^2/4 \rfloor > 2(n-1)$ and $M(n) = \lfloor n^2/4 \rfloor$, and let D be a directed multigraph on n vertices with $\lambda_D^{\max} \leq m-1$ and $|A(D)| = (m-1)\lfloor n^2/4 \rfloor$. Then*

(1) D is acyclic*: it has no directed cycle, and*

(2) D has a reachability skeleton R from which no arc can be removed without changing who reaches whom, and which, with directions ignored, is the complete bipartite graph $K_{\lceil n/2 \rceil, \lfloor n/2 \rfloor}$.

Proof. The proof of Theorem 1.18 removes skeletons $R_1, \ldots, R_{m-1}$, and then no arc is left. So $|A(D)| = \sum_i |A(R_i)|$. Each $|A(R_i)| \leq M(n)$, so equality forces $|A(R_i)| = M(n)$ for every i. Let $R := R_1$.

In the proof of Lemma A.12, $|A(R)| \le f(q) = 2(n-q) + \lfloor q^2/4 \rfloor$, where q is the number of strongly connected components. So $f(q) \ge \lfloor n^2/4 \rfloor = f(n)$. Since $n \ge 8$, $f(1) = 2(n-1) < \lfloor n^2/4 \rfloor = f(n)$. The function f is convex, so between 1 and n it stays below the straight line from $f(1)$ to $f(n)$. That line lies below $f(n)$ except at $q = n$, so $f(q) < f(n)$ for $q < n$. So $q = n$: every strongly connected component is a single vertex. This is (1).

With $q = n$ the trees of Step 1 are empty, so R consists of the arcs chosen for Q, and $|A(Q)| = \lfloor n^2/4 \rfloor$. The graph Q is triangle-free, and a triangle-free graph on n vertices with $\lfloor n^2/4 \rfloor$ edges is $K_{\lceil n/2 \rceil, \lfloor n/2 \rfloor}$ (the equality case of Mantel's theorem [21]). This is (2). The skeleton R is acyclic because D is, and no arc of R can be removed without changing reachability, because the same holds for Q. □

The next lemma uses that no arc of R can be removed.

Lemma A.21 (A minimal bipartite skeleton has no long path)**.** *Let R be as in Lemma A.20, with parts A and B. Then R contains no directed path with three arcs.*

Proof. Suppose $u \to v \to w \to x$ is such a path. The path alternates between the parts, so u and x lie in different parts. The underlying graph of R is complete bipartite, so R has an arc between u and x. It cannot be $x \to u$, since that would close a directed cycle, and D is acyclic. So $u \to x$ is in R. But u also reaches x along the path, so $u \to x$ could be removed without changing reachability. This is a contradiction. □

Theorem A.22 (Extremal classification on the quadratic branch)**.** *Let $m \ge 2$ and $n \ge \max(8, 2m)$, so that $M(n) = \lfloor n^2/4 \rfloor$ and n is on the quadratic branch. If D is a directed multigraph on n vertices with $\lambda_D^{\max} \le m-1$ and $|A(D)| = L_m^{\mathrm{dir}}(n) = (m-1)\lfloor n^2/4 \rfloor$, then*

$$D \in \left\{ (m-1) \cdot B_{\lceil n/2 \rceil, \lfloor n/2 \rfloor},\ (m-1) \cdot B_{\lfloor n/2 \rfloor, \lceil n/2 \rceil} \right\},$$

where $B_{a,b}$ has all arcs from its part of size a to its part of size b, each once, and $(m-1) \cdot B_{a,b}$ takes every arc $m-1$ times as in Lemma 1.19. These two digraphs are the same up to renaming the vertices when n is even, and reverses of each other when n is odd.

Proof. Take R, A and B from Lemma A.20, and write $\alpha = |A| \ge \beta = |B| = \lfloor n/2 \rfloor$.

Step 1: R has no directed path with two arcs. Suppose $a \to b \to a'$ with $a, a' \in A$ and $b \in B$. If a had an incoming arc $b_0 \to a$, then $b_0 \to a \to b \to a'$ would be a path with three arcs, which Lemma A.21 forbids. So a has no incoming arc. Since the underlying graph is complete bipartite, $a \to b''$ for every $b'' \in B$. In the same way a' has no outgoing arc, so $b'' \to a'$ for every $b'' \in B$. The β routes $a \to b'' \to a'$ use different arcs. So $\lambda_D(a, a') \ge \beta = \lfloor n/2 \rfloor \ge m$, which breaks the condition. A path $b \to a \to b'$ is excluded in the same way, since $\alpha \ge \beta \ge m$. This is the only step that uses $n \ge 2m$.

Step 2: all arcs of R go the same way. By Step 1, no vertex of R has both an incoming and an outgoing arc. Take $a \in A$. It is joined to every vertex of B. If all arcs at a go out, then every $b \in B$ has an incoming arc, so it has no outgoing arc. So all arcs go from A to B. If all arcs at a

come in, all arcs go from B to A in the same way. From now on, call the part that the arcs leave A, and the other part B.

Step 3: D has no arc outside R. In R, the only pairs (u, x) with u reaching x are the pairs in $A \times B$, and each such pair is an arc of R. If D has an arc $u \to x$, then u reaches x in D, so also in R. So $u \to x$ is an arc of R.

Step 4: every multiplicity is $m - 1$. Parallel arcs are disjoint routes, so every multiplicity is at most $m - 1$. There are $|A||B| \leq \lfloor n^2/4 \rfloor$ pairs, and $|A(D)| = (m-1)\lfloor n^2/4 \rfloor$. So every pair has multiplicity $m - 1$, and $|A||B| = \lfloor n^2/4 \rfloor$, so the parts have sizes $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$. □

The hypothesis $n \geq 2m$ is used only in Step 1. Every vertex of an extremiser has degree at least $(m-1)\lfloor n/2 \rfloor$, because deleting it leaves at most $(m-1)M(n-1)$ arcs. For even n, every vertex has degree exactly $(m-1)n/2$.

Part III. Hypergraphs

Menger for hypergraphs, simple hypergraphs that attain the edge bound, the vertex problem at $m = 3$, multigraphs with vertex separation, and directed hypergraphs.

A.6 The hypergraph bounds

There are several notions of connectivity in hypergraphs, for example in Chekuri and Xu [23] and Dewar, Pike and Proos [24]. This thesis uses Berge paths throughout. We first need Menger's theorem for Berge paths. It also shows that the gate network of Chapter 2 computes the right value.

Theorem A.23 (Menger for multihypergraphs)**.** *Let u, v be distinct vertices of a multihypergraph $\mathcal{H}$, and count parallel copies of a hyperedge separately. The largest number of u–v Berge paths that share no hyperedge copy equals the smallest number of hyperedge copies whose removal separates u from v.*

Proof. We turn $\mathcal{H}$ into a directed multigraph N, the gate network of Figure 2.4b, and use Menger's theorem for directed multigraphs (Theorem 1.1). Let c be the total number of hyperedge copies. Every hyperedge e with $\mu(e)$ copies gets two new nodes e^- and e^+ and $\mu(e)$ parallel arcs $e^- \to e^+$, its *gate*. Every vertex $x \in e$ gets $c + 1$ parallel arcs $x \to e^-$ and $c + 1$ parallel arcs $e^+ \to x$.

Paths. A Berge path $u, e_1, v_1, \ldots, e_k, v$ becomes the directed path $u \to e_1^- \to e_1^+ \to v_1 \to \cdots \to v$, and back. In a family of Berge paths that share no copy, at most $\mu(e)$ paths use e, so each can take its own gate arc. There are enough copies of the other arcs. So arc-disjoint directed paths in N and Berge paths that share no copy come in families of the same sizes.

Cuts. Removing all c gate arcs separates u from v. So a smallest separating set of arcs in N has at most c arcs, and never removes all $c+1$ copies of a non-gate arc. Removing only some copies changes nothing, so a smallest separating set consists of gate arcs only, and it takes all gate arcs of a hyperedge or none. Such a set separates u from v in N exactly when the same hyperedge copies separate u from v in $\mathcal{H}$.

So both sides of Menger's theorem have the same value in N and in $\mathcal{H}$. □

The same proof works for directed hyperedges and directed Berge paths, with arcs $x \to e^-$ only for the tails x of e, and arcs $e^+ \to x$ only for its heads.

The next theorem shows that for $r \geq 3$ and $m - 1 \leq \binom{n-2}{r-2}$, simple hypergraphs attain the same bound. For graphs ($r = 2$) this is different: there, simplicity lowers the bound from $(m-1)(n-1)$ to $\lfloor m(n-1)/2 \rfloor$.

Theorem A.24 (Simplicity is free for $r \geq 3$)**.** *Let $r \geq 3$, $n \geq r$, and $2 \leq m$ with $m - 1 \leq \binom{n-2}{r-2}$. Then the maximum number of hyperedges in a* simple *r-uniform hypergraph on n vertices with $\lambda^{\max} \leq m - 1$ is exactly $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$.*

Proof. The upper bound is Theorem 1.13. For the lower bound, take a hub h and let $V' = V \setminus \{h\}$. By Lemma A.1 with uniformity $r-1$ and degree bound $d = m-1$ on the $n-1$ vertices of V', there is a simple $(r-1)$-uniform hypergraph $\mathcal{G}^*$ on V' with maximum degree at most $m-1$ and $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$ hyperedges. Add h to each of its hyperedges. The new hyperedges are different r-sets, so the result $\mathcal{H}$ is simple. Every vertex $w \neq h$ lies in at most $m-1$ hyperedges, and every route from or to w uses one of them. Every pair contains such a vertex, so $\lambda_{\mathcal{H}}^{\max} \leq m-1$. □

Remark A.25 (The condition is needed). Without the condition $m - 1 \leq \binom{n-2}{r-2}$, simple hypergraphs may not reach the bound. The smallest example is $n = r = m = 3$. There is only one 3-set, so a simple hypergraph has at most one hyperedge, while the bound is $\lfloor 2 \cdot 2/2 \rfloor = 2$. Two copies of the one hyperedge do reach the bound. Theorem 1.16 needs the same condition with $m - 1 = 2$, which is the range $2 < r < n$.

Construction A.26 (Twelve hyperedge copies on six vertices)**.** Take a hub h and five other vertices a, b, c, d, e. Include all ten triples consisting of h and two of the other vertices. Add a second copy of $\{h, d, e\}$ and one copy of $\{a, b, c\}$. The resulting 3-uniform multihypergraph has 12 hyperedge copies and $\lambda^{\max} \leq 5$. Consequently, the multihypergraph edge value at $(n, m, r) = (6, 6, 3)$ is exactly 12, even though $(r-1)$ does not divide $(n-1)$.

Proof. Initially each non-hub vertex lies in four of the ten triples. The repeated triple raises the degrees of d and e to five, while the final triple does the same for a, b and c. Every pair of distinct vertices therefore includes a non-hub vertex whose five incident hyperedge copies form a cut separating it from the other endpoint. Thus no pair has more than five hyperedge-disjoint routes. The construction has $10 + 1 + 1 = 12$ copies, matching the upper bound $\lfloor 5(6-1)/(3-1) \rfloor = 12$ from Theorem 1.13. □

Since $\kappa \leq \lambda$, the same multihypergraph also gives 12 for the vertex problem. We do not know whether 12 is optimal there.

A.7 The hypergraph vertex problem

The vertex problem for hypergraphs becomes a problem about graphs (Figure A.4). Recall the *incidence graph* $I(\mathcal{H})$: one node per vertex, one node per hyperedge, and an edge when the vertex lies in the hyperedge. A Berge path is the same as a path in $I(\mathcal{H})$ between two vertex nodes. Its inner nodes are the hyperedges it uses and the vertices where it changes hyperedge. So two Berge routes share no hyperedge and no inner vertex exactly when the two paths in $I(\mathcal{H})$ are internally disjoint. So

$$\kappa_{\mathcal{H}}(u, v) \;=\; \kappa_{I(\mathcal{H})}(u, v).$$

The vertex problem asks for the largest number of hyperedge nodes, each of degree r, such that no two of the n vertex nodes are joined by m internally disjoint paths.

Proposition A.27 (General lower bound)**.** *For $r \geq 3$, $m \geq 2$ and $m - 1 \leq \binom{n-2}{r-2}$,*

$$k_m^{(r)}(n) \;\geq\; \left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor,$$

attained by a simple hypergraph.

Proof. The simple hypergraph of Theorem A.24 has $\lambda^{\max} \leq m - 1$. Since $\kappa \leq \lambda$, it is also feasible for the vertex problem. □

That construction needs different $(r-1)$-sets, which explains the condition $m - 1 \leq \binom{n-2}{r-2}$. A multihypergraph can repeat a hyperedge instead. This covers other cases.

Proposition A.28 (The multihypergraph lower bound)**.** *For $r \geq 2$, $m \geq 2$ and $(r-1) \mid (n-1)$,*

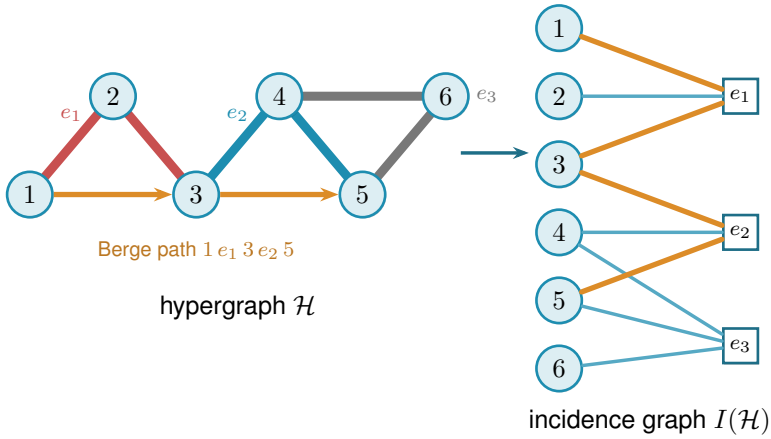
$$K_m^{(r)}(n) \;\geq\; \frac{(m-1)(n-1)}{r-1},$$

attained by the star hypertree of Figure 1.9 with every hyperedge taken at multiplicity $m - 1$. At $m = 3$ this meets the upper bound of Theorem 1.16, so

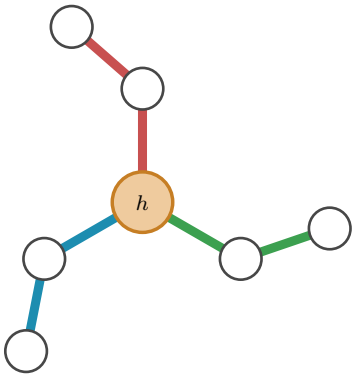
$$K_3^{(r)}(n) \;=\; \left\lfloor \frac{2(n-1)}{r-1} \right\rfloor \qquad \textit{whenever } (r-1) \mid (n-1).$$

Proof. Take the hub h and split the other $n - 1$ vertices into groups $G_1, \ldots, G_q$ of size $r - 1$, where $q = (n-1)/(r-1)$. Give each hyperedge $\{h\} \cup G_i$ multiplicity $m-1$. This gives $(m-1)q$ hyperedges.

Now take two vertices $u \neq w$. Suppose first that both lie in $\{h\} \cup G_i$. One of them lies in G_i, and every route from or to it uses one of the $m - 1$ copies of $\{h\} \cup G_i$. So $\kappa(u, w) \leq \lambda(u, w) \leq m - 1$. Now suppose $u \in G_i$ and $w \in G_j$ with $i \neq j$. Every route from u to w passes through h, so $\kappa(u, w) \leq 1$. So the hypergraph is feasible.



(a) A 3-uniform hypergraph $\mathcal{H}$ (left) and its bipartite incidence graph $I(\mathcal{H})$ (right): circles for vertices, hollow squares for hyperedges, an edge when the vertex lies in the hyperedge. The Berge path $1\, e_1\, 3\, e_2\, 5$ is traced in orange in both pictures. In $\mathcal{H}$ it rides the red line e_1 and the blue line e_2, changing at the shared vertex 3, while the grey line e_3 lies off it. In $I(\mathcal{H})$ it is the ordinary path $1-e_1-3-e_2-5$ alternating sides.



(b) A star hypertree: every hyperedge $\{h\} \cup G_i$ shares only the hub h (orange), and the other $r-1=2$ vertices of each block sit in a single hyperedge. Its incidence graph is a tree, so $\kappa^{\max} \leq 1$.

Figure A.4. The translation the vertex problem rests on. Internally vertex-disjoint Berge paths in $\mathcal{H}$ correspond exactly to internally disjoint paths in $I(\mathcal{H})$, so $\kappa_{\mathcal{H}}(u,v) = \kappa_{I(\mathcal{H})}(u,v)$. **(top)** A hypergraph and its incidence graph with one Berge path drawn in both. **(bottom)** The star hypertree of Theorem 1.15, whose incidence graph is a tree.

At $m = 3$ there are $2(n-1)/(r-1)$ hyperedges, which equals the upper bound of Theorem 1.16. □

The upper bound at $m = 3$ follows from a bound on the cycles of a graph, proved next. For a connected multigraph G, the *cycle rank* is $\operatorname{rank}(G) = |E(G)| - |V(G)| + 1$. A tree has cycle rank 0, and each extra edge raises it by one.

We use the words cut vertex and k-connected from Section A.1.1. A *separating pair* is a pair of vertices whose removal disconnects the graph. By Menger's theorem [4, Ch. 3], a graph is 3-connected exactly when every two vertices are joined by 3 internally disjoint paths.

Lemma A.29 (Incidence rank lemma)**.** *Let G be a connected loopless multigraph with a partition $V(G) = X \dot\cup Z$ such that (i) no two vertices of Z are adjacent, (ii) no edge incident to Z is parallel to another edge, (iii) every $z \in Z$ has degree at least 2, and (iv) $\kappa_G(x, x') \le 2$ for all distinct $x, x' \in X$. Then $\operatorname{rank}(G) \le |X| - 1$.*

Read X as the vertices of a hypergraph and Z as its hyperedges, so that G is the incidence graph. Then (i) says that two hyperedge nodes are never adjacent, (ii) that no hyperedge repeats a vertex, (iii) that every hyperedge has at least two vertices, and (iv) that the hypergraph is feasible at $m = 3$. We allow edges inside X, because Step 2 of the proof creates them.

Proof. We use induction on $|V|+|E|$. Step 1 handles cut vertices. Steps 2 and 3 remove vertices of degree two. Step 4 handles what is left: a 2-connected graph with minimum degree at least three.

Base: $|V| \le 2$. A vertex in Z has degree at least 2 by (iii), and by (ii) its edges go to different vertices. So with at most two vertices, Z is empty and all vertices lie in X. With one vertex, $\operatorname{rank} = 0 = |X| - 1$. With two vertices, the parallel edges between them are internally disjoint routes, so by (iv) there are at most two, and $\operatorname{rank} \le 1 = |X| - 1$.

Step 1: a cut vertex. Suppose G has a cut vertex c. Split the components of $G - c$ into two non-empty groups. Let G_1 and G_2 consist of c together with the first and the second group, with all their edges. They share only the vertex c, so $\operatorname{rank}(G) = \operatorname{rank}(G_1) + \operatorname{rank}(G_2)$. Each G_i is connected and smaller than G, and (i), (ii) and (iv) still hold in it. Condition (iii) can only fail at c, when $c \in Z$ has a single neighbour in G_i. Then delete c from G_i. This removes one vertex and one edge, so the rank and the set of X-vertices do not change. By induction, $\operatorname{rank}(G_i) \le |X \cap V(G_i)| - 1$. The two X-sets together have $|X| + 1$ vertices if $c \in X$, and $|X|$ vertices if $c \in Z$. Either way $\operatorname{rank}(G) \le (|X| + 1) - 2 = |X| - 1$.

Step 2: a Z-vertex of degree two. From now on G has no cut vertex and at least three vertices, so it is 2-connected. Suppose some $z_0 \in Z$ has degree 2. By (i) its two neighbours x, x' lie in X, and by (ii) they are different. *Suppress* z_0: delete it and add a new edge xx'. Then $|E|$ and $|V|$ both drop by one, so the rank does not change. The new edge lies inside X, so $|X|$ does not change, and (i), (ii), (iii) still hold. A route through z_0 becomes a route through the new edge, so every $\kappa(x_1, x_2)$ stays the same and (iv) holds. The graph is smaller, so the induction hypothesis gives the bound.

Step 3: an X-vertex of degree two. Now every Z-vertex has degree at least 3. Suppose some $x_0 \in X$ has degree 2, and delete it. Since G is 2-connected, the result is connected. We remove two edges and one vertex, so the rank drops by one, and $|X|$ drops by one. A neighbour in Z keeps degree at least 2, so (iii) holds. Deleting a vertex cannot raise a connectivity, so (iv) holds. Also $|X| \geq 2$: otherwise, by (i) and (ii), every z would have degree at most 1. By induction, $\mathrm{rank}(G) - 1 \leq (|X| - 1) - 1$.

Step 4: the rest. Now G is 2-connected, has at least three vertices, and every degree is at least 3.

First, G is simple. By (ii), parallel edges can only join two vertices x, x' of X. Suppose $\mu(x, x') \geq 2$. Since x' is not a cut vertex and G has a third vertex, x has a neighbour $w \neq x'$. Since x is not a cut vertex, $G - x$ has a path from w to x'. With x in front, this is an x–x' path with an inner vertex. With the two parallel edges this gives $\kappa(x, x') \geq 3$, against (iv). So G is simple, and since every degree is at least three, $|V| \geq 4$.

If G is 3-connected, then every pair has $\kappa \geq 3$. By (iv), X then has at most one vertex, and by (i) every z has degree at most 1, which is impossible.

So G has a separating pair $\{a, b\}$. Remove it, and divide the remaining components into two non-empty groups with vertex sets U and W. Since G is 2-connected, a and b both have neighbours in U and in W, and each side contains an a–b path. Let $s = |X \cap \{a, b\}|$. We build two smaller graphs G_U and G_W, the *pieces*. G_U is the subgraph on $U \cup \{a, b\}$ with one extra a–b connection, and G_W is built in the same way:

- if $ab \in E(G)$, retain that edge on both sides.
- if $ab \notin E(G)$ and $s \geq 1$, add one new edge ab to each side.
- if $s = 0$, add to each side a fresh X-vertex together with the two edges joining it to a and to b.

These cases cover everything: by (i), if $ab \in E(G)$ then $s \geq 1$. In that case $s = 1$. Indeed, if $a, b \in X$, the edge ab and one a–b path on each side would give $\kappa(a, b) \geq 3$.

Each piece satisfies (i) to (iv). (i): every added edge has an end in X. In the third case this is the new vertex. (ii): an added edge joins two vertices that had no edge. (iii): a Z-vertex in U keeps all its neighbours. If a or b lies in Z, it keeps a neighbour on its side and gains the added connection. (iv): a new X-vertex has degree two, so its connectivity is at most two. For two old X-vertices, internally disjoint paths can share no edge except xx' itself, so at most one path uses the added connection. Replace it by an a–b path through the other side. So three internally disjoint paths in a piece would give three in G, which (iv) forbids.

Both pieces are smaller in $|V| + |E|$. Indeed, neither side is a single vertex: in a simple graph it would have only a and b as neighbours, but every degree is at least three. So a piece loses at least two vertices and three edges, and gains at most one vertex and two edges.

Counting. Let r_U, r_W be the ranks of the pieces and X_U, X_W their X-sets. From $\mathrm{rank} = |E| - |V| + 1$:

- first case: $\operatorname{rank}(G) = r_U + r_W$ and $|X_U| + |X_W| = |X| + 1$.
- second case: $\operatorname{rank}(G) = r_U + r_W - 1$ and $|X_U| + |X_W| = |X| + s$.
- third case: $\operatorname{rank}(G) = r_U + r_W - 1$ and $|X_U| + |X_W| = |X| + 2$.

By induction $r_U \le |X_U| - 1$ and $r_W \le |X_W| - 1$. This gives $\operatorname{rank}(G) \le |X| - 1$ in the first and third cases, and $\operatorname{rank}(G) \le |X| + s - 3 \le |X| - 1$ in the second, since $s \le 2$. □

We now prove Theorem 1.16.

Proof of Theorem 1.16. Apply Lemma A.29 to each component of the incidence graph $I(\mathcal{H})$, with X the vertex nodes and Z the hyperedge nodes. The conditions hold: no two hyperedge nodes are adjacent, a hyperedge contains each vertex once, hyperedge nodes have degree $r \ge 2$, every component contains a vertex node, and $\kappa_I(u, v) = \kappa_{\mathcal{H}}(u, v) \le 2$. Let C be the number of components. $I(\mathcal{H})$ has rq edges and $n + q$ nodes, so the ranks add up to $rq - (n + q) + C$. The bounds add up to $n - C$. So $rq - (n+q) + C \le n - C$, that is, $q(r-1) \le 2(n-1) - 2(C-1) \le 2(n-1)$. The lower bound is Proposition A.27 at $m = 3$. □

Remark A.30 (Edges as gates at $r = 2$, and the boundary of the method). At $r = 2$ the theorem is about multigraphs. It gives $K_3(n) = 2(n-1)$ (Corollary A.34). The proof of Lemma A.29 uses the condition $\kappa \le 2$ in several places. For example, two parallel edges and one more route already break it, and so does a 3-connected graph. The lemma is false for $\kappa \le 3$: two X-vertices joined by three parallel edges have rank two, while $|X| - 1 = 1$. For $m = 4$ a different bound would be needed, probably with a decomposition along sets of three vertices. We do not have one.

A.8 The multigraph incidence problems

At $r = 2$ the hypergraph vertex problems are about multigraphs. They are not the same as the simple problems. Recall that q parallel copies of uv count as q separated routes. Write $K_m(n)$ for the largest number of edges, counted with multiplicity, of a multigraph on n vertices with $\kappa^{\max} \le m - 1$.

The parallel edges and the longer routes do not interfere, so we can count them separately.

Lemma A.31 (Splitting the vertex measure)**.** *Let G be a multigraph with multiplicity function μ and underlying simple graph G_0. For any two vertices $u \ne v$, write $\pi(u, v)$ for the largest number of pairwise internally disjoint u–v paths of length at least two in G_0. Then*

$$\kappa_G(u, v) \;=\; \mu(u, v) \;+\; \pi(u, v).$$

Proof. A route of length one is a copy of the edge uv. It has no inner vertex, so it is separated from every other route.

Lower bound. Take all $\mu(u,v)$ copies of uv, and $\pi(u,v)$ internally disjoint u–v paths of length at least two in G_0. Use one copy of each edge on these paths. The inner vertices do not change, so all these routes are separated.

Upper bound. Take a family of separated u–v routes in G. At most $\mu(u,v)$ of them have length one, since they use different copies of uv. Map each longer route to its sequence of vertices, a path in G_0. Two longer routes have disjoint, non-empty sets of inner vertices. So their images are different and internally disjoint, and there are at most $\pi(u,v)$ of them. □

So a multigraph is feasible exactly when $\mu(u,v) \le m - 1 - \pi(u,v)$ on every edge and $\pi(u,v) \le m-1$ on every pair.

Theorem A.32 (The multigraph incidence value at $m=2$)**.** $K_2(n) = n-1$*, attained exactly by the spanning trees.*

Proof. By Lemma A.31, $\mu(u,v)+\pi(u,v) \le 1$ on every edge. If G_0 had a cycle, an edge uv on it would have $\mu(u,v) \ge 1$ and $\pi(u,v) \ge 1$. So G_0 is a forest and every multiplicity is one. A forest has at most $n-1$ edges, and a spanning tree has exactly $n-1$. □

Remark A.33. At $m=2$ a second copy of an edge is already a second route, so every feasible multigraph is simple. So $K_2(n) = k_2(n) = n-1$. This is also Theorem 1.15 at $r=2$.

The value at $m=3$ follows from the hypergraph theorem.

Corollary A.34 (The multigraph incidence value at $m=3$)**.** $K_3(n) = 2(n-1)$ *for every* $n \ge 2$*.*

Proof. Apply Theorem 1.16 and Proposition A.28 at $r=2$. The divisibility condition is automatic, and the doubled star attains the upper bound $2(n-1)$. □

So at $m=3$ the vertex value equals the edge value $L_3(n)$ of Theorem 1.8. At $m=5$ they differ: the thickened theta below gives $K_5(4) \ge 14$, while $L_5(4) = 12$. For $m=4$ this is open. For $m \ge 3$ there are three constructions.

- *The thickened tree.* Take a spanning tree and give every edge multiplicity $m-1$. Tree edges have $\pi = 0$, and other pairs have $\pi = 1$, so it is feasible. It has $(m-1)(n-1)$ edges, the edge value of Theorem 1.8.

- *The thickened complete graph.* Give every edge of K_n multiplicity q. Here $\pi(u,v) = n-2$, so it is feasible when $q \le m+1-n$. It has $\binom{n}{2}(m+1-n)$ edges, which is positive only when $n \le m$.

- *The thickened theta.* Choose two *poles*. Join each of the other $n-2$ vertices to both poles with multiplicity $m-2$, and join the poles with multiplicity $m+1-n$ if this is positive. A pole and another vertex have $\pi = 1$, two non-poles have $\pi = 2$, and the two poles have $\pi = n-2$. So it is feasible when $n \le m+1$, and it has $2(n-2)(m-2) + \max(0,\, m+1-n)$ edges.

For $n \le m+1$, the theta has as many edges as the thickened tree at $m = 4$, and more for $m \ge 5$ (when $n \ge 3$). For $n \ge m+2$ the other two constructions are not feasible, so among these three the tree is best.

For $m \ge 5$, one triangle added to the thickened tree already beats it, also when $n \ge m+2$. The next result makes this precise.

Fix a connected simple graph G_0. By Lemma A.31, the best multigraph on G_0 takes $\mu(u,v) = m-1-\pi(u,v)$ on every edge. Its size is $(m-1)\,|E(G_0)| - \sum_{uv\in E(G_0)} \pi(u,v)$. Since $|E(G_0)| = n-1+\operatorname{rank}(G_0)$, it beats the thickened tree exactly when

$$\sum_{uv\in E(G_0)} \pi(u,v) \;<\; (m-1)\operatorname{rank}(G_0).$$

Theorem A.35 (Thickened complete graphs sharing a vertex beat the thickened tree)**.** *Let $3 \le s \le m$ and $q = m+1-s \ge 1$. Let $1 \le k \le \lfloor (n-1)/(s-1) \rfloor$. Take k copies of K_s that share one vertex and are otherwise disjoint, and give every edge multiplicity q. Attach the remaining vertices as a path with every edge of multiplicity $m-1$. This multigraph has $\kappa^{\max} \le m-1$, and its number of edges is*

$$(m-1)(n-1) \;+\; k\cdot \operatorname{gain}(s,m), \qquad \operatorname{gain}(s,m) := \frac{(s-1)(s-2)(m-1-s)}{2}.$$

So it beats the thickened tree exactly when $s \le m-2$. Such an s exists when $m \ge 5$.

Proof. Write G_0 for the underlying simple graph.

Feasibility. The copies of K_s and the path edges meet only in cut vertices. A path between two vertices of the same block cannot leave the block, since it would pass a cut vertex twice. Take an edge uv in a copy of K_s. The other $s-2$ vertices of the copy give $s-2$ internally disjoint routes of length two, and they also form a cut, so $\pi(u,v) = s-2$. By Lemma A.31, $\kappa_G(u,v) = q + (s-2) = m-1$. Two vertices in different blocks are separated by a cut vertex, so $\kappa_G(u,v) \le 1$. For an edge uv on the path, $\pi(u,v) = 0$, so $\kappa_G(u,v) = m-1$. Inside a block every two vertices are adjacent, so no other pairs need checking.

Count. Since $q \ge 1$, the underlying simple graph G_0 is connected, so $|E(G_0)| = n - 1 + \operatorname{rank}(G_0)$. Only the k copies of K_s contain cycles or have $\pi > 0$. Each copy has rank $(s-1)(s-2)/2$, and its $\binom{s}{2}$ edges each have $\pi = s-2$. So the number of edges is

$$(m-1)\Big[n-1+k\tfrac{(s-1)(s-2)}{2}\Big] - k\tfrac{s(s-1)(s-2)}{2} \;=\; (m-1)(n-1) + k\operatorname{gain}(s,m).$$

□

At $s = 3$, $\operatorname{gain}(3,m) = m-4$. This is zero at $m = 4$ and positive for $m \ge 5$. So for every $m \ge 5$ the thickened tree is not optimal, also when $n \ge m+2$. Here is the smallest example. Take $m = 5$ and $n = 7$. Put a triangle on $\{u_1, u_2, u_3\}$ with multiplicity 3, and attach a path of four more vertices to u_1 with multiplicity 4. Two triangle vertices have three parallel edges and one

route through the third vertex, so $\kappa = 4$. Every other pair is separated by a cut vertex or joined only by its parallel edges. So $\kappa^{\max} = 4 = m - 1$. There are $3 \cdot 3 + 4 \cdot 4 = 25$ edges, one more than the thickened tree's 24. With k triangles the gain is k, so the gap grows linearly in n.

Sharing a vertex lets more copies fit, since each new copy uses only $s - 1$ new vertices. For example, at $m = 10$, $s = 6$ and $n = 11$, two copies sharing a vertex give 150 edges. Disjoint copies would allow only one copy, with 75 edges, and 120 edges after attaching the other five vertices.

The best s is about $(m + 1)/2$, where the gain per added vertex, $\mathrm{gain}(s, m)/(s - 1)$, is of order m^2. So for $n/m \to \infty$ the construction gives $K_m(n) \geq \Theta(m^2)\, n$, while the thickened tree gives only $(m - 1)(n - 1)$.

This construction need not be optimal. The next result gives an upper bound of the same order.

Proposition A.36 (An upper bound of the same order)**.** *For all $m \geq 2$ and $n \geq 2$,*

$$K_m(n) \;\leq\; (m-1)\, k_m(n) \;<\; 2(m-1)^2\, n.$$

Proof. Let G be feasible, with underlying simple graph G_0.

Every multiplicity is at most $m - 1$. By Lemma A.31, $\kappa_G(u, v) = \mu(u, v) + \pi(u, v) \geq \mu(u, v)$, so feasibility gives $\mu(u, v) \leq m - 1$ on every pair and the total is at most $(m - 1)|E(G_0)|$.

G_0 is feasible for the simple vertex problem. Take $u \neq v$. If they are adjacent, then $\mu(u, v) \geq 1$, and the routes of G_0 between them are the edge uv and the longer routes, so $\kappa_{G_0}(u, v) = 1 + \pi(u, v) \leq \mu(u, v) + \pi(u, v) = \kappa_G(u, v) \leq m - 1$. If they are not adjacent, then $\kappa_{G_0}(u, v) = \pi(u, v) = \kappa_G(u, v) \leq m - 1$. So $|E(G_0)| \leq k_m(n)$, which gives the first inequality.

The second inequality is $k_m(n) < 2(m - 1)n$ from Remark A.7. □

So for $n/m \to \infty$ and then $m \to \infty$,

$$\left(\tfrac{1}{8} + o_m(1)\right) m^2\, n \;\leq\; K_m(n) \;<\; 2m^2 n.$$

This needs n large: for example $K_m(2) = m - 1$. Theorem A.39 below gives a better lower constant. The first inequality, $K_m(n) \leq (m - 1)k_m(n)$, says that $K_m(n)$ is at most $m - 1$ times the simple vertex value. It is tight at $m = 2$ (Remark A.33).

A.8.1 The value is a block problem

The next two results reduce $K_m(n)$ to 2-connected blocks, as in Section A.1.1.

Lemma A.37 (The objective in closed form)**.** *For a simple graph G_0 on n vertices with $\kappa_{G_0}^{\max} \leq m - 1$, the largest total multiplicity of a feasible multigraph with underlying simple graph exactly G_0 is*

$$W_m(G_0) \;:=\; \sum_{uv \in E(G_0)} \big(m - \kappa_{G_0}(u, v)\big),$$

and every simple graph arising as the underlying graph of a feasible multigraph satisfies $\kappa_{G_0}^{\max} \leq m-1$. *Hence*

$$K_m(n) \;=\; \max\bigl\{\, W_m(G_0) \;:\; G_0 \text{ simple on } n \text{ vertices},\ \kappa_{G_0}^{\max} \leq m-1 \,\bigr\}.$$

Proof. By Lemma A.31, a multigraph on G_0 is feasible exactly when $\mu(u,v) \leq m-1-\pi(u,v)$ on every edge, so the best choice is $\mu(u,v) = m-1-\pi(u,v)$. On an edge, $\kappa_{G_0}(u,v) = 1+\pi(u,v)$, so this is $m-\kappa_{G_0}(u,v)$, and the total is $W_m(G_0)$. It is at least one on every edge, since $\kappa_{G_0}(u,v) \leq m-1$, so the underlying graph is really G_0. The proof of Proposition A.36 shows that the underlying graph of a feasible multigraph is feasible. So maximising W_m over feasible G_0 gives $K_m(n)$. □

The objective is a sum over edges of a quantity that depends only on the block of the edge. This gives the next result.

Theorem A.38 (The multigraph incidence problem reduces to blocks)**.** *For a* 2*-connected graph* B*, or a single edge, write*

$$g_m(b) \;:=\; \max\bigl\{\, W_m(B) \;:\; B \text{ 2-connected or an edge},\ |V(B)| = b,\ \kappa_B^{\max} \leq m-1 \,\bigr\},$$

with $g_m(b) := -\infty$ *when no such* B *exists. Then for all* $m \geq 2$ *and* $n \geq 2$*,*

$$K_m(n) \;=\; \max\Bigl\{ \sum_i g_m(b_i) \;:\; b_i \geq 2,\ \sum_i (b_i - 1) \leq n-1 \Bigr\},$$

the maximum over all finite multisets of block sizes. In particular the value is determined by the finitely many numbers $g_m(2), \ldots, g_m(n)$*.*

Proof. This is the proof of Theorem A.6 with W_m in place of the number of edges. As there, $\kappa_{G_0}(u,v) = \kappa_B(u,v)$ for u, v in a common block B, so $m-\kappa_{G_0}(u,v)$ depends only on the block of the edge uv, and $W_m(G_0) = \sum_B W_m(B)$. The block sizes satisfy $\sum_B (|V(B)|-1) \leq n-1$, and gluing blocks at one common vertex realises every choice of sizes, as there. With Lemma A.37 this gives the claim. □

So $K_m(n)$ can be computed from 2-connected graphs: compute W_m for each, and choose the best block sizes.

Theorem A.39 (Thickened complete bipartite blocks)**.** *Let* $1 \leq s \leq t \leq m-1$*. Thicken every edge of* $K_{s,t}$ *to multiplicity* $m-s$*. The result is feasible,* $\kappa^{\max} \leq m-1$*, on* $s+t$ *vertices with total multiplicity* $st(m-s)$*. Hanging* k *such blocks off one shared vertex gives, for* $n = k(s+t-1)+1$*,*

$$K_m(n) \;\geq\; \frac{st\,(m-s)}{s+t-1}\,(n-1).$$

Optimised over s *and* t *this rate is* $\bigl(3-2\sqrt{2}+o_m(1)\bigr)m^2$*, attained at* $t = m-1$ *and* $s \sim (\sqrt{2}-1)m$*, against the* $\bigl(\frac{1}{8}+o_m(1)\bigr)m^2$ *of Theorem A.35. The improvement is by a factor* $8(3-2\sqrt{2}) =$

$24 - 16\sqrt{2} \approx 1.373$.

Proof. Write S and T for the two sides, $|S| = s$ and $|T| = t$.

The connectivities of $K_{s,t}$. For $u \neq u'$ in S, the t two-step routes through distinct vertices of T are internally disjoint. Every route meets T, so $\kappa(u, u') = t$. Symmetrically $\kappa(v, v') = s$ for $v \neq v'$ in T. For adjacent $u \in S$ and $v \in T$, delete the edge uv. The sets $T \setminus \{v\}$ and $S \setminus \{u\}$ both separate u from v, so $\pi(u, v) \leq \min(s-1, t-1) = s-1$. Choose distinct $u_i \in S \setminus \{u\}$ and distinct $v_i \in T \setminus \{v\}$ for $1 \leq i \leq s-1$, possible because $s \leq t$. The routes u, v_i, u_i, v attain the bound. Thus $\pi(u, v) = s - 1$ and $\kappa(u, v) = s$.

Feasibility and count. By Lemma A.31 the multigraph has $\kappa = (m - s) + (s - 1) = m - 1$ on an edge, and $\kappa = t$ or s, both at most $m-1$, on a non-adjacent pair. It has st edges of multiplicity $m - s$.

The bouquet. If $s \geq 2$, then $K_{s,t}$ is 2-connected, so Theorem A.38 applies and k copies at one shared vertex are feasible with total $k\,st(m - s)$ on $n = k(s + t - 1) + 1$ vertices. If $s = 1$, then $K_{1,t}$ is a tree (a single edge when $t = 1$), and after thickening every edge to multiplicity $m - 1$, sharing vertices among copies still produces a thickened tree and is feasible directly. Its count is again $k\,t(m-1) = k\,st(m-s)$. Thus the displayed rate holds throughout the theorem's full range $1 \leq s \leq t$.

The optimum. For fixed $s > 1$, the factor $t/(s + t - 1)$ is strictly increasing in t, and for $s = 1$ it is constant. Thus some optimum has $t = m - 1$, its largest permitted value. Put $s = \alpha m$. The rate becomes

$$\frac{s\,(m-1)(m-s)}{s+m-2} \;=\; \frac{\alpha(1-\alpha)}{1+\alpha}\,m^2 \;+\; O(m),$$

and $h(\alpha) = \alpha(1 - \alpha)/(1 + \alpha)$ has $h'(\alpha) = (1 - 2\alpha - \alpha^2)/(1 + \alpha)^2$, vanishing at $\alpha = \sqrt{2} - 1$, where $h = 3 - 2\sqrt{2}$. Since $3 - 2\sqrt{2} > \frac{1}{8}$, this beats Theorem A.35 by the stated factor. □

In the best choice, one side has about $0.41m$ vertices and the other has $m - 1$. So $3 - 2\sqrt{2} - o(1) \leq K_m(n)/(m^2 n) < 2$ for $m \to \infty$ and $n/m \to \infty$. Whether this ratio has a limit is open.

A.8.2 The directed problem

The splitting of κ into parallel arcs and longer routes also works for directed graphs (Lemma A.40). Corollary A.41 below gives the exact value at $m = 2$ and the leading term for every fixed m.

Lemma A.40 (Splitting the directed vertex measure)**.** *Let D be a directed multigraph with multiplicity function μ and underlying simple digraph D_0. For an ordered pair $u \neq v$, write $\pi(u, v)$ for the largest number of pairwise internally disjoint directed u–v paths of length at least two in D_0. Then $\kappa_D(u, v) = \mu(u, v) + \pi(u, v)$.*

Proof. The proof of Lemma A.31 works word for word with directed paths. A route of length one is a copy of the arc $u \to v$ and has no inner vertex. Longer routes give internally disjoint directed paths in D_0, and conversely, using one copy of each arc. Copies of the reverse arc $v \to u$ are never used.

□

Corollary A.41 (The directed multigraph incidence value)**.** *For every* $n \geq 2$*,*

$$K_2^{\mathrm{dir}}(n) \;=\; k_2^{\mathrm{dir}}(n) \;=\; \ell_2^{\mathrm{dir}}(n) \;=\; \max\Bigl(2(n-1),\; \Bigl\lfloor \frac{n^2}{4} \Bigr\rfloor\Bigr),$$

and for each fixed $m \geq 2$*,*

$$(m-1)M(n) \;\leq\; K_m^{\mathrm{dir}}(n) \;\leq\; (m-1)\Bigl\lfloor \frac{n^2}{4} \Bigr\rfloor + 4(m-1)^2(n-1),$$

so that $K_m^{\mathrm{dir}}(n) = (m-1)n^2/4 + O_m(n)$ *and the bipartite digraph of Theorem 1.18 is asymptotically optimal for this variant too.*

Proof. At $m = 2$, two parallel arcs would themselves give two internally disjoint routes, so every feasible directed multigraph is simple. The exact value therefore follows from Theorems 1.11 and 1.12. For general m, Theorem 1.18 and $\kappa \leq \lambda$ give the lower bound, while Theorem 1.22 at $r = 2$ gives the upper bound. Since $M(n) = \lfloor n^2/4 \rfloor$ for $n \geq 7$, these bounds imply the stated asymptotic. □

At $n = 3$ and $m = 6$, the thickened complete digraph of Construction A.42 has 24 arcs, more than $(m-1)M(3) = 20$. It is feasible, because every route of length at least two passes through the third vertex. So for the vertex problem the arc construction is not optimal.

Construction A.42 (Thickened complete digraph, and its bouquet)**.** For $2 \leq b \leq m$, take the complete digraph on b vertices and give every arc multiplicity $m + 1 - b$. At $b = 2$ this is one bidirected edge at multiplicity $m - 1$. Hanging k such blocks off one shared vertex, pairwise disjoint otherwise, uses $k(b-1)+1$ vertices, and blocks of different sizes may be mixed.

Proposition A.43 (Gluing complete digraphs)**.** *Glue blocks of Construction A.42 with sizes* b_i *at one common vertex, keep them disjoint otherwise, and add isolated vertices to reach* n*. The result is feasible, so*

$$K_m^{\mathrm{dir}}(n) \;\geq\; \max\Bigl\{ \sum_i b_i(b_i-1)(m+1-b_i) \;:\; 2 \leq b_i \leq m,\; \sum_i (b_i - 1) \leq n-1 \Bigr\}.$$

Proof. Take $u \neq v$ in one block of size b. A route that leaves the block must come back through the common vertex, which a path visits only once. So every u–v route stays in the block. Longer routes inside the block have disjoint sets of inner vertices, taken from the other $b-2$ vertices, and the $b-2$ routes of length two reach this. So $\pi(u,v) = b-2$, and Lemma A.40 gives $\kappa(u,v) = (m+1-b) + (b-2) = m-1$. Two vertices in different blocks are separated by the common vertex, so $\kappa \leq 1$. A block of size b has $b(b-1)$ ordered pairs, each with $m+1-b$ arcs. □

For small n the block construction is larger, and for large n the bipartite construction is larger. At $m = 3$ the best block is a bidirected edge, so the bouquet is the thickened bidirected tree,

and the bipartite construction is larger from $n = 8$. At $m = 6$ the best blocks have three or four vertices. The bouquet has 84 arcs at $n = 8$, against 80, and the bipartite construction is larger from $n = 9$.

A.9 The directed hypergraph

The last four variants use directed hyperedges, in the forward, backward or general model of Section 1.3. The first result gives an upper and a lower bound that differ by a factor of four. Theorem 1.22 then closes this gap.

Proposition A.44 (Directed hypergraphs: first bounds)**.** *Let $n \geq r \geq 2$ and $m \geq 2$, and take the forward model. If every ordered pair of vertices has at most $m-1$ hyperedge-disjoint directed Berge routes, then*

$$|E(\mathcal{H})| \;\leq\; \frac{(m-1)\,n(n-1)}{r-1},$$

and, for every $1 \leq \alpha \leq n-r+1$, there is a feasible directed multihypergraph with

$$\alpha \left\lfloor \frac{(m-1)(n-\alpha)}{r-1} \right\rfloor$$

hyperedges. This construction is simple when $m-1 \leq \binom{n-\alpha-1}{r-2}$. For fixed r and m and large n we can take $\alpha = \lfloor n/2 \rfloor$, so the maximum is $\Theta_{r,m}(n^2)$.

Proof. Upper bound. A hyperedge with tail u and heads $h_1, \ldots, h_{r-1}$ joins the $r-1$ ordered pairs (u, h_i). If m hyperedges joined the same pair (u, v), they would give m disjoint routes of length one. So each of the $n(n-1)$ ordered pairs is joined by at most $m-1$ hyperedges. Counting the pairs (hyperedge, joined pair) in two ways gives

$$(r-1)|E| \leq (m-1)\,n(n-1).$$

Lower bound. Split V into a set A of α tails and a set B of $\beta := n - \alpha \geq r-1$ heads. Let $k := r-1$, $d := m-1$ and $N := \left\lfloor \frac{d\beta}{k} \right\rfloor$. Number the vertices of B as $0, 1, \ldots, \beta-1$, and read the repeating sequence

$$0, 1, \ldots, \beta-1, 0, 1, \ldots$$

in consecutive blocks of k numbers. The first N blocks are the hyperedges of a k-uniform multihypergraph $\mathcal{G}^*$ on B. Since $k \leq \beta$, each block has k different vertices. The blocks use Nk consecutive positions of the sequence, so every vertex appears $\lfloor Nk/\beta \rfloor$ or $\lceil Nk/\beta \rceil$ times. Since $Nk \leq d\beta$, every vertex has degree at most d.

Now give every tail $a \in A$ the hyperedges (a, H) for $H \in \mathcal{G}^*$. No vertex of B is a tail, so every route has length one. So for $a \in A$ and $w \in B$ we get $\lambda(a, w) = \deg_{\mathcal{G}^*}(w) \leq m-1$, and there are no other routes. The number of hyperedges is $\alpha N = \alpha \left\lfloor \frac{(m-1)(n-\alpha)}{r-1} \right\rfloor$. If $d \leq \binom{\beta-1}{k-1}$, we can use Lemma A.1 instead, which gives different hyperedges, so the directed hypergraph is simple. □

Construction A.45 (Bounded outgoing hyperedge count)**.** Let $n \geq r \geq 2$ and $m \geq 2$, and let $d = \min\{m-1, \binom{n-1}{r-1}\}$. Give every vertex d different outgoing forward hyperedges, with any d different sets of $r-1$ heads. This gives a simple directed hypergraph with nd hyperedges, feasible for both separations. If repeated hyperedges are allowed, give every vertex $m-1$ copies of one outgoing hyperedge, for $n(m-1)$ hyperedges in total.

Proof. Every directed route from u starts with a hyperedge whose tail is u. So removing the outgoing hyperedges of u separates u from every other vertex. There are at most $m-1$ of them, so $\lambda(u,v) \leq m-1$ for every ordered pair, and so also $\kappa(u,v) \leq m-1$. Hyperedges with different tails are different, so the hypergraph is simple when the head sets at each vertex are different. □

For small n this construction can beat the two-layer construction. For example, at $n = m = 6$ and $r = 3$ it gives 30 hyperedges, even in the simple model.

We write a directed hyperedge as (T, H), with tail set T and head set H. For $r \geq 4$ the general model has splits with both parts of size at least two. We show that the backward model is the forward model reversed, and that the general model has the same leading term as the forward model.

Lemma A.46 (Backward is the reverse of forward)**.** *Reversing every hyperedge, $(T,H) \mapsto (H,T)$, turns forward hypergraphs into backward ones and keeps the number of hyperedges. The reversed hypergraph $\mathcal{H}^R$ satisfies $\lambda_{\mathcal{H}}(u,v) = \lambda_{\mathcal{H}^R}(v,u)$ and $\kappa_{\mathcal{H}}(u,v) = \kappa_{\mathcal{H}^R}(v,u)$. So the backward model has the same extremal values as the forward model.*

Proof. Reversal turns every directed u–v Berge path into a v–u path with the same hyperedges and the same inner vertices. So it keeps both kinds of disjointness. □

Proposition A.47 (The general model obeys the forward bound)**.** *Take the general model. If every ordered pair has at most $m-1$ hyperedge-disjoint, or at most $m-1$ internally vertex-disjoint, directed Berge routes, then*

$$|E(\mathcal{H})| \;\leq\; \frac{(m-1)\,n(n-1)}{r-1}.$$

The construction of Proposition A.44 also works here, so the maximum is $\Theta_{r,m}(n^2)$.

Proof. Fix an ordered pair (u,v). Each hyperedge (T,H) with $u \in T$ and $v \in H$ gives a route $u \to v$ of length one. Different such hyperedges give separated routes, so there are at most $m-1$ of them. A hyperedge (T,H) joins exactly $|T|\,|H|$ ordered pairs. Summing over all pairs gives $\sum_{(T,H)} |T|\,|H| \leq (m-1)\,n(n-1)$. Since $|T|+|H| = r$ and both are at least one, $|T|\,|H| \geq r-1$. So $(r-1)|E| \leq (m-1)n(n-1)$. □

So both models lie between two bounds that differ by a factor of 4. This fixes the order n^2, but not the leading constant. The rest of this section finds the constant for both models. Their exact values may still differ.

The proof uses the one-step shadow digraph. Step 1 bounds the hyperedges by the shadow arcs. Step 2 shows that the shadow satisfies a two-step budget. Step 3 applies Lemma A.18.

Definition A.48 (One-step shadow)**.** Let $\mathcal{H}$ be a forward directed r-uniform hypergraph on V. Its *shadow* is the simple digraph $R(\mathcal{H})$ on V with an arc (u, v) when some hyperedge of $\mathcal{H}$ has tail u and head v. Let $\operatorname{cod}(u, v)$ be the number of such hyperedges.

Proof of Theorem 1.22. Let $R = R(\mathcal{H})$. It has no loops, since the tail of a hyperedge is not one of its heads.

Step 1: hyperedges and shadow arcs. The $\operatorname{cod}(u, v)$ hyperedges with tail u and head v give that many routes of length one, which are separated. So $\operatorname{cod}(u, v) \le m - 1$. Every hyperedge gives exactly $r - 1$ arcs of R. So

$$(r-1)\,|E(\mathcal{H})| \;=\; \sum_{(u,v)\in A(R)} \operatorname{cod}(u,v) \;\le\; (m-1)\,|A(R)|.$$

Proposition A.44 used the bound $|A(R)| \le n(n-1)$ here, which loses the factor four. We now bound $|A(R)|$ better.

Step 2: the shadow is budgeted. Fix an ordered pair (u, v). Let T be the set of midpoints counted by $p_2(u, v)$, together with v if $(u, v) \in A(R)$. So $|T| = \mathbf{1}_{(u,v)\in A(R)} + p_2(u, v)$. Every $t \in T$ is a head of some hyperedge with tail u. Pick hyperedges with tail u one at a time. Each time, pick one that contains an element t of T not contained in an earlier pick, and assign it to that t. Stop when every element of T lies in a picked hyperedge. Each pick contains at most $r-1$ elements of T, so there are $k \ge |T|/(r-1)$ picks. The picks are different hyperedges, assigned to different elements of T.

Now turn the picks into routes. If a pick e is assigned to v, take the route $u \to v$ through e. If it is assigned to a midpoint x, take the route $u \to x \to v$: enter through e, and leave through a hyperedge with tail x and head v (one exists, since $(x, v) \in A(R)$). The leaving hyperedges have different tails, so they are different, and they differ from the entering ones, which have tail u. So the routes share no hyperedge. Their inner vertices are different midpoints, so they are also separated. So

$$\frac{\mathbf{1}_{(u,v)\in A(R)} + p_2(u,v)}{r-1} \;\le\; k \;\le\; \kappa_{\mathcal{H}}(u,v) \;\le\; m-1,$$

and R is $(r-1)(m-1)$-budgeted.

Step 3: conclusion. Lemma A.18 with $C = (r-1)(m-1)$ gives $|A(R)| \le \lfloor n^2/4 \rfloor + 4(r-1)(m-1)(n-1)$. With Step 1,

$$|E(\mathcal{H})| \;\le\; \frac{m-1}{r-1}\left[\left\lfloor \frac{n^2}{4} \right\rfloor + 4(r-1)(m-1)(n-1)\right] \;=\; \frac{m-1}{r-1}\left\lfloor \frac{n^2}{4} \right\rfloor + 4(m-1)^2(n-1).$$

For the lower bound, Proposition A.44 with $\alpha = \lfloor n/2 \rfloor$ gives $(m-1)n^2/(4(r-1)) - O_{m,r}(n)$ hyperedges. This construction is feasible for both separations. For fixed $r \ge 3$ and large n it is simple. So the upper and lower bounds have the same n^2 term. □

Two remarks. For small n, the bound of Proposition A.44 is smaller, roughly until $n \approx 16(m-1)(r-1)/3$. The theorem is only better asymptotically. At $r = 2$ the shadow is the digraph itself, and the theorem gives the quadratic term of Theorem 1.18 again.

The general model needs a different Step 2. The proof above used twice that a hyperedge has one tail: leaving hyperedges with different tails are different, and they differ from the entering hyperedges. With several tails this fails. Two midpoints can leave through the same hyperedge, and for $r \geq 4$ one hyperedge can be the entrance for one target and the exit for another. Instead we take a maximal family of routes and ask why no route can be added. Each unserved target is blocked by a hyperedge that is already used, and a used hyperedge has only r vertices.

Theorem A.49 (The general orientation model shares the constant)**.** *Let $r \geq 2$ and $m \geq 2$. In the general model, every directed r-uniform multihypergraph $\mathcal{H}$ on $n \geq 2$ vertices with $\kappa^{\max}_{\mathcal{H}} \leq m-1$ satisfies*

$$|E(\mathcal{H})| \;\leq\; \frac{m-1}{r-1}\left\lfloor \frac{n^2}{4} \right\rfloor \;+\; \frac{4(2r+1)(m-1)^2}{r-1}\,(n-1).$$

Since $\kappa \leq \lambda$, the same bound holds when $\lambda^{\max}_{\mathcal{H}} \leq m-1$. So for fixed r and m the general model has the same leading term $\big(1+o(1)\big)(m-1)n^2/\big(4(r-1)\big)$ as the forward model, under both separations.

Proof. Let R be the shadow in the general model: $(u,v) \in A(R)$ when some hyperedge has u among its tails and v among its heads. Let $\operatorname{cod}(u,v)$ be the number of such hyperedges. Tails and heads are disjoint, so R has no loops.

Step 1. As before, $\operatorname{cod}(u,v) \leq m-1$. A hyperedge (T,H) gives $|T|\,|H| \geq r-1$ arcs of R. So

$$(r-1)\,|E(\mathcal{H})| \;\leq \sum_{(T,H)\in E(\mathcal{H})} |T|\,|H| \;=\; \sum_{(u,v)\in A(R)} \operatorname{cod}(u,v) \;\leq\; (m-1)\,|A(R)|.$$

Step 2: the shadow is budgeted. Fix an ordered pair (u,v). Let $\mathcal{T}$ be the set of midpoints counted by $p_2(u,v)$, together with v if $(u,v) \in A(R)$. Call a hyperedge e an *entrance* for $t \in \mathcal{T}$ if u is a tail of e and t a head. Call e an *exit* for a midpoint t if t is a tail of e and v a head. Every $t \in \mathcal{T}$ has an entrance, and every midpoint has an exit. No hyperedge is both an entrance and an exit for the same t, since t cannot be both a tail and a head.

For each target we use one route. For v it is a route of length one through an entrance. For a midpoint t it is the route $u \to t \to v$ through an entrance and an exit. Build a family of such routes, for different targets and with all hyperedges different, adding routes one at a time until no route can be added. Let $\mathcal{F}$ be the result, with M routes, serving the set S of targets, and let U be the set of hyperedges it uses. Then $|U| \leq 2M$. The routes share no hyperedge, and their inner vertices are different, so $M \leq \kappa_{\mathcal{H}}(u,v) \leq m-1$.

Now take $t \in \mathcal{T} \setminus S$. If t had an entrance outside U and, for a midpoint, also an exit outside U, we could add a route for t. This contradicts maximality. So t is a tail or a head of some hyperedge

in U. So

$$\mathcal{T} \setminus S \;\subseteq\; \bigcup_{e \in U} \bigl(T_e \cup H_e\bigr), \qquad \text{and} \qquad |\mathcal{T}| \;\le\; M + \sum_{e \in U} \bigl(|T_e| + |H_e|\bigr) \;=\; M + r\,|U|.$$

With $|U| \le 2M$ this gives $|\mathcal{T}| \le (2r+1)M \le (2r+1)(m-1)$. So R is $(2r+1)(m-1)$-budgeted.

Step 3. Lemma A.18 with $C = (2r+1)(m-1)$ gives $|A(R)| \le \lfloor n^2/4 \rfloor + 4(2r+1)(m-1)(n-1)$, and Step 1 gives the bound. The construction of Proposition A.44 also works in the general model, so the n^2 terms agree. □

The constant $2r + 1$ only affects the linear term.

Appendix B

The Program

Chapter 2 prints the main parts of `program/erdos915.py`. This appendix prints the remaining parts, in the order they appear in the file, and the output of one full run.

B.1 The remaining parts of the program

The file starts with a short description and the imports.

```
"""Erdős Problem 915 in sixteen variants: measure connectivity, search for
dense feasible graphs, and write a certificate that can be checked by hand.

Run:  python erdos915.py     (prints one line of numbers per case)
"""
import itertools
import math
import random
import subprocess
import time
from dataclasses import dataclass
from enum import Enum
from multiprocessing import Pool

import networkx
#</header>


#<models>
@dataclass(frozen=True)
class Model:
    NAME: str
    IS_DIRECTED: bool
    ALLOWS_REPEATED_EDGES: bool
    EDGE_SIZE: int               # r: 2 for graphs, at least 3 otherwise
    TAIL_COUNTS: tuple = (1,)    # directed: the allowed numbers of tails


def ALL_MODELS(HYPEREDGE_SIZE=3):
    r = HYPEREDGE_SIZE
    splits = [("one tail", (1,)), ("one head", (r - 1,)),
              ("any split", tuple(range(1, r)))]
    return [
        Model("simple graph",             False, False, 2),
```

```
        Model("multigraph",                False, True,  2),
        Model("simple hypergraph",         False, False, r),
        Model("multihypergraph",           False, True,  r),
        Model("simple digraph",            True,  False, 2),
        Model("directed multigraph",       True,  True,  2),
    ] + [
        Model(f"{name}, {split}", True, repeated, r, tail_counts)
        for name, repeated in [("directed hypergraph", False),
                               ("directed multihypergraph", True)]
        for split, tail_counts in splits
    ]


class Separation(Enum):
    EDGE_DISJOINT = "edge"
    VERTEX_DISJOINT = "vertex"


@dataclass(frozen=True)
class Problem:
    MODEL: Model
    SEPARATION: Separation
    VERTEX_COUNT: int            # n
    THRESHOLD_M: int             # feasible: fewer than m disjoint routes
#</models>


#<hyperedges>
@dataclass(frozen=True, order=True)
class Hyperedge:
    TAILS: tuple                 # empty for an undirected hyperedge
    MEMBERS: tuple               # all its vertices, sorted

    def entry_vertices(self):
        return self.TAILS or self.MEMBERS

    def exit_vertices(self):
        return tuple(v for v in self.MEMBERS if v not in self.TAILS)


def possible_hyperedges(problem):
    model, n = problem.MODEL, problem.VERTEX_COUNT
    hyperedges = []
    for members in itertools.combinations(range(n), model.EDGE_SIZE):
        if not model.IS_DIRECTED:
            hyperedges.append(Hyperedge((), members))
            continue
        for tail_count in model.TAIL_COUNTS:
            hyperedges += [Hyperedge(tails, members) for tails
                           in itertools.combinations(members, tail_count)]
```

```
    return hyperedges


def largest_multiplicity(problem):
    if problem.MODEL.ALLOWS_REPEATED_EDGES:
        return problem.THRESHOLD_M - 1
    return 1
#</hyperedges>


#<pairs>
def vertex_pairs(problem):
    vertices = range(problem.VERTEX_COUNT)
    if problem.MODEL.IS_DIRECTED:
        return list(itertools.permutations(vertices, 2))
    return list(itertools.combinations(vertices, 2))


def edge_count(multiplicity):
    return sum(multiplicity.values())
#</pairs>


#<network>
def build_flow_network(problem, multiplicity):
    network = networkx.DiGraph()
    if problem.SEPARATION is Separation.VERTEX_DISJOINT:
        through_vertex = 1
    else:
        through_vertex = math.inf
    for v in range(problem.VERTEX_COUNT):
        network.add_edge(("in", v), ("out", v), capacity=through_vertex)
    for hyperedge, copies in multiplicity.items():
        if copies == 0:
            continue
        enter, leave = ("enter", hyperedge), ("exit", hyperedge)
        network.add_edge(enter, leave, capacity=copies)
        for v in hyperedge.entry_vertices():
            network.add_edge(("out", v), enter, capacity=math.inf)
        for v in hyperedge.exit_vertices():
            network.add_edge(leave, ("in", v), capacity=math.inf)
    return network
#</network>


#<measure>
def disjoint_routes(network, source, target, stop_at=None):
    return networkx.maximum_flow_value(
        network, ("out", source), ("in", target),
        flow_func=networkx.algorithms.flow.edmonds_karp, cutoff=stop_at)
```

```
136
137
138 def is_feasible(problem, multiplicity):
139     m = problem.THRESHOLD_M
140     network = build_flow_network(problem, multiplicity)
141     return all(disjoint_routes(network, u, v, stop_at=m) < m
142                for u, v in vertex_pairs(problem))
143 #</measure>
144
145
146 #<connectivities>
147 def all_connectivities(problem, multiplicity):
148     network = build_flow_network(problem, multiplicity)
149     return {(u, v): disjoint_routes(network, u, v)
150             for u, v in vertex_pairs(problem)}
151
152
153 class OutOfTime(Exception):
154     pass
155 #</connectivities>
156
157
158 #<exhaustive>
159 def exhaustive_search(problem, deadline):
160     hyperedges = possible_hyperedges(problem)
161     cap = largest_multiplicity(problem)
162     best = {"size": -1, "graph": {}}
163     multiplicity = {}
164
165     def decide(index, size):
166         if time.time() > deadline:
167             raise OutOfTime
168         undecided = len(hyperedges) - index
169         if size + undecided * cap <= best["size"]:
170             return                          # cannot beat the best
171         if index == len(hyperedges):
172             best["size"], best["graph"] = size, dict(multiplicity)
173             return
174         hyperedge = hyperedges[index]
175         for copies in range(1, cap + 1):
176             multiplicity[hyperedge] = copies
177             if not is_feasible(problem, multiplicity):
178                 break                       # more copies stay infeasible
179             decide(index + 1, size + copies)
180         multiplicity.pop(hyperedge, None)
181         decide(index + 1, size)
182
183     try:
184         decide(0, 0)
185         return best["size"], best["graph"], True
```

```
    except OutOfTime:
        return best["size"], best["graph"], False
#</exhaustive>


#<nauty>
def best_digraphs_on_support(problem, support_edges, deadline):
    m = problem.THRESHOLD_M
    choices = [(forward, backward) for forward in range(m)
               for backward in range(m) if forward + backward > 0]
    best = {"size": -1, "graphs": []}
    multiplicity = {}

    def choose(index, size):
        if time.time() > deadline:
            raise OutOfTime
        undecided = len(support_edges) - index
        if size + undecided * 2 * (m - 1) < best["size"]:
            return
        if index == len(support_edges):
            if size > best["size"]:
                best["size"], best["graphs"] = size, []
            best["graphs"].append(dict(multiplicity))
            return
        a, b = support_edges[index]
        a_to_b, b_to_a = Hyperedge((a,), (a, b)), Hyperedge((b,), (a, b))
        for forward, backward in choices:
            multiplicity[a_to_b], multiplicity[b_to_a] = forward, backward
            if is_feasible(problem, multiplicity):
                choose(index + 1, size + forward + backward)
        del multiplicity[a_to_b], multiplicity[b_to_a]

    try:
        choose(0, 0)
        return best["size"], best["graphs"], True
    except OutOfTime:
        return best["size"], best["graphs"], False
#</nauty>


#<supports>
def undirected_supports(vertex_count):
    """Streams the output of geng, one graph at a time."""
    geng = subprocess.Popen(["geng", "-q", str(vertex_count)],
                            stdout=subprocess.PIPE)
    try:
        for line in geng.stdout:
            yield networkx.from_graph6_bytes(line.strip())
    finally:
        geng.kill()
```

```
        geng.wait()


def canonical_form(vertex_count, multiplicity):
    arcs = [(e.TAILS[0], e.exit_vertices()[0], copies)
            for e, copies in multiplicity.items() if copies]
    return min(tuple(sorted((order[t], order[h], c) for t, h, c in arcs))
               for order in itertools.permutations(range(vertex_count)))


def generating_search(problem, deadline):
    best_size, extremal, finished = 0, [{}], True
    for support in undirected_supports(problem.VERTEX_COUNT):
        if time.time() > deadline:
            finished = False
            break
        size, graphs, done = best_digraphs_on_support(
            problem, sorted(support.edges()), deadline)
        finished = finished and done
        if size > best_size:
            best_size, extremal = size, []
        if size == best_size:
            extremal += graphs
    classes = None
    if finished:
        classes = len({canonical_form(problem.VERTEX_COUNT, graph)
                       for graph in extremal})
    return best_size, extremal[0], classes, finished
#</supports>


#<energy>
PENALTY_PER_EXCESS_ROUTE = 10


def energy(size, highest_connectivity, threshold_m):
    excess = max(0, highest_connectivity - (threshold_m - 1))
    return -size + PENALTY_PER_EXCESS_ROUTE * excess
#</energy>


#<neighbours>
def moves(problem, multiplicity):
    cap = largest_multiplicity(problem)
    for hyperedge in possible_hyperedges(problem):
        copies = multiplicity.get(hyperedge, 0)
        if copies < cap:
            yield hyperedge, +1
        if copies > 0:
            yield hyperedge, -1
```

```
286
287
288 def highest_after(problem, multiplicity, move, connectivity):
289     """Measures only near the cap: one copy more raises a value by at
290     most one, so only pairs already at m-1 need a check, and one copy
291     less never raises a value."""
292     cap = problem.THRESHOLD_M - 1
293     current_highest = max(connectivity.values(), default=0)
294     _, change = move
295     if change < 0 and current_highest <= cap:
296         return current_highest
297     if change > 0 and current_highest < cap:
298         return current_highest + 1
299     network = build_flow_network(problem, apply(multiplicity, move))
300     at_cap = [pair for pair, value in connectivity.items()
301               if value >= cap]
302     return max((disjoint_routes(network, u, v) for u, v in at_cap),
303                default=0)
304 #</neighbours>
305
306
307 #<apply>
308 def apply(multiplicity, move):
309     hyperedge, change = move
310     changed = dict(multiplicity)
311     changed[hyperedge] = changed.get(hyperedge, 0) + change
312     return changed
313 #</apply>
314
315
316 #<tabu>
317 TABU_TENURE = 7
318 STEPS_WITHOUT_PROGRESS_BEFORE_KICK = 30
319
320
321 def tabu_search(problem, deadline, seed):
322     generator = random.Random(seed)
323     m = problem.THRESHOLD_M
324     multiplicity, tabu_until, step = {}, {}, 0
325     connectivity = all_connectivities(problem, multiplicity)
326     best_size, best_graph, last_progress = 0, {}, 0
327     while time.time() < deadline:
328         step += 1
329         scored = []
330         for move in moves(problem, multiplicity):
331             highest = highest_after(problem, multiplicity, move,
332                                     connectivity)
333             size = edge_count(multiplicity) + move[1]
334             new_best = highest <= m - 1 and size > best_size
335             if tabu_until.get(move[0], 0) > step and not new_best:
```

```
                continue                        # tabu, unless a new best
            scored.append((energy(size, highest, m),
                           generator.random(), move))
        if not scored:
            break
        _, _, move = min(scored)
        multiplicity = apply(multiplicity, move)
        tabu_until[move[0]] = step + TABU_TENURE
        connectivity = all_connectivities(problem, multiplicity)
        size = edge_count(multiplicity)
        if max(connectivity.values(), default=0) <= m - 1 \
                and size > best_size:
            best_size, best_graph = size, multiplicity
            last_progress = step
        if step - last_progress > STEPS_WITHOUT_PROGRESS_BEFORE_KICK:
            kicked = generator.sample(list(multiplicity),
                                      min(3, len(multiplicity)))
            multiplicity = dict(multiplicity)
            for hyperedge in kicked:
                multiplicity[hyperedge] = 0
            connectivity = all_connectivities(problem, multiplicity)
            last_progress = step
    return best_size, best_graph
#</tabu>


#<annealing>
START_TEMPERATURE, COOLING_FACTOR = 2.0, 0.995


def simulated_annealing(problem, deadline, seed):
    generator = random.Random(seed)
    m = problem.THRESHOLD_M
    multiplicity, temperature = {}, START_TEMPERATURE
    connectivity = all_connectivities(problem, multiplicity)
    current = energy(0, 0, m)
    best_size, best_graph = 0, {}
    while time.time() < deadline:
        move = generator.choice(list(moves(problem, multiplicity)))
        highest = highest_after(problem, multiplicity, move,
                                connectivity)
        size = edge_count(multiplicity) + move[1]
        proposed = energy(size, highest, m)
        worse_by = proposed - current
        if worse_by <= 0 or \
                generator.random() < math.exp(-worse_by / temperature):
            multiplicity, current = apply(multiplicity, move), proposed
            connectivity = all_connectivities(problem, multiplicity)
            if highest <= m - 1 and size > best_size:
                best_size, best_graph = size, multiplicity
```

```
        temperature = max(temperature * COOLING_FACTOR, 0.01)
    return best_size, best_graph
#</annealing>


#<greedy>
def random_greedy_growth(problem, deadline, seed):
    generator = random.Random(seed)
    cap = largest_multiplicity(problem)
    best_size, best_graph = 0, {}
    while time.time() < deadline:
        copies = [hyperedge for hyperedge in possible_hyperedges(problem)
                  for _ in range(cap)]
        generator.shuffle(copies)
        multiplicity = {}
        for hyperedge in copies:
            if time.time() > deadline:
                break                       # the partial graph is feasible
            grown = apply(multiplicity, (hyperedge, +1))
            if is_feasible(problem, grown):
                multiplicity = grown
        if edge_count(multiplicity) > best_size:
            best_size, best_graph = edge_count(multiplicity), multiplicity
    return best_size, best_graph
#</greedy>


#<certificate>
def write_certificate(problem, multiplicity):
    network = build_flow_network(problem, multiplicity)
    certificate = {}
    for u, v in vertex_pairs(problem):
        _, (source_side, _) = networkx.minimum_cut(
            network, ("out", u), ("in", v))
        removed_vertices, removed_hyperedges = [], []
        for node in source_side:
            for neighbour in network.successors(node):
                if neighbour in source_side:
                    continue
                if node[0] == "in":
                    removed_vertices.append(node[1])
                else:
                    removed_hyperedges.append(node[1])
        certificate[(u, v)] = (removed_vertices, removed_hyperedges)
    return certificate
#</certificate>


#<checker>
def check_certificate(problem, multiplicity, certificate):
```

```
    """Independent of the code above: no flows, only following edges."""
    for u, v in vertex_pairs(problem):
        removed_vertices, removed_hyperedges = certificate[(u, v)]
        if problem.SEPARATION is Separation.EDGE_DISJOINT \
                and removed_vertices:
            return False
        if u in removed_vertices or v in removed_vertices:
            return False
        size = len(set(removed_vertices)) + sum(
            multiplicity[e] for e in set(removed_hyperedges))
        if size > problem.THRESHOLD_M - 1:
            return False
        reached, frontier = {u}, [u]
        while frontier:
            vertex = frontier.pop()
            for hyperedge, copies in multiplicity.items():
                if copies == 0 or hyperedge in removed_hyperedges:
                    continue
                if vertex not in hyperedge.entry_vertices():
                    continue
                for nxt in hyperedge.exit_vertices():
                    if nxt not in reached and nxt not in removed_vertices:
                        reached.add(nxt)
                        frontier.append(nxt)
        if v in reached:
            return False
    return True
#</checker>


#<driver>
SECONDS_PER_CASE = 30


def solve(problem):
    model = problem.MODEL
    deadline = time.time() + SECONDS_PER_CASE / 2
    if model.IS_DIRECTED and model.ALLOWS_REPEATED_EDGES \
            and model.EDGE_SIZE == 2:
        size, graph, _, exact = generating_search(problem, deadline)
    else:
        size, graph, exact = exhaustive_search(problem, deadline)
    if not exact:
        deadline = time.time() + SECONDS_PER_CASE / 2
        if model.EDGE_SIZE == 2:
            search = tabu_search
        else:
            search = random_greedy_growth
        found_size, found_graph = search(problem, deadline, seed=1)
        if found_size > size:
```

```
            size, graph = found_size, found_graph
    certificate = write_certificate(problem, graph)
    return size, exact, check_certificate(problem, graph, certificate)


#</driver>


#<main>
def print_table(model, separation, solved, largest_m):
    print(f"\n{model.NAME}, {separation.value} separation")
    print("n \\ m" + "".join(f"{m:>6}" for m in range(2, largest_m + 1)))
    rows = {}
    for problem, (size, exact, checked) in solved:
        mark = ("*" if exact else " ") if checked else "!"
        rows.setdefault(problem.VERTEX_COUNT, []).append(f"{size:>5}{mark}"
            ↪ )
    for n, cells in rows.items():
        print(f"{n:>5}" + "".join(cells), flush=True)


def main(largest_n=7, largest_m=7):
    problems = [Problem(model, separation, n, m)
                for model in ALL_MODELS() for separation in Separation
                for n in range(model.EDGE_SIZE, largest_n + 1)
                for m in range(2, largest_m + 1)]
    print("Edges of the densest feasible graph found, every one certified."
        ↪ )
    print("* the search finished, so the value is exact;",
          "! the certificate failed")
    with Pool() as pool:
        solved = zip(problems, pool.imap(solve, problems))
        for (model, separation), table in itertools.groupby(
                solved, key=lambda pair: (pair[0].MODEL, pair[0].SEPARATION
                    ↪ )):
            print_table(model, separation, table, largest_m)


if __name__ == "__main__":
    main()
#</main>
```

Listing B.1. The description and the imports.

Two helpers list the vertex pairs that must be checked, ordered in directed models, and count the edges of a graph.

```
def vertex_pairs(problem):
    vertices = range(problem.VERTEX_COUNT)
    if problem.MODEL.IS_DIRECTED:
        return list(itertools.permutations(vertices, 2))
```

```
    return list(itertools.combinations(vertices, 2))


def edge_count(multiplicity):
    return sum(multiplicity.values())
```

Listing B.2. The vertex pairs and the edge count.

The heuristic searches keep the number of disjoint routes of every pair. The exception `OutOfTime` stops a search at its time limit.

```
def all_connectivities(problem, multiplicity):
    network = build_flow_network(problem, multiplicity)
    return {(u, v): disjoint_routes(network, u, v)
            for u, v in vertex_pairs(problem)}


class OutOfTime(Exception):
    pass
```

Listing B.3. The connectivity of every pair, and the time limit.

The generating search of Listing 2.6 reads the supports from `geng`, keeps the densest digraphs over all supports, and counts them up to renaming the vertices when it finishes.

```
def undirected_supports(vertex_count):
    """Streams the output of geng, one graph at a time."""
    geng = subprocess.Popen(["geng", "-q", str(vertex_count)],
                            stdout=subprocess.PIPE)
    try:
        for line in geng.stdout:
            yield networkx.from_graph6_bytes(line.strip())
    finally:
        geng.kill()
        geng.wait()


def canonical_form(vertex_count, multiplicity):
    arcs = [(e.TAILS[0], e.exit_vertices()[0], copies)
            for e, copies in multiplicity.items() if copies]
    return min(tuple(sorted((order[t], order[h], c) for t, h, c in arcs))
               for order in itertools.permutations(range(vertex_count)))


def generating_search(problem, deadline):
    best_size, extremal, finished = 0, [{}], True
    for support in undirected_supports(problem.VERTEX_COUNT):
        if time.time() > deadline:
            finished = False
            break
        size, graphs, done = best_digraphs_on_support(
            problem, sorted(support.edges()), deadline)
```

```
        finished = finished and done
        if size > best_size:
            best_size, extremal = size, []
        if size == best_size:
            extremal += graphs
    classes = None
    if finished:
        classes = len({canonical_form(problem.VERTEX_COUNT, graph)
                       for graph in extremal})
    return best_size, extremal[0], classes, finished
```

Listing B.4. Reading the supports from geng, and the generating search over all supports.

A move gives a new table, so the old one stays unchanged.

```
def apply(multiplicity, move):
    hyperedge, change = move
    changed = dict(multiplicity)
    changed[hyperedge] = changed.get(hyperedge, 0) + change
    return changed
```

Listing B.5. Making a move.

Simulated annealing, described in Chapter 2.

```
START_TEMPERATURE, COOLING_FACTOR = 2.0, 0.995


def simulated_annealing(problem, deadline, seed):
    generator = random.Random(seed)
    m = problem.THRESHOLD_M
    multiplicity, temperature = {}, START_TEMPERATURE
    connectivity = all_connectivities(problem, multiplicity)
    current = energy(0, 0, m)
    best_size, best_graph = 0, {}
    while time.time() < deadline:
        move = generator.choice(list(moves(problem, multiplicity)))
        highest = highest_after(problem, multiplicity, move,
                                connectivity)
        size = edge_count(multiplicity) + move[1]
        proposed = energy(size, highest, m)
        worse_by = proposed - current
        if worse_by <= 0 or \
                generator.random() < math.exp(-worse_by / temperature):
            multiplicity, current = apply(multiplicity, move), proposed
            connectivity = all_connectivities(problem, multiplicity)
            if highest <= m - 1 and size > best_size:
                best_size, best_graph = size, multiplicity
        temperature = max(temperature * COOLING_FACTOR, 0.01)
    return best_size, best_graph
```

Listing B.6. Simulated annealing on the same energy.

The main function runs every case in parallel and prints one table per problem.

```
495 def print_table(model, separation, solved, largest_m):
496     print(f"\n{model.NAME}, {separation.value} separation")
497     print("n \\ m" + "".join(f"{m:>6}" for m in range(2, largest_m + 1)))
498     rows = {}
499     for problem, (size, exact, checked) in solved:
500         mark = ("*" if exact else " ") if checked else "!"
501         rows.setdefault(problem.VERTEX_COUNT, []).append(f"{size:>5}{mark}"
            ↪ )
502     for n, cells in rows.items():
503         print(f"{n:>5}" + "".join(cells), flush=True)
504
505
506 def main(largest_n=7, largest_m=7):
507     problems = [Problem(model, separation, n, m)
508                 for model in ALL_MODELS() for separation in Separation
509                 for n in range(model.EDGE_SIZE, largest_n + 1)
510                 for m in range(2, largest_m + 1)]
511     print("Edges of the densest feasible graph found, every one certified."
        ↪ )
512     print("* the search finished, so the value is exact;",
513           "! the certificate failed")
514     with Pool() as pool:
515         solved = zip(problems, pool.imap(solve, problems))
516         for (model, separation), table in itertools.groupby(
517                 solved, key=lambda pair: (pair[0].MODEL, pair[0].SEPARATION
                    ↪ )):
518             print_table(model, separation, table, largest_m)
519
520
521 if __name__ == "__main__":
522     main()
```

Listing B.7. Running every case and printing the tables.

B.2 Output

Listing B.8 is the output of the program, unchanged. The command `python erdos915.py > output.txt` in the folder `program` writes it to a file, and this thesis reads that file directly, so no step lies between the program and the page. Each case had thirty seconds, on a computer with ten processor cores. Every entry is the number of edges, arcs or hyperedges of the densest feasible graph found, and every one of these graphs passed its certificate. An entry marked $*$ comes from a finished search, so it is the exact maximum. An unmarked entry is a lower bound only, and a longer run may raise it. For hypergraphs the rows start at $n = 3$. Each directed hypergraph problem has three tables, one for each version of Section 1.3. Wherever both searches finished, the tables with one tail and with one head agree, as Lemma A.46 predicts. In the general model the search finished almost only at $n = 3$. Its other entries are lower bounds,

and some lie below the one-tail entries, although every one-tail hypergraph also belongs to the general model.

```
Edges of the densest feasible graph found, every one certified.
* the search finished, so the value is exact; ! the certificate failed

simple graph, edge separation
n \ m     2     3     4     5     6     7
    2    1*    1*    1*    1*    1*    1*
    3    2*    3*    3*    3*    3*    3*
    4    3*    4*    6*    6*    6*    6*
    5    4*    6*    8*   10*   10*   10*
    6    5*    7    10*   12*   15*   15*
    7    6     9    12    15    18*   21*

simple graph, vertex separation
n \ m     2     3     4     5     6     7
    2    1*    1*    1*    1*    1*    1*
    3    2*    3*    3*    3*    3*    3*
    4    3*    4*    6*    6*    6*    6*
    5    4*    6*    8*   10*   10*   10*
    6    5*    7    10*   12*   15*   15*
    7    6     9    12    15    18*   21*

multigraph, edge separation
n \ m     2     3     4     5     6     7
    2    1*    2*    3*    4*    5*    6*
    3    2*    4*    6*    8*   10*   12*
    4    3*    6*    9*   12*   15*   18*
    5    4*    8*   12    16    19    23
    6    5*   10    14    20    23    28
    7    6    12    17    23    26    35

multigraph, vertex separation
n \ m     2     3     4     5     6     7
    2    1*    2*    3*    4*    5*    6*
    3    2*    4*    6*    9*   12*   15*
    4    3*    6*    9*   14*   19*   24*
    5    4*    8*   12    19    25    33
    6    5*   10    15    24    33    42
    7    6    12    18    29    33    51

simple hypergraph, edge separation
n \ m     2     3     4     5     6     7
    3    1*    1*    1*    1*    1*    1*
    4    1*    3*    4*    4*    4*    4*
    5    2*    4*    6*    7*    8*   10*
    6    2*    5     7    10    11    13
    7    3     6     9    12    15    17

simple hypergraph, vertex separation
n \ m     2     3     4     5     6     7
    3    1*    1*    1*    1*    1*    1*
    4    1*    3*    4*    4*    4*    4*
    5    2*    4*    6*    7*    8*   10*
    6    2*    5     7    10    11    13
    7    3     6     9    12    15    17

multihypergraph, edge separation
n \ m     2     3     4     5     6     7
    3    1*    2*    3*    4*    5*    6*
    4    1*    3*    4*    6*    7*    9*
    5    2*    4*    6*    8    10    11
    6    2*    5     7    10    12    14
    7    3     6     9    12    15    17

multihypergraph, vertex separation
n \ m     2     3     4     5     6     7
    3    1*    2*    3*    4*    5*    6*
    4    1*    3*    4*    6*    7*    9*
```

```
    5    2*    4*    6*    8    10    12
    6    2*    5     7     10   12    14
    7    3     6     9     12   15    17

simple digraph, edge separation
n \ m     2     3     4     5     6     7
    2    2*    2*    2*    2*    2*    2*
    3    4*    6*    6*    6*    6*    6*
    4    6*    9*   12*   12*   12*   12*
    5    8    12    16*   20*   20*   20*
    6   10    15    20    25    30*   30*
    7   12    18    24    30    36    42*

simple digraph, vertex separation
n \ m     2     3     4     5     6     7
    2    2*    2*    2*    2*    2*    2*
    3    4*    6*    6*    6*    6*    6*
    4    6*    9*   12*   12*   12*   12*
    5    8    12    16*   20*   20*   20*
    6   10    15    20    25    30*   30*
    7   12    18    24    30    36    42*

directed multigraph, edge separation
n \ m     2     3     4     5     6     7
    2    2*    4*    6*    8*   10*   12*
    3    4*    8*   12*   16*   20    24
    4    6*   12    18    24    30    36
    5    8*   16    24    32    37    43
    6   10    20    28    37    43    40
    7   12    20    33    33    42    49

directed multigraph, vertex separation
n \ m     2     3     4     5     6     7
    2    2*    4*    6*    8*   10*   12*
    3    4*    8*   12*   18*   24*   30
    4    6*   12    18    28    38    48
    5    8*   16    24    36    49    66
    6   10    20    29    42    57    81
    7   12    21    36    46    73    76

directed hypergraph, one tail, edge separation
n \ m     2     3     4     5     6     7
    3    3*    3*    3*    3*    3*    3*
    4    4*    8*   12*   12*   12*   12*
    5    6    10    15    20    25    30*
    6    7    12    18    24    30    36
    7    8    14    21    28    35    42

directed hypergraph, one tail, vertex separation
n \ m     2     3     4     5     6     7
    3    3*    3*    3*    3*    3*    3*
    4    4*    8*   12*   12*   12*   12*
    5    6    10    15    20    25    30*
    6    7    12    18    24    30    36
    7    8    14    21    28    35    42

directed hypergraph, one head, edge separation
n \ m     2     3     4     5     6     7
    3    3*    3*    3*    3*    3*    3*
    4    4*    8*   12*   12*   12*   12*
    5    6    10    15    20    25    30*
    6    7    12    18    24    30    36
    7    8    14    21    28    35    42

directed hypergraph, one head, vertex separation
n \ m     2     3     4     5     6     7
    3    3*    3*    3*    3*    3*    3*
    4    4*    8*   12*   12*   12*   12*
    5    6    10    15    20    25    30*
    6    7    12    18    24    30    36
```

```
    7    8    14    21    28    35    42

directed hypergraph, any split, edge separation
n \ m     2     3     4     5     6     7
    3    3*    4*    6*    6*    6*    6*
    4    4*    8    11    14    16    18
    5    6    10    14    18    21    24
    6    7    12    17    21    25    30
    7    8    13    18    24    29    35

directed hypergraph, any split, vertex separation
n \ m     2     3     4     5     6     7
    3    3*    4*    6*    6*    6*    6*
    4    4*    8    11    14    18    24*
    5    6    10    14    18    23    31
    6    7    12    17    21    27    36
    7    8    13    18    24    31    41

directed multihypergraph, one tail, edge separation
n \ m     2     3     4     5     6     7
    3    3*    6*    9*   12*   15*   18*
    4    4*    8    12    16    20    24
    5    6    11    15    20    25    30
    6    7    12    18    24    30    36
    7    8    14    21    28    35    42

directed multihypergraph, one tail, vertex separation
n \ m     2     3     4     5     6     7
    3    3*    6*    9*   12*   15*   18*
    4    4*    8    12    16    20    24
    5    6    11    15    20    25    31
    6    7    13    18    24    30    37
    7    8    14    21    28    35    42

directed multihypergraph, one head, edge separation
n \ m     2     3     4     5     6     7
    3    3*    6*    9*   12*   15*   18*
    4    4*    8    12    16    20    24
    5    6    11    16    20    25    30
    6    7    13    18    24    30    36
    7    8    14    21    28    35    42

directed multihypergraph, one head, vertex separation
n \ m     2     3     4     5     6     7
    3    3*    6*    9*   12*   15*   18*
    4    4*    8*   12    16    21    24
    5    6    11    16    20    26    32
    6    7    13    19    25    30    37
    7    8    14    21    28    35    42

directed multihypergraph, any split, edge separation
n \ m     2     3     4     5     6     7
    3    3*    6*    9*   12*   15*   18*
    4    4*    8    12    15    17    20
    5    6    10    14    17    22    24
    6    7    11    15    21    24    30
    7    8    14    18    24    29    35

directed multihypergraph, any split, vertex separation
n \ m     2     3     4     5     6     7
    3    3*    6*    9*   12*   15*   18*
    4    4*    8    12    15    19    24
    5    6    10    14    18    24    30
    6    7    11    16    21    26    35
    7    8    14    18    24    33    41
```

Listing B.8. The output of the program for $2 \leq m \leq 7$ and $n \leq 7$.

Bibliography

[1] Louis Vandenbruwaene. Maximising Edge Density in Graphs Subject to Connectivity Constraints: Erdős Problem 915, from Proof to Search. https://github.com/louisvandenbruwaene/thesis, 2026. GitHub repository, accessed 28 August 2026.

[2] Béla Bollobás and Pál Erdős. Extremal problems in graph theory. *Matematikai Lapok*, 13:143–152, 1962.

[3] Thomas F. Bloom. Erdős problem #915. https://www.erdosproblems.com/915. Accessed 28 August 2026.

[4] Reinhard Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, 5th edition, 2017.

[5] Karl Menger. Zur allgemeinen Kurventheorie. *Fundamenta Mathematicae*, 10:96–115, 1927.

[6] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956.

[7] Wolfgang Mader. Ein Extremalproblem des Zusammenhangs von Graphen. *Mathematische Zeitschrift*, 131:223–231, 1973.

[8] Ralph E. Gomory and Tien Chung Hu. Multi-terminal network flows. *Journal of the Society for Industrial and Applied Mathematics*, 9(4):551–570, 1961.

[9] Pál Bártfai. Schweitzer verseny megoldás (Erdős gráffeladatára). *Matematikai Lapok*, 11:175–176, 1960. In Hungarian. Solves Erdős's graph problem for the $m = 3$ vertex-disjoint case.

[10] Béla Bollobás. On graphs with at most three independent paths connecting any two vertices. *Studia Scientiarum Mathematicarum Hungarica*, 1:137–140, 1966.

[11] John L. Leonard. On graphs with at most four line-disjoint paths connecting any two vertices. *Journal of Combinatorial Theory, Series B*, 13:242–250, 1972.

[12] B. A. Sørensen and Carsten Thomassen. On k-rails in graphs. *Journal of Combinatorial Theory, Series B*, 17(2):143–159, 1974.

[13] Zejun Huang and Zhenhua Lyu. Extremal digraphs containing at most t paths of length 2 with the same endpoints. *Discrete Applied Mathematics*, 388:1–10, 2026. Turán number for $t + 1$ two-step paths sharing endpoints.

[14] Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, II. *Journal of Symbolic Computation*, 60:94–112, 2014. Describes the nauty/Traces canonical-labelling tools underlying geng.

[15] Fred Glover. Tabu search—part I. *ORSA Journal on Computing*, 1(3):190–206, 1989.

[16] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[17] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

[18] M. Fekete. Über die Verteilung der Wurzeln bei gewissen algebraischen Gleichungen mit ganzzahligen Koeffizienten. *Mathematische Zeitschrift*, 17:228–249, 1923. The subadditive lemma, used here in its superadditive form.

[19] Wolfgang Mader. Existenz n-fach zusammenhängender Teilgraphen in Graphen genügend großer Kantendichte. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 37:86–97, 1972.

[20] G. A. Dirac. A property of 4-chromatic graphs and some remarks on critical graphs. *Journal of the London Mathematical Society*, 27:85–92, 1952.

[21] W. Mantel. Problem 28. *Wiskundige Opgaven*, 10:60–61, 1907.

[22] Pál Turán. On an extremal problem in graph theory. *Matematikai és Fizikai Lapok*, 48:436–452, 1941.

[23] Chandra Chekuri and Chao Xu. Computing minimum cuts in hypergraphs. In *Proceedings of the 28th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1085–1100, 2017.

[24] Megan Dewar, David Pike, and John Proos. Connectivity in hypergraphs. *Canadian Mathematical Bulletin*, 61(2):252–271, 2018.

DEPARTMENT OF MATHEMATICS
Celestijnenlaan 200B
3001 Leuven, Belgium
wis.kuleuven.be

